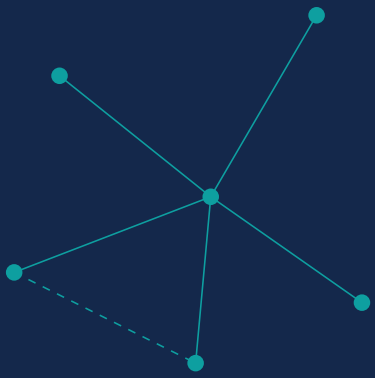


NETZWERK VERSTEHEN

Band 4

NETZWERKPROTOKOLLE IN DER PRAXIS

DNS, DHCP, HTTP, HTTPS, SSH und weitere wichtige Netzwerkdienste verstehen

Eugen Becker

Netzwerk verstehen, Band 4
Grundlagen der Netzwerktechnik

Autor: Eugen Becker

Erste Ausgabe

Dieses Buch ist urheberrechtlich geschützt. Die Verwendung der Inhalte außerhalb der gesetzlichen Grenzen bedarf der Zustimmung des Autors.

Die Angaben in diesem Buch wurden sorgfältig geprüft. Netzwerkgeräte, Betriebssysteme und Normen entwickeln sich weiter. Prüfe bei Konfigurationsfragen immer die Dokumentation deines Herstellers oder Betriebssystems.

Inhaltsverzeichnis

Vorwort **1**

1 Einführung in Netzwerkprotokolle **2**

1.1 Was ist ein Netzwerkprotokoll? 2
1.2 Warum benötigen Computer Protokolle? 2
1.3 Protokollregeln und Standards 2
1.4 Die Anwendungsschicht 3
1.5 Zusammenspiel mit TCP und UDP 3
1.6 Client-Server-Kommunikation 3
1.7 Request und Response 4
1.8 Ports und Dienste 4
1.9 Protokollstapel 4
1.10 Protokoll und Netzwerkdienst 5

2 DNS: Domain Name System **7**

2.1 Was ist DNS? 7
2.2 Warum wird DNS benötigt? 7
2.3 Domainnamen und IP-Adressen 7
2.4 DNS-Hierarchie 8
2.5 Root-Nameserver 8
2.6 TLD-Nameserver 9
2.7 Autoritative Nameserver 9
2.8 Rekursive Resolver 9
2.9 Rekursive und iterative Auflösung 9
2.10 DNS-Caching 10
2.11 TTL 10
2.12 Forward Lookup 11
2.13 Reverse Lookup 11
2.14 DNS-Zonen 11
2.15 Forward-Zonen und Reverse-Zonen 11
2.16 DNS-Records 11
2.17 DNS über UDP und TCP 14

2.18 DNSSEC . . . 14
2.19 DNS over TLS und DNS over HTTPS . . . 14
2.20 Split DNS . . . 15
2.21 Häufige DNS-Fehler . . . 15

3 DHCP: Dynamic Host Configuration Protocol . . . 17
3.1 Was ist DHCP? . . . 17
3.2 Warum benötigen wir DHCP? . . . 17
3.3 Statische und dynamische Adressvergabe . . . 17
3.4 DHCP-Server und DHCP-Client . . . 18
3.5 DHCP-Scope . . . 18
3.6 DHCP-Lease . . . 18
3.7 DHCP-Reservierung . . . 18
3.8 DHCP-Optionen . . . 19
3.9 DHCP-Relay . . . 19
3.10 DHCPv4 und DHCPv6 im Vergleich . . . 19
3.11 Unterschied zwischen DHCPv6 und SLAAC . . . 20
3.12 Der DORA-Ablauf . . . 20
3.13 Lease Renewal und Rebinding . . . 21
3.14 DHCP-Lease-Lebenszyklus . . . 21
3.15 DHCP-Fehler . . . 22
3.16 APIPA . . . 22
3.17 DHCP-Sicherheit . . . 22
3.18 Rogue DHCP Server . . . 23
3.19 Praxis mit DHCP . . . 23

4 HTTP: Hypertext Transfer Protocol . . . 24
4.1 Was ist HTTP? . . . 24
4.2 Client und Webserver . . . 24
4.3 HTTP-Anfragen . . . 24
4.4 HTTP-Antworten . . . 25
4.5 URLs und URIs . . . 25
4.6 HTTP-Methoden . . . 26
4.7 HTTP-Statuscodes . . . 26
4.8 HTTP-Header . . . 27
4.9 Cookies . . . 28
4.10 Sessions . . . 28
4.11 Content-Type und Caching . . . 28
4.12 HTTP/1.1, HTTP/2 und HTTP/3 . . . 28
4.13 Sicherheitsaspekte bei HTTP . . . 29
4.14 HTTP im Alltag . . . 29
4.15 Untersuchung unter Windows und Linux . . . 29
4.16 Typische Fehler . . . 30

5 HTTPS und TLS . . . 31
5.1 Was ist HTTPS? . . . 31
5.2 Warum ist Verschlüsselung wichtig? . . . 31
5.3 Unterschied zwischen HTTP und HTTPS . . . 31
5.4 Transport Layer Security . . . 32
5.5 TLS 1.2 und TLS 1.3 . . . 32
5.6 Öffentliche und private Schlüssel . . . 33
5.7 Symmetrische und asymmetrische Kryptografie . . . 33
5.8 Zertifikate . . . 33
5.9 Certificate Authorities . . . 34
5.10 Public Key Infrastructure . . . 34
5.11 Zertifikatsketten . . . 34
5.12 TLS-Handshake . . . 35
5.13 Zertifikatsprüfung . . . 35
5.14 Perfect Forward Secrecy . . . 36
5.15 SNI . . . 36
5.16 ALPN . . . 36
5.17 Häufige Zertifikatsfehler . . . 36
5.18 Praxis mit HTTPS und TLS . . . 36

6 FTP, FTPS und SFTP . . . 38
6.1 Dateiübertragung über Netzwerke . . . 38
6.2 FTP . . . 38
6.3 Kontrollverbindung . . . 38
6.4 Datenverbindung . . . 39
6.5 Active Mode . . . 39
6.6 Passive Mode . . . 39
6.7 FTP-Authentifizierung . . . 39
6.8 Sicherheitsprobleme . . . 39
6.9 FTPS . . . 40
6.10 SFTP . . . 40
6.11 Unterschiede zwischen FTP, FTPS und SFTP . . . 41
6.12 Sichere Dateiübertragung . . . 41
6.13 Anwendungsfälle . . . 41
6.14 Praxis unter Windows und Linux . . . 42
6.15 Typische Verbindungsfehler . . . 42

7 SSH: Secure Shell . . . 44
7.1 Was ist SSH? . . . 44
7.2 Remote Administration . . . 44
7.3 SSH-Client und SSH-Server . . . 44
7.4 Port 22 . . . 45
7.5 SSH-Protokollaufbau . . . 45

7.6 Host Keys . . . 46
7.7 Public-Key-Authentifizierung . . . 46
7.8 Passwortauthentifizierung . . . 46
7.9 SSH-Key-Paare . . . 47
7.10 Known Hosts . . . 47
7.11 SSH-Agent . . . 47
7.12 SSH-Tunneling und Port Forwarding . . . 47
7.13 SCP und SFTP . . . 47
7.14 Sicherheitsmaßnahmen . . . 48
7.15 Praxis mit SSH . . . 48
7.16 Typische Fehler . . . 48

8 E-Mail-Protokolle . . . 50
8.1 Wie funktioniert E-Mail? . . . 50
8.2 SMTP . . . 50
8.3 Mailserver . . . 51
8.4 Mail User Agent . . . 51
8.5 Mail Transfer Agent . . . 51
8.6 Mail Delivery Agent . . . 51
8.7 MX-Records . . . 51
8.8 SMTP-Relay . . . 52
8.9 Submission . . . 52
8.10 POP3 und IMAP . . . 53
8.11 MIME . . . 53
8.12 STARTTLS . . . 54
8.13 SPF . . . 54
8.14 DKIM . . . 54
8.15 DMARC . . . 54
8.16 E-Mail-Zustellung . . . 54
8.17 Spam-Schutz . . . 55
8.18 Praxis mit E-Mail-Protokollen . . . 55

9 NTP: Network Time Protocol . . . 57
9.1 Warum ist Zeitsynchronisation wichtig? . . . 57
9.2 NTP . . . 57
9.3 UTC . . . 57
9.4 Stratum . . . 58
9.5 NTP-Hierarchie . . . 58
9.6 NTP-Server und NTP-Clients . . . 58
9.7 Zeitabweichung . . . 59
9.8 Zeitkorrektur . . . 59
9.9 NTP-Nachrichtenaustausch . . . 59
9.10 NTP-Sicherheit . . . 60

9.11 NTS als Einführung . . . 60
9.12 Zeitsynchronisation unter Windows und Linux . . . 60
9.13 Typische Fehler . . . 60

10 SNMP: Simple Network Management Protocol . . . 62
10.1 Netzwerküberwachung . . . 62
10.2 SNMP . . . 62
10.3 SNMP-Manager . . . 62
10.4 SNMP-Agent . . . 63
10.5 MIB . . . 63
10.6 OID . . . 63
10.7 GET, GETNEXT und GETBULK . . . 63
10.8 SET . . . 63
10.9 TRAP und INFORM . . . 64
10.10 SNMPv1 . . . 64
10.11 SNMPv2c . . . 65
10.12 SNMPv3 . . . 65
10.13 Authentifizierung . . . 65
10.14 Verschlüsselung . . . 66
10.15 Monitoring . . . 66
10.16 SNMP-Grundlagen unter Linux . . . 66
10.17 Sicherheitskonfiguration . . . 66
10.18 Typische Fehler . . . 66

11 SMB und Netzwerkfreigaben . . . 68
11.1 Was ist SMB? . . . 68
11.2 Datei- und Druckerfreigaben . . . 68
11.3 SMB-Client und SMB-Server . . . 68
11.4 Windows-Freigaben . . . 69
11.5 Samba unter Linux . . . 69
11.6 Authentifizierung . . . 69
11.7 Berechtigungen . . . 69
11.8 SMB-Versionen . . . 69
11.9 SMB-Signing . . . 70
11.10 SMB-Verschlüsselung . . . 70
11.11 Typische Sicherheitsrisiken . . . 70
11.12 Unterschied zwischen Freigabe- und Dateisystemberechtigungen . . . 71
11.13 Praxis mit SMB . . . 71

12 LDAP und Verzeichnisdienste . . . 72
12.1 Was ist LDAP? . . . 72
12.2 Verzeichnisdienste . . . 72
12.3 Distinguished Name . . . 72

12.4 Relative Distinguished Name . . . 73
12.5 Directory Information Tree . . . 73
12.6 LDAP-Einträge . . . 73
12.7 Attribute . . . 73
12.8 LDAP-Suche . . . 74
12.9 LDAP-Bind . . . 74
12.10 LDAP und TLS . . . 74
12.11 Active Directory als Anwendungsbeispiel . . . 75
12.12 LDAP und Kerberos unterscheiden . . . 75
12.13 Authentifizierung und Autorisierung . . . 75
12.14 Praxis mit LDAP . . . 75
12.15 Typische Fehler . . . 76

13 Weitere wichtige Netzwerkprotokolle . . . 77
13.1 Telnet . . . 77
13.2 TFTP . . . 77
13.3 RDP . . . 77
13.4 SIP . . . 78
13.5 RTP . . . 78
13.6 RTCP . . . 78
13.7 MQTT . . . 78
13.8 WebSocket . . . 78
13.9 mDNS . . . 79
13.10 SSDP . . . 79
13.11 Vergleichstabelle . . . 79

14 Wie Netzwerkprotokolle zusammenarbeiten . . . 81
14.1 Beispiel 1: Ein Benutzer öffnet eine Webseite . . . 81
14.2 Beispiel 2: Ein Benutzer verschickt eine E-Mail . . . 81
14.3 Beispiel 3: Ein Administrator verbindet sich mit einem Linux-Server . . . 82
14.4 Beispiel 4: Ein neuer Computer wird mit dem Netzwerk verbunden . . . 82

15 Netzwerkanalyse mit Wireshark . . . 87
15.1 Protokolle erkennen . . . 87
15.2 DNS-Verkehr analysieren . . . 87
15.3 DHCP-DORA untersuchen . . . 87
15.4 HTTP-Anfragen analysieren . . . 88
15.5 TLS-Handshake erkennen . . . 88
15.6 SSH-Verbindungen untersuchen . . . 88
15.7 NTP-Verkehr analysieren . . . 88
15.8 Protokollfilter verwenden . . . 88
15.9 Fehler erkennen . . . 89

16 Praktische Laborübungen . . . 91

16.1 DNS . . . 91
16.2 DHCP . . . 93
16.3 HTTP und HTTPS . . . 94
16.4 SSH und SFTP . . . 96
16.5 Zeit . . . 98
16.6 SMB . . . 98
16.7 Vergleich . . . 99

17 Troubleshooting und Fehlersuche . . . 101
17.1 Systematisches Vorgehen . . . 101
17.2 DNS funktioniert nicht . . . 102
17.3 DHCP vergibt keine Adresse . . . 102
17.4 Webseite nicht erreichbar . . . 102
17.5 HTTPS-Zertifikatsfehler . . . 102
17.6 SSH-Verbindung verweigert . . . 103
17.7 E-Mail-Zustellung fehlgeschlagen . . . 103
17.8 SMB-Freigabe nicht erreichbar . . . 103
17.9 NTP synchronisiert nicht . . . 103
17.10 Firewall blockiert einen Dienst . . . 104
17.11 Falsche Portnummer . . . 104
17.12 Fehler-Ursache-Lösung-Übersicht . . . 104

18 Wissensüberprüfung und Prüfungsvorbereitung . . . 106
18.1 Zusammenfassung der Protokolle . . . 106
18.2 Multiple-Choice . . . 106
18.3 Zuordnungsaufgaben . . . 108
18.4 Praxisaufgaben . . . 109
18.5 Protokollanalyse und Fehlerdiagnose . . . 109
18.6 Vollständige Lösungen . . . 109

Anhang . . . 111

Vorwort

In den letzten Bänden hast du gelernt, wie Daten durch das Netz kommen. Du weißt, wie IP Pakete zwischen Netzen weiterleitet, wie TCP eine verlässliche Verbindung aufbaut und wie UDP schlank Datagramme verschickt. Dieser Band zeigt, was damit geschieht. Wenn du eine Webseite öffnest, einen Server verwaltest oder eine E-Mail schreibst, nutzt du keine nackten Pakete, sondern Protokolle, die auf TCP oder UDP aufsetzen und festlegen, was eine Nachricht bedeutet.

Netzwerkprotokolle sind Regelwerke. Sie bestimmen, welche Nachrichten es gibt, in welcher Reihenfolge sie folgen und wie der Empfänger sie zu deuten hat. Wer diese Regeln kennt, versteht nicht nur, warum eine Verbindung funktioniert, sondern auch, warum sie scheitert. Ein fehlender DNS-Eintrag, ein abgelaufenes Zertifikat oder eine falsch gesetzte Portweiterleitung sehen von außen oft gleich aus. Mit dem Wissen über das Protokoll lassen sie sich unterscheiden.

Die Kapitel dieses Bandes folgen einem festen Muster. Zuerst erklären wir, was ein Protokoll ist und welche Aufgabe es hat. Danach zeigen wir, wie es arbeitet, welche Ports es nutzt und wie eine typische Kommunikation aussieht. Sicherheit, Alltag, Untersuchung unter Windows und Linux sowie typische Fehler runden jedes Kapitel ab. Die Abbildungen zeigen den Ablauf der Nachrichten, denn ein Protokoll versteht man am besten, wenn man die Reihenfolge der Nachrichten vor sich sieht.

Wir beginnen mit den Grundlagen und wenden uns dann den Diensten zu, die im Alltag am häufigsten vorkommen. DNS übersetzt Namen in Adressen, DHCP vergibt sie an Geräte. Es folgen HTTP und HTTPS für das Web, FTP und SSH für Dateien und Fernzugriff, E-Mail, Zeitsynchronisation und Freigaben. Am Ende stehen eine Analyse mit Wireshark, Laborübungen und eine Zusammenfassung.

Ein Tipp zum Lernen: Wähle ein Protokoll und führe es auf deinem eigenen Rechner aus, bevor du das nächste Kapitel liest. Ein Befehl wie nslookup oder curl macht den Ablauf greifbar, und die Wireshark-Mitschnitte zeigen, was tatsächlich über das Netz geht. Die Übungen dafür finden sich in den Laborkapiteln.

Viel Erfolg beim Lesen, Eugen Becker

KAPITEL 1

Einführung in Netzwerkprotokolle

Wenn du zwei Menschen beobachtest, die sich begrüßen, folgen sie einem unausgesprochenen Ablauf. Einer sagt Hallo, der andere antwortet, dann folgt das eigentliche Gespräch. Niemand hat diesen Ablauf je schriftlich festgelegt, und doch funktioniert er. Computer brauchen etwas Ähnliches, nur explizit. Sie können nicht raten, was ein Gegenüber meint. Deshalb gibt es Netzwerkprotokolle, die jede Nachricht eindeutig machen.

1.1 Was ist ein Netzwerkprotokoll?

Ein Netzwerkprotokoll ist eine Vereinbarung darüber, wie zwei oder mehr Geräte miteinander kommunizieren. Es legt fest, welche Nachrichten es gibt, wie sie aufgebaut sind, in welcher Reihenfolge sie gesendet werden und was als Antwort gilt. Man kann es mit einer Sprache vergleichen, die beide Seiten beherrschen. Wer die Sprache nicht spricht, kann nicht antworten, auch wenn er die Wörter hört.

1.2 Warum benötigen Computer Protokolle?

Ohne Protokolle wüsste ein Empfänger nicht, wo eine Nachricht endet, welche Bytes Text und welche Steuerinformation sind oder ob eine Anfrage eine Antwort verlangt. Protokolle lösen diese Fragen. Sie machen aus einem Strom von Bits eine verständliche Nachricht. Außerdem sorgen sie dafür, dass Geräte verschiedener Hersteller zusammenarbeiten können, weil alle dieselben Regeln befolgen.

1.3 Protokollregeln und Standards

Die wichtigsten Protokolle des Internets sind in Standards festgelegt. Die Internet Engineering Task Force veröffentlicht diese Spezifikationen als RFC-Dokumente. Ein RFC ist kein Gesetz, sondern eine Empfehlung, die sich durch Verbreitung und Praxis durchsetzt. Ein Standard kann später durch einen neuen ersetzt werden. Deshalb ist es wichtig zu wissen, ob ein Dokument noch gilt oder veraltet ist. Mehrere Standards lassen sich auch

nebeneinander verwenden, zum Beispiel HTTP/1.1, HTTP/2 und HTTP/3.

1.4 Die Anwendungsschicht

Die Anwendungsschicht ist die oberste Schicht des TCP/IP-Modells. Hier sitzen die Protokolle, mit denen Programme arbeiten: HTTP für Webseiten, SMTP für E-Mails, DNS für Namen, DHCP für Adressen. Die Abbildung zeigt, wie diese Schicht auf den darunterliegenden Transport aufsetzt.



Abbildung 1.1. Das TCP/IP-Modell mit der Anwendungsschicht oben. Die Protokolle dieses Bandes gehören alle zur obersten Schicht und nutzen darunter TCP oder UDP. Schematische Darstellung.

1.5 Zusammenspiel mit TCP und UDP

Jedes Anwendungsprotokoll wählt einen Transport. Ein Webbrowser verlangt vollständige Daten und nutzt daher TCP. Ein DNS-Client stellt eine kurze Frage und nutzt UDP, weil eine Wiederholung schneller geht als ein Verbindungsaufbau. Das Anwendungsprotokoll selbst sagt, was in den Nutzdaten steht, der Transport sagt, wie sie ankommen. Beide arbeiten zusammen, und die Trennung ist der Grund, warum ein neues Anwendungsprotokoll nicht jedes Mal das Netz neu erfinden muss.

1.6 Client-Server-Kommunikation

Die meisten Netzwerkdienste folgen dem Client-Server-Modell, das wir in Band 1 kennengelernt haben. Der Server wartet an einem bekannten Port, der Client stellt eine Anfrage.

Die Rollen sind klar verteilt: Der Client beginnt, der Server antwortet. Diese Asymmetrie macht die Dienste berechenbar und ermöglicht es, einen Server zu betreiben, den viele Clients gleichzeitig nutzen.

1.7 Request und Response

Die Grundform fast aller Anwendungsprotokolle ist Anfrage und Antwort. Der Client sendet eine Request-Nachricht mit dem Wunsch, der Server antwortet mit einer Response-Nachricht, die das Ergebnis oder einen Fehler enthält. Die folgende Abbildung zeigt diesen Ablauf allgemein. Jedes Protokoll dieses Bandes folgt diesem Muster, mit eigenen Nachrichten und eigenen Regeln.

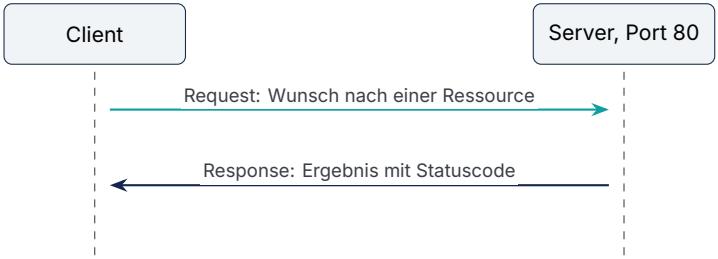


Abbildung 1.2. Anfrage und Antwort im Client-Server-Modell. Der Client sendet einen Request, der Server antwortet mit einer Response. Schematische Darstellung.

1.8 Ports und Dienste

Ein Server betreibt oft mehrere Dienste auf derselben IP-Adresse. Die Ports unterscheiden sie, wie wir in Band 3 gesehen haben. Der Port 80 bezeichnet HTTP, der Port 25 SMTP. Die Portnummer ist also der Hinweis, welches Protokoll gemeint ist. Ein Dienst ist dabei mehr als das Protokoll: Er ist das laufende Programm, das die Regeln umsetzt.

1.9 Protokollstapel

Ein Protokollstapel stellt alle Protokolle dar, die zusammen eine Nachricht transportieren. Beim Senden wird die Nachricht von oben nach unten verpackt, beim Empfangen von unten nach oben entpackt. Jede Schicht fügt ihren Kopfteil hinzu. Die folgende Abbildung

zeigt den Stapel für eine HTTP-Anfrage über TCP und IP.

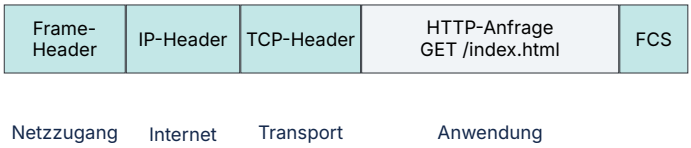


Abbildung 1.3. Der Protokollstapel einer HTTP-Anfrage. Die Anwendungsdaten stehen innen, jede Schicht umhüllt sie mit ihrem Kopfteil. Schematische Darstellung, nicht maßstabsgetreu.

1.10 Protokoll und Netzwerkdienst

Der Unterschied ist wichtig und wird oft verwechselt. Ein Protokoll ist die Regel, ein Netzwerkdienst ist das Angebot, das auf dieser Regel beruht. HTTP ist ein Protokoll, ein Webserver, der HTTP spricht, ist ein Dienst. Derselbe Dienst kann verschiedene Protokolle unterstützen, und dasselbe Protokoll kann von verschiedenen Diensten umgesetzt werden. Wenn ein Dienst ausfällt, liegt das Problem also selten am Protokoll selbst, sondern an der Umsetzung oder an der Konfiguration.

> **Merke**
>
> Ein Netzwerkprotokoll legt die Regeln einer Kommunikation fest. Anwendungsprotokolle wie DNS, DHCP oder HTTP laufen auf TCP oder UDP, folgen meist dem Muster Anfrage und Antwort und nutzen feste Ports.

Kontrollfragen

1. Nenne zwei Gründe, warum Computer Protokolle benötigen.
2. Warum wählt ein DNS-Client UDP, während ein Webbrowser TCP wählt?
3. Was ist der Unterschied zwischen einem Protokoll und einem Netzwerkdienst?
4. Welche Schicht des TCP/IP-Modells gehört zu den Anwendungsprotokollen?

Musterlösungen: 1. Protokolle legen fest, wo eine Nachricht endet und wie sie zu deuten ist, und sie ermöglichen die Zusammenarbeit von Geräten verschiedener Her-

steller. 2. Eine DNS-Anfrage ist kurz, und eine Wiederholung ist schneller als ein Verbindungsaufbau. Der Browser braucht vollständige Daten, deshalb nutzt er TCP. 3. Das Protokoll ist die Regel, der Dienst ist das laufende Angebot, das diese Regel umsetzt. 4. Die Anwendungsschicht.

KAPITEL 2

DNS: Domain Name System

Wenn du im Browser eine Adresse wie www.example.com eingibst, rechnet dein Computer nicht mit diesem Namen. Er braucht eine IP-Adresse, denn Pakete werden nur an Adressen zugestellt. Das Domain Name System, kurz DNS, übernimmt diese Übersetzung. Es ist eines der ältesten und wichtigsten Dienste des Internets. Ohne DNS müsste man sich Millionen Zahlenfolgen merken, und jede Änderung einer Adresse würde sofort alle Nutzer treffen.

Dieses Kapitel ist ausführlich, weil DNS in fast jedem Fehlerbild eine Rolle spielt. Wer die Auflösung versteht, kann viele Störungen in wenigen Schritten eingrenzen.

2.1 Was ist DNS?

DNS ist ein verteiltes Verzeichnis, das Namen auf Informationen abbildet. Am häufigsten liefert es IP-Adressen zu Domainnamen, aber es speichert auch Angaben zu Mailservern, Diensten und Sicherheitsregeln. Das Verzeichnis ist auf viele Server verteilt, sodass kein einzelner Betreiber alles verwaltet. Die Grundlagen sind in RFC 1034 und RFC 1035 festgelegt.

2.2 Warum wird DNS benötigt?

Menschen merken sich Namen besser als Zahlen. Außerdem sind Adressen veränderlich. Ein Dienst kann umziehen, ohne dass sich sein Name ändert, und der Betreiber passt nur den Eintrag im DNS an. Ein Name verbindet also Menschen und Dienste dauerhaft mit einer Adresse, die sich ändern darf. Viele Dienste nutzen DNS zusätzlich, um Lastverteilung oder Standortwahl umzusetzen.

2.3 Domainnamen und IP-Adressen

Ein Domainname besteht aus Teilen, die durch Punkte getrennt sind. Der rechte Teil ist die oberste Ebene, der nächste die Domain, der linke Teil ein Rechner oder Dienst. In www.example.com ist com die Top-Level-Domain, example die Domain und www der

Host. Die Teile werden von rechts nach links aufgelöst.

2.4 DNS-Hierarchie

DNS ist hierarchisch aufgebaut, ähnlich einem Verzeichnisbaum. An der Spitze stehen die Root-Server, darunter die Server der Top-Level-Domains wie com, de oder org, darunter die Server der einzelnen Domains. Die folgende Abbildung zeigt diesen Aufbau mit einem Beispiel.

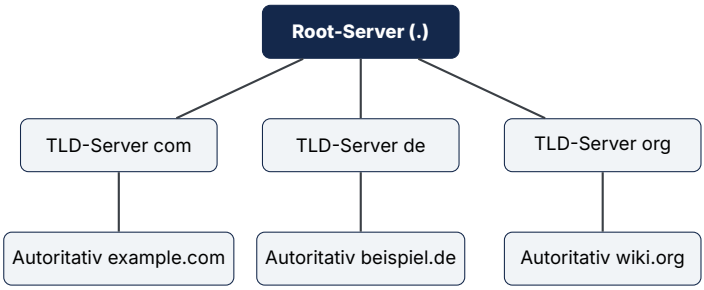


Abbildung 2.1. Hierarchie des DNS mit Root-Servern, Top-Level-Domain-Servern und autoritativen Servern. Die Beispieldomains sind Platzhalter. Schematische Darstellung.

Die Root-Server bilden die oberste Ebene, deren Namen mit einem Punkt bezeichnet werden. Es gibt dreizehn Root-Server-Identitäten, die von zwölf Organisationen betrieben werden. Jede Identität wird an vielen Standorten weltweit gespiegelt, was die Erreichbarkeit verbessert.

2.5 Root-Nameserver

Root-Nameserver kennen nicht die Adressen einzelner Domains. Sie wissen nur, welche Server für jede Top-Level-Domain zuständig sind. Eine Frage nach www.example.com beantwortet ein Root-Server also mit einem Verweis auf die Server der Domain com. Diese Aufgabe klingt bescheiden, sie ist aber der Ausgangspunkt jeder Auflösung.

2.6 TLD-Nameserver

TLD-Nameserver verwalten die Domains unterhalb einer Endung. Der Server für com kennt die autoritativen Server von example.com und verweist darauf. Die Betreiber der Endungen pflegen diese Verzeichnisse, und die Delegation ist der Mechanismus, durch den die Verantwortung weitergegeben wird.

2.7 Autoritative Nameserver

Ein autoritativer Nameserver ist der Server, der die endgültigen Angaben zu einer Domain kennt. Er gibt die Antwort für www.example.com als Adresse zurück, ohne weitere Verweise. Autoritativ bedeutet, dass der Server die Daten selbst verwaltet und nicht nur weiterleitet. Eine Domain hat in der Regel mehrere autoritative Server, damit der Ausfall eines Servers nicht die ganze Domain unerreichbar macht.

2.8 Rekursive Resolver

Der Rechner, mit dem dein Computer spricht, ist meist kein autoritativer Server, sondern ein rekursiver Resolver. Das ist der Dienst deines Internetanbieters, deines Routers oder ein öffentlicher Dienst. Er übernimmt die Arbeit der Auflösung für dich, fragt die anderen Server in der Kette und speichert die Ergebnisse. Dein Computer stellt also nur eine einzige Frage, der Resolver erledigt den Rest.

2.9 Rekursive und iterative Auflösung

Die Auflösung ist eine Kette von Anfragen. Der Client stellt eine rekursive Anfrage an den Resolver. Das bedeutet: Der Resolver soll die Antwort liefern, egal wie viele Schritte nötig sind. Der Resolver stellt dagegen iterative Anfragen an die anderen Server. Jeder Server antwortet mit dem besten Wissen, das er hat, meist einem Verweis. Der Resolver folgt den Verweisen, bis er die Antwort vom autoritativen Server hat.

Die folgende Abbildung zeigt die vollständige Auflösung von www.example.com durch einen Resolver, der noch keine Einträge gespeichert hat.

Der Vorteil der Trennung liegt in der Verteilung der Last. Kein Root-Server muss jede einzelne Domain kennen, und der Resolver kann Zwischenergebnisse für viele Anfragen wiederverwenden.

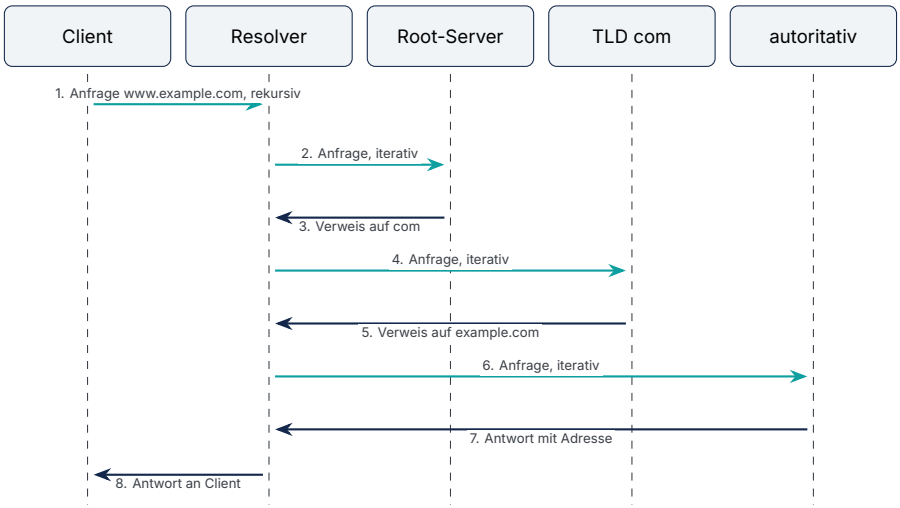


Abbildung 2.2. Vollständige Auflösung von www.example.com. Der Client fragt den Resolver rekursiv, der Resolver fragt die Server nacheinander iterativ. Die Antworten sind Verweise, bis der autoritative Server die Adresse liefert. Schematische Darstellung.

2.10 DNS-Caching

Ein Resolver speichert Antworten für eine gewisse Zeit, damit er nicht bei jeder Anfrage die ganze Kette durchlaufen muss. Der Cache macht DNS schnell, denn die meisten Auflösungen kommen aus dem Speicher. Auch der Client kann Einträge zwischenspeichern. Unter Windows zeigt ipconfig /displaydns den lokalen Cache an.

Der Cache hat einen Nachteil. Ändert sich ein Eintrag beim Betreiber, sehen Clients die alte Adresse so lange, bis der Cache abläuft. Diese Zeit steuert die TTL.

2.11 TTL

Die Time to Live, kurz TTL, ist eine Angabe in Sekunden, die jedem Eintrag beigegeben ist. Sie bestimmt, wie lange ein Resolver oder Client die Antwort speichern darf. Eine TTL von 3600 bedeutet eine Stunde. Ein kurzer Wert ermöglicht schnelle Umzüge, erhöht aber die Last auf den autoritativen Servern. Ein langer Wert entlastet die Server, verzögert aber Änderungen. Der Betreiber wählt den Wert bewusst.

2.12 Forward Lookup

Ein Forward Lookup fragt zu einem Namen die zugehörige Adresse ab. Das ist die häufigste Abfrage, die dein Computer stellt, wenn du eine Webseite öffnest. Der Name wird in die Adresse übersetzt, und die Verbindung kann beginnen.

2.13 Reverse Lookup

Ein Reverse Lookup fragt in die andere Richtung: Zu einer IP-Adresse wird der zugehörige Name gesucht. Dafür gibt es eigene Zonen. IPv4-Adressen werden in der Unterdomain in-addr.arpa abgelegt, indem man die Oktette umgedreht anhängt. Die Adresse 192.0.2.10 wird also als 10.2.0.192.in-addr.arpa abgefragt. IPv6-Adressen liegen unter ip6.arpa in Nibble-Schreibweise.

Der Reverse Lookup ist wichtig für Mailserver, die prüfen, ob der Absender einen passenden Namen hat, und für Logdateien, die Adressen lesbar machen sollen. Er ist aber keine Pflicht für jeden Dienst.

2.14 DNS-Zonen

Eine Zone ist der Teil des DNS-Namensraums, für den ein Server die Daten verwaltet. Eine Zone kann kleiner sein als eine Domain, wenn Teile an andere Server delegiert sind. Die Zone example.com enthält zum Beispiel alle Einträge der Domain, außer jenen, die an eine Subdomain delegiert wurden.

2.15 Forward-Zonen und Reverse-Zonen

Forward-Zonen enthalten die Namen und ihre Adressen, Reverse-Zonen enthalten die Adressen und ihre Namen. Beide Arten werden getrennt gepflegt. Eine Forward-Zone für example.com hat etwa einen Eintrag für www, eine Reverse-Zone für das zugehörige Netz einen PTR-Eintrag, der auf den Namen zeigt.

2.16 DNS-Records

Die Einträge einer Zone heißen Resource Records. Jeder Record hat einen Namen, einen Typ, eine TTL und einen Wert. Die wichtigsten Typen zeigt die folgende Tabelle. Die

Erklärungen folgen danach.

Typ	Zweck	Beispielwert
A	IPv4-Adresse	192.0.2.10
AAAA	IPv6-Adresse	2001:db8::10
CNAME	Alias auf einen anderen Namen	www auf example.com
MX	Mailserver mit Priorität	10 mail.example.com
NS	zuständige Nameserver der Zone	ns1.example.com
PTR	Name zu einer Adresse	host.example.com
TXT	freier Text, etwa für Prüfregeln	v=spf1 …
SOA	Eigenschaften der Zone	Seriennummer, Zeitwerte
SRV	Adresse und Port eines Dienstes	0 5 5060 sip.example.com
CAA	erlaubte Zertifizierungsstellen	0 issue ”letsencrypt.org”

Tabelle 2.1. Die wichtigsten DNS-Records mit Zweck und Beispielwert. Die Beispiele sind schematisch.

A und AAAA

Der A-Record verbindet einen Namen mit einer IPv4-Adresse, der AAAA-Record mit einer IPv6-Adresse. Ein Name kann beide Records besitzen, dann erhalten Clients je nach Netz die passende Adresse. Die Bezeichnung AAAA stammt von der vierfachen Länge gegenüber dem A-Record.

CNAME

Der CNAME-Record ist ein Alias. Er verweist auf einen anderen Namen, dessen Adresse dann gilt. Ein CNAME darf nicht zusammen mit anderen Records am selben Namen stehen, und er darf nicht an der Wurzel einer Zone verwendet werden.

MX

Der MX-Record nennt den Mailserver einer Domain. Die Zahl davor ist die Priorität, eine niedrigere Zahl wird zuerst versucht. Mehrere MX-Einträge mit verschiedenen Prioritäten machen den Mailempfang ausfallsicher. Der MX-Record wird bei E-Mails in Kapitel 8 genauer betrachtet.

NS

Der NS-Record nennt die autoritativen Nameserver einer Zone. Er bildet die Delegation ab, die wir oben kennengelernt haben. Eine Zone sollte mehrere NS-Einträge auf getrennten Servern haben.

PTR

Der PTR-Record ist das Gegenstück zum A-Record im Reverse Lookup. Er liegt in der Reverse-Zone und nennt den Namen zu einer Adresse.

TXT

Der TXT-Record enthält freien Text. Er wird für Prüfregeln verwendet, etwa für SPF, DKIM und DMARC im E-Mail-Verkehr, die Kapitel 8 erklärt. Auch Domainbesitz-Nachweise werden über TXT-Records bestätigt.

SOA

Der SOA-Record beschreibt die Zone selbst. Er nennt den primären Server, die Kontaktadresse, eine Seriennummer und Zeitwerte für die Aktualisierung zwischen primärem und sekundärem Server. Die Seriennummer zeigt, ob sich die Zone geändert hat.

SRV

Der SRV-Record beschreibt Dienste, die Adresse und Port nicht fest kennen. Er nennt Priorität, Gewicht, Port und Zielhost. Clients suchen so zum Beispiel den Server eines Dienstes, ohne dessen Port zu kennen.

CAA

Der CAA-Record legt fest, welche Zertifizierungsstellen Zertifikate für eine Domain ausstellen dürfen. Er ist ein Schutz gegen falsch ausgestellte Zertifikate. Das Zertifikatsthema behandeln wir in Kapitel 5.

2.17 DNS über UDP und TCP

DNS nutzt standardmäßig Port 53, und zwar meist über UDP. Eine klassische DNS-Antwort war auf 512 Byte begrenzt, weil UDP-Pakete damit sicher durch das Netz kommen. Größere Antworten, etwa bei DNSSEC, werden per EDNS erweitert und können größer sein. Ist eine Antwort zu groß oder wird ein Zonentransfer übertragen, wechselt DNS zu TCP auf demselben Port. Deshalb muss eine Firewall für DNS beide Protokolle erlauben.

2.18 DNSSEC

DNSSEC fügt den Antworten digitale Signaturen hinzu. Ein Resolver kann prüfen, ob eine Antwort vom echten Betreiber stammt und unterwegs nicht verändert wurde. DNSSEC schützt also gegen gefälschte Antworten, verschlüsselt aber nichts. Wer die Verbindung verschlüsseln will, braucht die folgenden Verfahren.

2.19 DNS over TLS und DNS over HTTPS

DNS over TLS, kurz DoT, verschickt DNS-Anfragen verschlüsselt über TLS, meist auf Port 853. DNS over HTTPS, kurz DoH, verpackt die Anfragen in HTTPS-Verkehr auf Port 443. Beide verhindern, dass Beobachter im Netz die Namen mitlesen. DoH ist dabei schwerer zu blockieren, weil es wie normaler Webverkehr aussieht. Diese Eigenschaft ist für Schutz hilfreich und für Netzbetreiber problematisch.

2.20 Split DNS

Split DNS liefert verschiedenen Clients verschiedene Antworten für denselben Namen. Ein Unternehmen kann intern eine andere Adresse für einen Dienst hinterlegen als im öffentlichen DNS. Interne Clients erreichen den Server direkt, externe über den öffentlichen Eintrag. Das ist eine verbreitete Praxis, die bei Fehlern zu Verwirrung führt, wenn man nicht weiß, welche Antwort man gerade bekommt.

2.21 Häufige DNS-Fehler

Typische Fehler sind ein falscher Eintrag, ein veralteter Cache, eine fehlende Delegation, eine blockierte Firewall auf Port 53 oder ein Resolver, der nicht erreichbar ist. Ein besonders häufiger Fehler ist die Verwechslung von Ursache und Symptom. Eine Webseite lässt sich nicht öffnen, obwohl die Adresse funktioniert. Dann liegt das Problem oft bei der Auflösung und nicht beim Server. Die Praxisübungen zeigen, wie man das prüft.

Praxis mit DNS-Befehlen

Unter Windows fragt nslookup einen Namen ab, ipconfig /flushdns leert den lokalen Cache, und Resolve-DnsName liefert die Antwort in PowerShell. Unter Linux arbeiten dig und host mit den Records. Der Befehl dig www.example.com A fragt den A-Record ab und zeigt die TTL in Sekunden an. Diese Befehle werden in den Laborübungen ausführlich eingesetzt.

> **Merke**
>
> DNS übersetzt Namen in Informationen, verteilt sich über eine Hierarchie aus Root-, TLD- und autoritativen Servern und nutzt Caches mit TTL. Ein rekursiver Resolver fragt für den Client, und Records wie A, AAAA, MX und TXT bilden die Daten einer Zone.

Kontrollfragen

1. Warum ist die DNS-Hierarchie wichtig, und welche Aufgabe hat ein Root-Server?
2. Was ist der Unterschied zwischen einer rekursiven und einer iterativen Anfrage?

3. Welcher Record verbindet einen Namen mit einer IPv6-Adresse, und welcher beschreibt den Mailserver?
4. Eine Domain wird nach einer Änderung noch mit der alten Adresse angezeigt. Welche Angabe erklärt das?

Musterlösungen: 1. Die Hierarchie verteilt die Verantwortung auf viele Server. Ein Root-Server kennt die Server der Top-Level-Domains, aber nicht die einzelnen Domains. 2. Bei einer rekursiven Anfrage liefert der Server die endgültige Antwort, auch wenn er dafür andere Server befragen muss. Bei einer iterativen Anfrage gibt der Server nur das beste Wissen zurück, meist einen Verweis, und der Fragende stellt die nächste Frage. 3. Der AAAA-Record verbindet mit einer IPv6-Adresse, der MX-Record nennt den Mailserver. 4. Die TTL. Der Cache behält die alte Antwort, bis die TTL abgelaufen ist.

KAPITEL 3

DHCP: Dynamic Host Configuration Protocol

Wenn du deinen Laptop in ein fremdes WLAN einwählst, funktioniert die Verbindung meist sofort. Du hast keine IP-Adresse eingetragen, kein Standardgateway gesetzt und keinen DNS-Server angegeben. Das verdankst du DHCP. Das Protokoll vergibt Netzwerkeinstellungen automatisch an jedes Gerät, das ins Netz kommt. Ohne DHCP müsste jeder Administrator jede Adresse von Hand eintragen, und jede Verwechslung würde einen Konflikt auslösen.

3.1 Was ist DHCP?

DHCP ist ein Protokoll, mit dem ein Server einem Client die notwendige IP-Konfiguration übermittelt. Dazu gehören die IP-Adresse, die Subnetzmaske, das Standardgateway und die DNS-Server, oft noch weitere Angaben. Die Grundlage ist RFC 2131 für IPv4. Für IPv6 gilt RFC 8415.

3.2 Warum benötigen wir DHCP?

In einem Netz mit hundert Arbeitsplätzen, Laptops, Smartphones und Gästen wäre die manuelle Pflege aller Einstellungen kaum zu bewältigen. DHCP spart diese Arbeit und sorgt dafür, dass Geräte nach einem Neustart oder Ortswechsel sofort funktionieren. Es verhindert außerdem doppelte Adressen, weil der Server über alle vergebenen Adressen Buch führt.

3.3 Statische und dynamische Adressvergabe

Es gibt zwei grundlegende Arten der Adressvergabe. Bei der statischen Vergabe trägt der Administrator eine feste Adresse im Gerät ein. Das eignet sich für Server, Router und Drucker, die immer unter derselben Adresse erreichbar sein sollen. Bei der dynamischen Vergabe bekommt das Gerät vom DHCP-Server eine Adresse für eine begrenzte Zeit.

Das eignet sich für Arbeitsplätze und Gäste, die das Netz wechseln.

3.4 DHCP-Server und DHCP-Client

Der DHCP-Server verwaltet einen Pool von Adressen und die Einstellungen dazu. Er lauscht auf Port 67. Der DHCP-Client ist das Gerät, das eine Konfiguration anfordert, und lauscht auf Port 68. Auf vielen Heimroutern laufen beide Rollen: Der Router vergibt Adressen an die Geräte im Haus.

3.5 DHCP-Scope

Ein Scope ist der Adressbereich, den ein DHCP-Server für ein Netz vergibt. Er umfasst einen Adressbereich, eine Subnetzmaske und die Optionen, die an die Clients gehen. Ein Scope für das Netz 192.168.1.0/24 könnte die Adressen von .100 bis .199 vergeben, während .1 für den Router reserviert ist und .200 bis .254 für feste Geräte bleiben. Die genaue Aufteilung ist eine Planungsaufgabe.

3.6 DHCP-Lease

Die Vergabe ist zeitlich begrenzt. Diese Zeit heißt Lease. Der Client darf die Adresse während der Lease-Zeit nutzen, danach muss er sie erneuern oder aufgeben. Typische Lease-Zeiten reichen von wenigen Stunden in Gastnetzen bis zu mehreren Tagen in Unternehmen. Die Begrenzung sorgt dafür, dass nicht mehr benötigte Adressen wieder frei werden.

3.7 DHCP-Reservierung

Eine Reservierung verbindet eine bestimmte MAC-Adresse fest mit einer IP-Adresse. Das Gerät bekommt dadurch immer dieselbe Adresse aus dem dynamischen Pool. Das ist praktisch für Drucker oder Netzwerkspeicher, die über den Namen oder die Adresse erreichbar sein sollen, ohne dass ihre Konfiguration wechselt.

3.8 DHCP-Optionen

Neben Adresse und Maske übermittelt DHCP weitere Angaben, die man Optionen nennt. Die wichtigsten sind das Standardgateway, die DNS-Server, der Domainname und die Zeitserver. Jede Option hat eine Nummer und einen Wert. So kann ein Client das Netz vollständig konfigurieren, ohne dass der Benutzer etwas eingibt.

3.9 DHCP-Relay

DHCP-Anfragen sind Broadcasts, die ein Router nicht weiterleitet. Ein Client im Netz ohne eigenen DHCP-Server würde deshalb keine Antwort bekommen. Ein DHCP-Relay ist ein Dienst auf einem Router, der diese Broadcasts entgegennimmt und gezielt an den Server in einem anderen Netz weiterleitet. Der Server erkennt daran das Netz des Clients und wählt den passenden Scope.

3.10 DHCPv4 und DHCPv6 im Vergleich

DHCPv4 vergibt IPv4-Adressen über die Ports 67 und 68. DHCPv6 nutzt die Ports 546 für Clients und 547 für Server und arbeitet mit IPv6-Adressen. DHCPv6 kann die Adresse vollständig vergeben oder nur zusätzliche Angaben wie DNS-Server liefern. Die Gegenüberstellung zeigt die Unterschiede in der Übersicht.

Merkmal	DHCPv4	DHCPv6
Adressen	IPv4	IPv6
Ports	67 und 68	546 und 547
Start der Anfrage	Broadcast, DORA	Multicast, Solicit und andere Nachrichten
Standardgateway	über DHCP	über Router Advertisement
Zusatzangaben	über Optionen	über Optionen

Tabelle 3.1. Die wichtigsten Unterschiede zwischen DHCPv4 und DHCPv6. Die Nachrichtennamen von DHCPv6 unterscheiden sich vom DORA-Ablauf.

3.11 Unterschied zwischen DHCPv6 und SLAAC

IPv6 kennt einen zweiten Weg zur Konfiguration, SLAAC, Stateless Address Autoconfiguration. Dabei lernt das Gerät das Präfix aus Router Advertisements, die der Router im Netz verschickt, und bildet seine Adresse selbst. Ein DHCP-Server ist dafür nicht nötig. DHCPv6 hingegen liefert die Adresse oder Zusatzangaben vom Server. Beide können kombiniert werden. Welches Verfahren ein Netz nutzt, legt der Router mit Flags in seinen Ankündigungen fest.

3.12 Der DORA-Ablauf

Der Kern von DHCPv4 ist der DORA-Ablauf. Der Name besteht aus den Anfangsbuchstaben der vier Nachrichten: Discover, Offer, Request und Acknowledge. Das Gerät, das noch keine Adresse hat, startet den Ablauf mit einem Broadcast. Die folgende Abbildung zeigt die vier Schritte und die Adressen, die dabei verwendet werden.

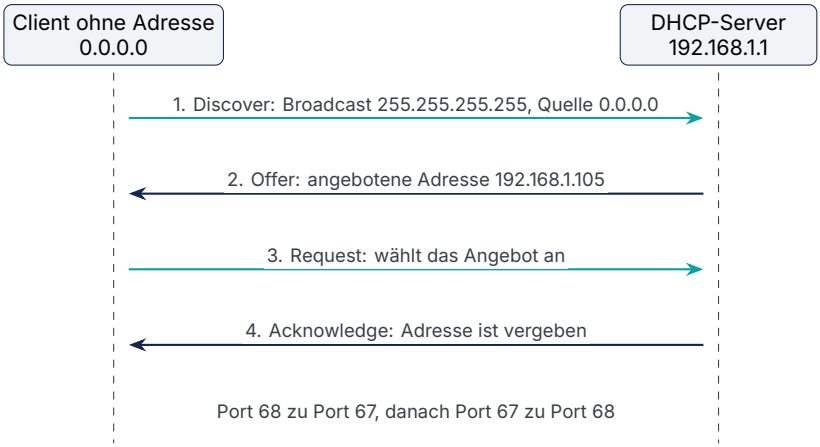


Abbildung 3.1. Der DORA-Ablauf zwischen einem Client ohne Adresse und einem DHCP-Server. Die Adressen sind Beispiele. Schematische Darstellung.

Discover

Der Client weiß nicht, welcher Server antwortet, und besitzt noch keine Adresse. Er sendet deshalb einen Discover als Broadcast an 255.255.255.255. Als Quelle nutzt er 0.0.0.0, als Quellport 68, als Zielport 67. Der Discover enthält außerdem die MAC-

Adresse des Clients und Wünsche wie die Lease-Zeit.

Offer

Jeder DHCP-Server im Netz, der die Anfrage erhält, kann mit einem Offer antworten. Das Angebot enthält eine Adresse aus seinem Pool, die Subnetzmaske, die Lease-Zeit und weitere Optionen. Mehrere Server können so unterschiedliche Angebote machen. Der Client wählt eines aus.

Request

Der Client sendet einen Request als Broadcast und nennt darin den Server, dessen Angebot er annimmt. Der Broadcast erreicht auch die anderen Server. Diese erkennen, dass ihr Angebot abgelehnt wurde, und geben die Adresse wieder frei. Der Request ist also zugleich eine Antwort an den gewählten Server und eine Absage an die anderen.

Acknowledge

Der gewählte Server bestätigt mit einem Acknowledge, kurz ACK. Ab diesem Moment gehört die Adresse dem Client für die Lease-Zeit. Der Client setzt die Konfiguration und kann kommunizieren. Die Abbildung zeigt diese Abfolge.

3.13 Lease Renewal und Rebinding

Die Lease läuft nicht einfach aus. Nach der Hälfte der Lease-Zeit, dem Zeitpunkt T1, sendet der Client eine Erneuerungsanfrage direkt an den Server, der ihm die Adresse vergeben hat. Antwortet der Server mit einem ACK, verlängert sich die Lease. Kommt keine Antwort, versucht der Client es bei 87,5 Prozent der Zeit, dem Zeitpunkt T2, mit einem Broadcast an beliebige Server. Dieses zweite Verfahren heißt Rebinding. Gelingt auch das nicht, muss der Client die Adresse aufgeben.

3.14 DHCP-Lease-Lebenszyklus

Die folgende Abbildung zeigt den Ablauf einer Lease über die Zeit mit den Zeitpunkten T1 und T2. Sie macht sichtbar, wann der Client handelt und wann die Adresse endgültig verloren geht.

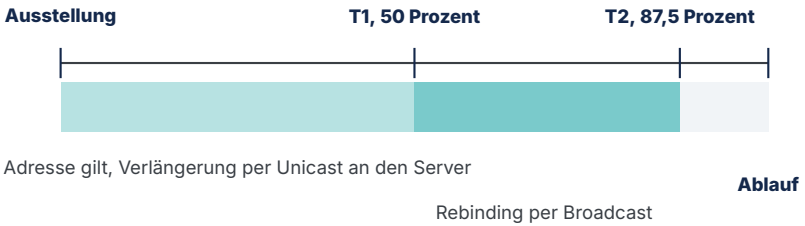


Abbildung 3.2. Lebenszyklus einer DHCP-Lease. Nach der Hälfte der Zeit erneuert der Client die Lease beim Server, nach 87,5 Prozent versucht er es per Broadcast. Schematische Darstellung, nicht maßstabsgetreu.

3.15 DHCP-Fehler

Häufige Fehler sind ein erschöpfter Pool, ein falscher Scope, ein Relay, das nicht richtig konfiguriert ist, oder ein Server, der nicht erreichbar ist. Ein Client ohne Antwort bekommt keine Adresse und wählt eine APIPA-Adresse aus dem Bereich 169.254.0.0/16. Diese Adresse erlaubt nur die Kommunikation im eigenen Segment. Wer sie sieht, weiß, dass der DHCP-Ablauf gescheitert ist, auch wenn der Server im Netz läuft. Dann helfen die Befehle ipconfig /release und ipconfig /renew, um den Ablauf neu zu starten.

3.16 APIPA

APIPA steht für Automatic Private IP Addressing und ist die Notlösung, die Windows und andere Systeme verwenden, wenn kein DHCP-Server antwortet. Die Adresse ist nur lokal gültig und nicht im Internet nutzbar. Sie zeigt also einen Fehler an, den man beheben muss, und ist keine Lösung.

3.17 DHCP-Sicherheit

DHCP vertraut jedem Server, der eine Antwort sendet. Das macht es angreifbar. Ein Angreifer kann einen eigenen DHCP-Server ins Netz stellen und den Clients eine falsche Standardroute oder falsche DNS-Server mitgeben. Dann leitet er den Verkehr über sich um. Schutz bieten Switches mit DHCP-Snooping, das nur von bestimmten Ports DHCP-Antworten zulässt.

3.18 Rogue DHCP Server

Ein Rogue DHCP Server ist ein nicht autorisierter DHCP-Server im Netz. Er kann unbemerkt Adressen verteilen, die den echten Server verdrängen, oder falsche Optionen liefern. Erkennbar ist er daran, dass Clients Adressen aus unerwarteten Bereichen oder mit falschen Gateways erhalten. Die Prüfung von DHCP-Verkehr mit Wireshark zeigt, von welcher Adresse die Angebote kommen.

3.19 Praxis mit DHCP

Unter Windows zeigt ipconfig /all die aktuell vergebene Konfiguration einschließlich des DHCP-Servers und der Lease-Zeiten. ipconfig /release gibt die Adresse frei, ipconfig /renew fordert eine neue an. Unter Linux liefert je nach System der Befehl ip addr die Adresse, während DHCP-Details in den Logs des Dienstes stehen, etwa über journalctl. Ein Mitschnitt mit Wireshark zeigt den DORA-Ablauf mit dem Filter dhcp.

> **Merke**
>
> DHCP vergibt Netzwerkeinstellungen für eine begrenzte Zeit, die Lease. Der Ablauf DORA besteht aus Discover, Offer, Request und Acknowledge. Erneuerung erfolgt nach der Hälfte der Lease-Zeit, Rebinding nach 87,5 Prozent.

Kontrollfragen

1. Nenne die vier Nachrichten des DORA-Ablaufs in der richtigen Reihenfolge.
2. Warum sendet der Client den Discover als Broadcast, und welche Quelladresse nutzt er?
3. Ein Client erhält die Adresse 169.254.8.44. Was bedeutet das, und welche Ursache ist wahrscheinlich?
4. Zu welchem Zeitpunkt versucht der Client, seine Lease zu erneuern, und wann per Broadcast?

Musterlösungen: 1. Discover, Offer, Request, Acknowledge. 2. Der Client kennt die Adresse des Servers nicht und hat noch keine eigene. Er nutzt die Quelladresse 0.0.0.0. 3. Es handelt sich um eine APIPA-Adresse. Der Client hat keine Antwort vom DHCP-Server erhalten, der Ablauf ist gescheitert. Wahrscheinlich ist der Server nicht erreichbar oder der Pool erschöpft. 4. Die Erneuerung erfolgt bei 50 Prozent der Lease-Zeit als Unicast an den Server. Der Broadcast folgt bei 87,5 Prozent als Rebinding.

KAPITEL 4

HTTP: Hypertext Transfer Protocol

Jedes Mal, wenn du eine Webseite öffnest, sendet dein Browser eine Frage an einen Server, und der Server antwortet mit dem Inhalt. Diese Sprache heißt HTTP. Sie ist so verbreitet, dass viele Programme sie nutzen, nicht nur Browser: Apps laden Daten, Geräte melden Messwerte, Dienste tauschen Informationen aus. HTTP beschreibt, wie eine Anfrage aussieht, wie eine Antwort aussieht und was die einzelnen Angaben bedeuten. Dieses Kapitel erklärt die Grundlagen und die wichtigsten Details, die man im Alltag braucht.

4.1 Was ist HTTP?

HTTP ist ein Anwendungsprotokoll nach dem Muster Anfrage und Antwort. Der Client stellt eine Anfrage an eine Ressource, der Server liefert eine Antwort. Eine Ressource ist alles, was der Server anbietet: eine Seite, ein Bild, eine Datenschnittstelle. HTTP selbst ist zustandslos, jede Anfrage steht für sich. Zustände wie ein Login werden deshalb über zusätzliche Mechanismen wie Cookies geführt, dazu kommen wir später. Die Semantik ist in RFC 9110 festgelegt, die Übertragung in HTTP/1.1 nach RFC 9112.

4.2 Client und Webserver

Der Client ist meist ein Browser oder ein Programm. Der Webserver ist ein Dienst, der auf Port 80 für HTTP oder Port 443 für HTTPS lauscht und Anfragen beantwortet. Dazwischen können Proxys, Caches und Lastverteiler liegen, die Anfragen weiterreichen oder zwischenspeichern. Für den Client sieht das meist wie eine direkte Verbindung aus.

4.3 HTTP-Anfragen

Eine Anfrage besteht aus einer Startzeile, Headern und optional einem Körper. Die Startzeile nennt die Methode, den Pfad der Ressource und die Protokollversion. Danach folgen Header als Zeilen mit Name und Wert, zum Beispiel der Host, für den die Anfrage bestimmt ist, oder der bevorzugte Dateityp. Eine leere Zeile trennt den Kopf vom Körper.

4.4 HTTP-Antworten

Eine Antwort beginnt mit einer Statuszeile, die Version, einen dreistelligen Statuscode und eine Begründung enthält. Danach folgen Header, eine leere Zeile und der Körper mit dem Inhalt. Der Statuscode sagt dem Client sofort, was passiert ist, ob die Anfrage erfolgreich war, umgeleitet wird oder scheiterte.

4.5 URLs und URIs

Ein Uniform Resource Identifier, kurz URI, bezeichnet eine Ressource. Eine URL, Uniform Resource Locator, ist eine URI, die zugleich angibt, wie man die Ressource erreicht. In https://www.example.org/hilfe/seite.html steht https für das Protokoll, www.example.org für den Host, /hilfe/seite.html für den Pfad. Ein Teil nach einem Fragezeichen heißt Query und übergibt Parameter.

Die folgende Abbildung zeigt den Aufbau einer Anfrage und einer Antwort nebeneinander.



Startzeile, Header, leere Zeile und Körper. Die Beispiele sind gekürzt. Schematische Darstellung.

Abbildung 4.1. Aufbau einer HTTP-Anfrage und einer HTTP-Antwort. Die Startzeile enthält bei der Anfrage Methode, Pfad und Version, bei der Antwort Version, Statuscode und Begründung.

4.6 HTTP-Methoden

Die Methode gibt an, was der Client mit der Ressource tun möchte. Die wichtigsten sind die folgenden.

GET fordert eine Ressource an. Es verändert nichts auf dem Server und ist deshalb sicher und wiederholbar. POST sendet Daten an den Server, etwa ein Formular, und kann Änderungen auslösen. PUT ersetzt eine Ressource vollständig, PATCH ändert nur Teile davon. DELETE entfernt eine Ressource. HEAD verhält sich wie GET, liefert aber nur die Header, nicht den Körper. OPTIONS fragt ab, welche Methoden ein Server für eine Ressource akzeptiert.

Ein wichtiges Begriffspaar sind sichere und idempotente Methoden. Sicher bedeutet, dass die Methode keine Veränderung bewirkt. Idempotent bedeutet, dass eine mehrfache Ausführung dasselbe Ergebnis hat wie eine einzige. GET ist sicher und idempotent, PUT ist idempotent, POST ist weder das eine noch das andere. Deshalb fragt ein Browser beim Neuladen einer Seite mit POST nach einer Bestätigung.

4.7 HTTP-Statuscodes

Der Statuscode ist eine dreistellige Zahl, deren erste Ziffer die Klasse angibt. Die folgende Tabelle zeigt die Klassen und die wichtigsten Beispiele.

Klasse	Code	Bedeutung
1xx	100	Informationen, Anfrage läuft weiter
2xx	200, 201	erfolgreich, 201 bedeutet neu angelegt
3xx	301, 302, 304	Umleitung oder Nutzung des Caches
4xx	400, 401, 403, 404	Fehler auf Seiten des Clients
5xx	500, 502, 503	Fehler auf Seiten des Servers

Tabelle 4.1. Die Klassen der HTTP-Statuscodes mit Beispielen.

Die wichtigsten Einzelcodes sind 200 OK für eine erfolgreiche Antwort, 201 Created für eine neu angelegte Ressource, 301 Moved Permanently für eine dauerhafte Umleitung

und 302 Found für eine vorübergehende. 304 Not Modified sagt dem Client, dass seine zwischengespeicherte Version noch gilt. 400 Bad Request bedeutet eine fehlerhafte Anfrage, 401 Unauthorized eine fehlende oder falsche Anmeldung, 403 Forbidden eine verweigerte Berechtigung, 404 Not Found eine unbekannte Ressource. Bei den Serverfehlern steht 500 Internal Server Error für einen unbekannten Fehler, 502 Bad Gateway für einen Proxy, der keine gültige Antwort vom Ziel bekommt, und 503 Service Unavailable für einen Dienst, der vorübergehend nicht verfügbar ist.

Die Abbildung zeigt die Abfolge einer Anfrage mit einer Umleitung und anschließender Antwort.

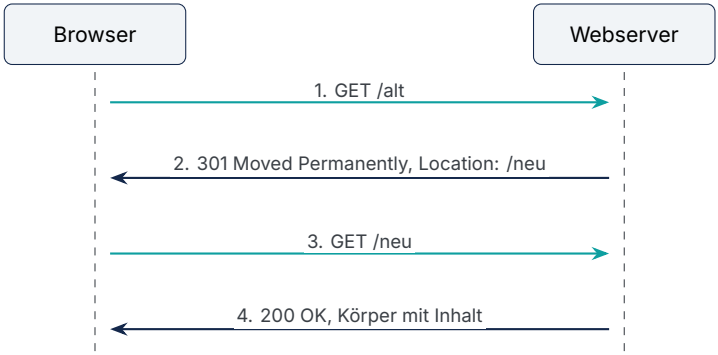


Abbildung 4.2. Anfrage mit Umleitung. Der Browser fragt eine alte Adresse ab, der Server verweist auf die neue, der Browser fragt dort erneut. Schematische Darstellung.

4.8 HTTP-Header

Header tragen Zusatzinformationen. Der Host-Header ist in HTTP/1.1 Pflicht und nennt den gewünschten Namen, weil ein Server mehrere Webseiten auf einer Adresse betreiben kann. Content-Type beschreibt, welche Art Daten der Körper enthält, zum Beispiel text/html oder application/json. Accept bezeichnet die gewünschten Typen auf Clientseite. Content-Length gibt die Länge des Körpers in Byte an.

Für das Caching gibt es Cache-Control, das festlegt, wie lange eine Antwort gespeichert werden darf, und ETag, ein Kennzeichen einer bestimmten Version. Mit einem ETag kann der Client gezielt fragen, ob sich etwas geändert hat, und bei Gleichheit eine 304 erhalten.

4.9 Cookies

Ein Cookie ist ein kleines Datenstück, das der Server dem Client mitgibt, und das der Client bei späteren Anfragen zurückschickt. Der Server setzt es mit dem Header Set-Cookie, der Client sendet es mit Cookie. Damit entsteht über zustandslose Anfragen hinweg ein Zusammenhang, etwa ein eingeloggter Benutzer. Die Regeln dafür sind in RFC 6265 beschrieben. Cookies mit den Attributen Secure und HttpOnly schützen vor unverschlüsseltem Versand und vor Zugriff durch Skripte.

4.10 Sessions

Eine Session ist die serverseitige Entsprechung. Der Server speichert Informationen über einen Benutzer und vergibt eine Kennung, die im Cookie steht. Der Client kennt nur die Kennung, die Daten bleiben beim Server. Das ist sicherer, weil nicht der Inhalt, sondern nur ein Zeiger übertragen wird.

4.11 Content-Type und Caching

Der Content-Type entscheidet, wie der Browser den Körper darstellt. Eine falsche Angabe führt zu unlesbarem Text oder zu Sicherheitsproblemen, weshalb Server ihn sorgfältig setzen. Caching spart Übertragungen. Ein Browser speichert Ressourcen, die er bereits geladen hat, und fragt nur bei Ablauf oder mit einem ETag nach.

4.12 HTTP/1.1, HTTP/2 und HTTP/3

HTTP/1.1 ist die klassische Textversion. Verbindungen bleiben standardmäßig offen, sogenannte persistente Verbindungen. Eine Antwort nach der anderen wird jedoch geliefert, was bei vielen Ressourcen bremst. HTTP/2 führt ein binäres Format ein, das mehrere Anfragen gleichzeitig auf einer Verbindung zulässt, und komprimiert die Header. Es ist in RFC 9113 beschrieben. HTTP/3 läuft über QUIC statt über TCP und behebt das Head-of-Line Blocking auf Transportebene, das HTTP/2 noch hat. Es ist in RFC 9114 festgelegt.

Merkmal	HTTP/1.1	HTTP/2
Format	Text	binär
Transport	TCP	TCP
Parallele Anfragen	über mehrere Verbindungen	Streams auf einer Verbindung
Header	Text, unkomprimiert	komprimiert
Head-of-Line Blocking	pro Verbindung	auf Transportebene

Tabelle 4.2. Vergleich von HTTP/1.1 und HTTP/2. HTTP/3 ersetzt TCP durch QUIC, siehe dazu Band 3.

4.13 Sicherheitsaspekte bei HTTP

Reines HTTP überträgt alles im Klartext. Passwörter, Cookies und Inhalte kann jeder im Netz mitlesen, der den Verkehr sieht. Deshalb nutzt man heute HTTPS. Auch Server selbst sind angreifbar, etwa über unvorsichtig verarbeitete Eingaben. Das ist Gegenstand der Webentwicklung, für Netzwerker wichtig ist die Verschlüsselung und die richtige Konfiguration der Header.

4.14 HTTP im Alltag

Jede Webseite, jede App, die Daten lädt, und jede Schnittstelle zwischen Diensten nutzt HTTP. Die Statuscodes erklären Fehler, die man im Browser sieht, und die Header zeigen, wie gecacht wird.

4.15 Untersuchung unter Windows und Linux

Die Entwicklertools der Browser zeigen Anfragen, Header und Statuscodes im Netzwerk-Tab. Unter Linux und Windows liefert curl die Antwort im Terminal. Der Befehl curl -I https://www.example.org/ zeigt nur die Header, curl -v zeigt den gesamten Austausch einschließlich der Anfrage. Unter Windows steht curl in aktuellen Versionen ebenfalls zur Verfügung. Die Laborübungen zeigen die Befehle im Detail.

4.16 Typische Fehler

Ein 404 bedeutet, dass die Adresse falsch ist oder die Ressource entfernt wurde. Ein 500 weist auf einen Fehler im Server hin, der nur der Betreiber beheben kann. Ein 502 oder 503 deutet auf einen Proxy oder ein Backend hin, das nicht antwortet. Ein 403 kommt von einer Berechtigung und nicht von einem Netzproblem. Wer diese Unterschiede kennt, spart viel Suche.

> **Merke**
>
> HTTP folgt dem Muster Anfrage und Antwort. Die Methode beschreibt die Absicht, der Statuscode die Antwort, und Header tragen Zusatzinformationen. Cookies und Sessions verbinden zustandslose Anfragen, HTTP/2 und HTTP/3 beschleunigen die Übertragung.

Kontrollfragen

1. Welche Methode eignet sich, um eine Ressource abzurufen, ohne etwas zu verändern?
2. Warum ist GET idempotent, POST dagegen nicht?
3. Ein Server antwortet mit 404. Welche Ursache liegt nahe, und welche nicht?
4. Was ist der Unterschied zwischen einem Cookie und einer Session?

Musterlösungen: 1. GET. 2. Mehrfaches GET liefert dieselbe Ressource, ohne sie zu verändern. Ein POST kann bei jeder Ausführung etwas anlegen, etwa eine Bestellung, deshalb ist er nicht idempotent. 3. Die Adresse ist falsch oder die Ressource wurde entfernt. Nicht die Ursache ist ein Netzausfall, denn dann käme gar keine Antwort. 4. Ein Cookie speichert die Daten oder eine Kennung auf dem Client, eine Session speichert die Daten auf dem Server und nutzt nur eine Kennung im Cookie.

KAPITEL 5

HTTPS und TLS

HTTP überträgt alles im Klartext. Wer im selben Netz sitzt oder den Weg der Pakete beobachten kann, liest Passwörter und Inhalte mit. HTTPS löst dieses Problem, indem es HTTP in eine verschlüsselte Verbindung packt. Die Verschlüsselung übernimmt TLS, Transport Layer Security. Dieses Kapitel erklärt, warum Verschlüsselung nötig ist, wie TLS mit Zertifikaten Vertrauen herstellt und wie der Handshake abläuft. Außerdem zeigen wir die Unterschiede zwischen TLS 1.2 und TLS 1.3.

5.1 Was ist HTTPS?

HTTPS ist HTTP über TLS. Der Inhalt der Nachrichten ist identisch, nur liegt er verschlüsselt auf dem Transport. Der Standardport ist 443. Der Browser zeigt das mit dem Schloss-Symbol und dem Präfix https an.

5.2 Warum ist Verschlüsselung wichtig?

Verschlüsselung schützt vor drei Gefahren. Die erste ist das Mitlesen, das Vertraulichkeit verletzt. Die zweite ist das unbemerkte Verändern von Inhalten auf dem Weg, das die Integrität verletzt. Die dritte ist das Vortäuschen eines falschen Gegenübers, das die Authentizität verletzt. TLS adressiert alle drei. Vertraulichkeit entsteht durch Schlüssel, Integrität durch Prüfsummen, Authentizität durch Zertifikate.

5.3 Unterschied zwischen HTTP und HTTPS

Ein HTTP-Verkehr ist für jeden Beobachter lesbar. Bei HTTPS sieht ein Beobachter nur, welcher Server angesprochen wird und ungefähr wie viel Verkehr fließt, nicht aber die Inhalte. Die Namen der Server können sichtbar sein, zum Beispiel über SNI, dazu gleich mehr. Die Abbildung vergleicht beide Verbindungen.



Abbildung 5.1. Gegenüberstellung von HTTP und HTTPS. Bei HTTP liest ein Beobachter den Inhalt mit, bei HTTPS sieht er nur verschlüsselte Daten. Schematische Darstellung.

5.4 Transport Layer Security

TLS ist ein Protokoll, das sich zwischen TCP und die Anwendung schaltet. Es verhandelt die Verschlüsselungsverfahren, prüft die Identität des Servers und schützt jede Nachricht. TLS ist in RFC 8446 für Version 1.3 festgelegt. Der Vorgänger TLS 1.2 ist in RFC 5246 beschrieben und wird weiterhin verwendet, ist aber veraltet.

5.5 TLS 1.2 und TLS 1.3

TLS 1.3 ist die aktuelle Version und ersetzt TLS 1.2 in modernen Umgebungen. Der Handshake braucht nur eine Runde, bevor verschlüsselte Daten fließen können. TLS 1.2 braucht zwei Runden. TLS 1.3 verzichtet auf ältere und unsichere Verfahren wie die statische RSA-Schlüsselvereinbarung und lässt nur authentifizierte Verschlüsselungsverfahren zu. TLS 1.3 verschlüsselt außerdem einen größeren Teil des Handshakes, wodurch weniger Informationen im Klartext sichtbar sind. TLS 1.2 gilt nicht als unsicher, wenn es mit modernen Verfahren konfiguriert ist, aber TLS 1.0 und 1.1 sind abgekündigt.

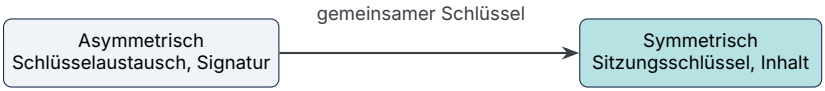
5.6 Öffentliche und private Schlüssel

Asymmetrische Kryptografie arbeitet mit einem Schlüsselpaar. Der öffentliche Schlüssel darf jeder kennen, der private Schlüssel bleibt geheim. Was mit dem einen verschlüsselt wird, lässt sich nur mit dem anderen entschlüsseln. Ein Server beweist mit seinem privaten Schlüssel, dass er der Besitzer ist, und der Client prüft das mit dem öffentlichen Schlüssel. Asymmetrische Verfahren sind rechenintensiv, deshalb nutzt TLS sie nur für den Austausch der Schlüssel und die Beweisführung.

5.7 Symmetrische und asymmetrische Kryptografie

Symmetrische Verfahren nutzen denselben Schlüssel zum Verschlüsseln und Entschlüsseln. Sie sind schnell und eignen sich für große Datenmengen. Das Problem ist die Übergabe des gemeinsamen Schlüssels. TLS löst es, indem es den Schlüssel asymmetrisch aushandelt und danach symmetrisch verschlüsselt.

Die folgende Abbildung zeigt das Zusammenspiel. Ein Schlüsselpaar sichert den Austausch, ein gemeinsamer Sitzungsschlüssel verschlüsselt den Inhalt.



Asymmetrisch sichert die Übergabe, symmetrisch schützt die Daten.

Abbildung 5.2. Zusammenspiel von asymmetrischer und symmetrischer Kryptografie in TLS. Der asymmetrische Teil übergibt einen gemeinsamen Sitzungsschlüssel, der symmetrische Teil verschlüsselt den Datenverkehr.

5.8 Zertifikate

Ein Zertifikat bindet einen öffentlichen Schlüssel an einen Namen, etwa www.example.org. Es enthält den Namen, den Schlüssel, die Gültigkeitszeit und die Signatur einer Stelle, der man vertraut. Das Format heißt X.509. Ohne Zertifikat könnte der Client nicht wissen, ob der Schlüssel wirklich dem Server gehört oder einem Angreifer.

5.9 Certificate Authorities

Eine Certificate Authority, kurz CA, stellt Zertifikate aus und bestätigt damit, dass der Name zum Schlüssel gehört. Die Vertrauensanker sind die Stammzertifikate der Root-CAs, die in Betriebssystemen und Browsern gespeichert sind. Wer einer Root-CA nicht vertraut, dem fehlt die Grundlage für die Prüfung.

5.10 Public Key Infrastructure

Die Public Key Infrastructure, kurz PKI, ist das Gesamtsystem aus CAs, Zertifikaten, Regeln und Prozessen, das diese Vertrauenskette organisiert. Sie umfasst auch das Widerrufen von Zertifikaten und die Prüfung der Gültigkeit.

5.11 Zertifikatsketten

Selten signiert eine Root-CA direkt ein Serverzertifikat. Üblich ist eine Kette: Das Serverzertifikat, das Blatt, wird von einer Zwischenstelle signiert, deren Zertifikat wiederum von der Root-CA signiert ist. Der Client prüft die Kette von unten nach oben, bis er einen Anker im Vertrauensspeicher erreicht. Die folgende Abbildung zeigt diese Kette.

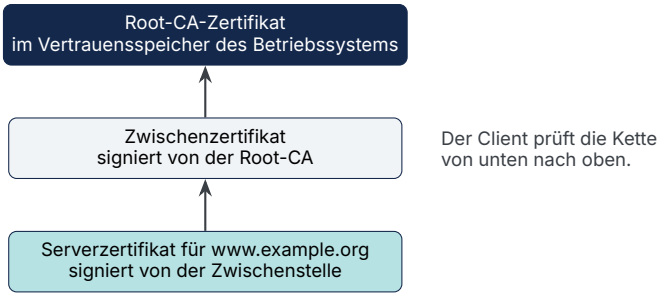


Abbildung 5.3. Zertifikatskette eines Webservers. Das Serverzertifikat ist von einer Zwischenstelle signiert, diese von der Root-CA. Der Pfeil zeigt die Signatur. Schematische Darstellung.

5.12 TLS-Handshake

Der Handshake baut die verschlüsselte Verbindung auf. Er besteht bei TLS 1.3 aus wenigen Nachrichten. Der Client sendet ClientHello mit unterstützten Verfahren, einem Schlüsselanteil und dem gewünschten Servernamen. Der Server antwortet mit ServerHello, seinem Schlüsselanteil und seinem Zertifikat sowie einem Nachweis, dass er den privaten Schlüssel besitzt. Beide Seiten berechnen daraus denselben Sitzungsschlüssel. Die folgende Abbildung zeigt den Ablauf vollständig.



Abbildung 5.4. TLS-1.3-Handshake mit Anfrage und Antwort. Nach zwei Runden ist die Verbindung verschlüsselt, die HTTP-Daten fließen danach. Schematische Darstellung, die Nachrichtennamen sind vereinfacht.

5.13 Zertifikatsprüfung

Der Client prüft mehrere Punkte. Er kontrolliert, ob die Kette zu einer vertrauenswürdigen Root-CA führt, ob das Zertifikat für den angefragten Namen gilt, ob es gültig und nicht gesperrt ist und ob der Server den privaten Schlüssel nachweisen kann. Schlägt eine Prüfung fehl, bricht der Browser ab und warnt. Eine Warnung ist deshalb meist ein echtes Sicherheitssignal und kein Fehler, den man wegklicken sollte.

5.14 Perfect Forward Secrecy

Perfect Forward Secrecy bedeutet, dass selbst der Verlust des privaten Schlüssels des Servers vergangene Verbindungen nicht offenlegt. Das gelingt, weil jede Sitzung einen eigenen, kurzlebigen Schlüssel aushandelt, der nach dem Ende verworfen wird. TLS 1.3 verwendet dafür immer kurzlebige Schlüssel, bei TLS 1.2 muss die Konfiguration sie aktiv vorsehen.

5.15 SNI

Server Name Indication, kurz SNI, ist eine Erweiterung im ClientHello. Der Client nennt darin den Namen, den er erreichen möchte. Das ist nötig, damit ein Server mit einer Adresse mehrere Zertifikate für verschiedene Namen anbieten kann. SNI wird unverschlüsselt übertragen und verrät deshalb den Namen der Seite, auch bei HTTPS. Neuere Verfahren wollen auch diese Angabe verschlüsseln.

5.16 ALPN

Application-Layer Protocol Negotiation, kurz ALPN, handelt im Handshake das Anwendungsprotokoll aus. Der Client nennt die Protokolle, die er spricht, etwa h2 für HTTP/2 oder http/1.1, der Server wählt eines. So weiß der Client früh, ob er HTTP/2 nutzen kann.

5.17 Häufige Zertifikatsfehler

Die häufigsten Fehler sind ein abgelaufenes Zertifikat, ein Name, der nicht zum Zertifikat passt, eine fehlende Zwischenzertifikatsangabe und eine unbekannte oder selbst signierte Root-CA. Auch eine falsche Systemzeit führt zu Fehlern, weil Gültigkeiten an der Uhr geprüft werden. Wer die Fehlermeldung genau liest, erkennt meist die Ursache.

5.18 Praxis mit HTTPS und TLS

Im Browser zeigt ein Klick auf das Schloss die Zertifikatsdetails, Aussteller und Gültigkeit. Der Befehl curl -v https://www.example.org/ zeigt den Handshake und das Zertifikat im Protokoll. Mit openssl s_client -connect www.example.org:443 -servername www.example.org lässt sich der Handshake gezielt untersuchen, die Ausgabe enthält die Zertifikatskette und die verhandelte Version. Die Laborübungen zeigen das Vorge-

hen.

Merke

HTTPS legt HTTP in eine TLS-Verbindung. Asymmetrische Verfahren sichern den Schlüsselaustausch, symmetrische den Datenverkehr. Zertifikate binden Schlüssel an Namen, und die Kette führt zu einer Root-CA im Vertrauensspeicher.

Kontrollfragen

1. Welche drei Schutzziele erfüllt TLS, und mit welchem Mittel erreicht es jedes?
2. Warum nutzt TLS asymmetrische Verfahren nur für den Schlüsselaustausch?
3. Der Browser meldet, dass der Name nicht zum Zertifikat passt. Was ist die Ursache?
4. Was bedeutet Perfect Forward Secrecy?

Musterlösungen: 1. Vertraulichkeit durch Verschlüsselung, Integrität durch Prüfsummen, Authentizität durch Zertifikate. 2. Asymmetrische Verfahren sind rechenintensiv. Für den großen Datenverkehr nutzt TLS schnelle symmetrische Verfahren mit dem ausgehandelten Schlüssel. 3. Das Zertifikat gilt für einen anderen Namen, als im Browser angefragt wurde. Der Client erkennt den Fehler, weil der Name nicht im Zertifikat steht. 4. Jede Sitzung hat einen eigenen, kurzlebigen Schlüssel. Selbst der spätere Verlust des privaten Schlüssels des Servers legt vergangene Verbindungen nicht offen.

KAPITEL 6

FTP, FTPS und SFTP

Dateien über das Netz zu verschicken ist eine der ältesten Aufgaben der Computernetze. Schon früh gab es Verfahren, um Dateien von einem Rechner auf einen anderen zu kopieren. Das bekannteste ist FTP, das File Transfer Protocol. Es ist alt, weit verbreitet und in vielen Betrieben noch im Einsatz, aber es hat eine gravierende Schwäche: Es überträgt Passwörter im Klartext. Dieses Kapitel erklärt, wie FTP funktioniert, welche Modi es gibt und wie FTPS und SFTP das Problem lösen.

6.1 Dateiübertragung über Netzwerke

Eine Dateiübertragung braucht drei Dinge: eine Anmeldung, einen Befehlskanal zur Steuerung und einen Datenkanal für den Inhalt. Diese Trennung ist bei FTP besonders deutlich und erklärt viele seiner Eigenheiten. Heute übernehmen oft andere Verfahren die Aufgabe, doch das Grundmodell ist überall ähnlich.

6.2 FTP

FTP ist in RFC 959 beschrieben. Der Client verbindet sich mit dem Server auf Port 21 und meldet sich an. Über diese Steuerverbindung laufen Befehle wie das Anmelden, das Auflisten eines Verzeichnisses oder das Wechseln des Ordners. Die eigentlichen Dateien fließen über eine zweite, separate Datenverbindung. Die Steuerverbindung bleibt dabei offen.

6.3 Kontrollverbindung

Die Kontrollverbindung trägt Befehle und Antworten als Text. Eine Antwort beginnt mit einem dreistelligen Code, etwa 230 für eine erfolgreiche Anmeldung oder 530 für eine abgelehnte. Der Client sendet zum Beispiel USER und PASS für die Anmeldung. Weil alles unverschlüsselt läuft, kann jeder im Netz diese Zugangsdaten mitlesen.

6.4 Datenverbindung

Für jede Übertragung einer Datei oder eines Verzeichnislistings entsteht eine eigene Datenverbindung. Sie wird geöffnet, der Inhalt fließt, und sie wird wieder geschlossen. Wie diese Verbindung aufgebaut wird, hängt vom Modus ab, und genau hier liegt die größte Stolperstelle.

6.5 Active Mode

Im aktiven Modus baut der Server die Datenverbindung selbst auf. Der Client teilt ihm über den Befehl PORT mit, auf welchem Port er auf die Verbindung wartet. Der Server verbindet sich dann von seinem Port 20 zu diesem Port des Clients. Das klingt einfach, scheitert aber oft an Firewalls und Adressumsetzung beim Client, denn eingehende Verbindungen sind dort meist blockiert.

6.6 Passive Mode

Im passiven Modus geht der Weg umgekehrt. Der Client sendet PASV oder das modernere EPSV. Der Server nennt daraufhin einen Port, auf dem er auf die Datenverbindung wartet. Der Client baut die Verbindung selbst auf. Da nun der Client die Verbindung öffnet, passt der passive Modus besser zu Firewalls, die nur ausgehende Verbindungen erlauben. Die Abbildung zeigt beide Modi.

6.7 FTP-Authentifizierung

Die Anmeldung erfolgt über Benutzername und Passwort. Anonymes FTP erlaubt den Zugriff mit dem Namen anonymous, oft ohne echtes Passwort. Das kann für öffentliche Downloads sinnvoll sein, muss aber sorgfältig eingerichtet werden, damit keine Schreibrechte entstehen.

6.8 Sicherheitsprobleme

FTP hat drei Schwächen. Die Zugangsdaten und die Dateien laufen im Klartext, jeder Beobachter im Netz kann beides lesen. Außerdem ist die Verbindung über zwei Kanäle schwer durch Firewalls zu führen. Und die Datenverbindung ist nicht an die

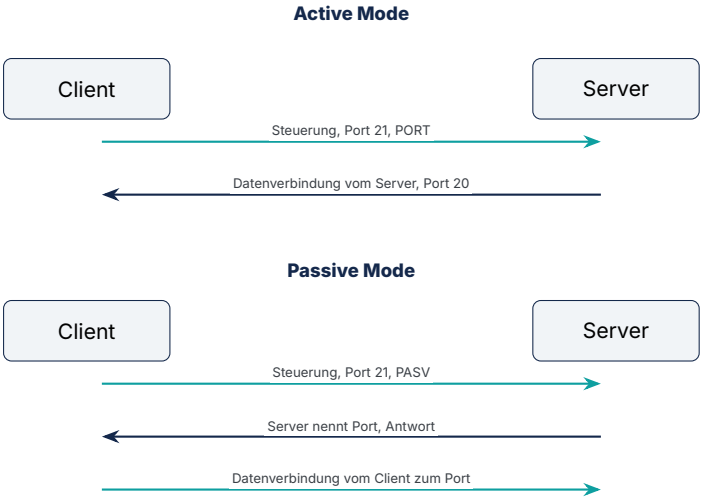


Abbildung 6.1. Active und Passive FTP im Vergleich. Im aktiven Modus verbindet sich der Server für die Daten, im passiven der Client. Die Ports sind vereinfacht. Schematische Darstellung.

Steuerverbindung gebunden, was in bestimmten Angriffsfällen ausgenutzt werden kann. Deshalb gilt FTP heute als unsicher, wenn es nicht durch Verschlüsselung ergänzt wird.

6.9 FTPS

FTPS ist FTP mit TLS-Verschlüsselung. Der Client verbindet sich auf Port 21, fordert mit AUTH TLS eine Verschlüsselung an, und die Steuer- und Datenverbindungen werden verschlüsselt. Die Grundlage bildet RFC 4217. Die ältere Variante mit einer Verschlüsselung von Beginn an auf Port 990 gilt als veraltet. FTPS behält das Modell mit zwei Verbindungen bei, und die Firewall-Probleme bleiben zum Teil bestehen.

6.10 SFTP

SFTP ist nicht FTP mit Verschlüsselung, sondern ein eigenes Protokoll, das innerhalb einer SSH-Verbindung läuft. Es nutzt Port 22, den auch SSH verwendet, und braucht deshalb nur eine einzige Verbindung. Die Spezifikation stammt aus dem Umfeld der

SSH-Arbeitsgruppe der IETF, ist aber kein RFC-Standard im engeren Sinne. SFTP bietet Dateiübertragung, Verzeichnisse und Rechteverwaltung, und die Anmeldung läuft über die Verfahren von SSH, die wir im nächsten Kapitel betrachten.

6.11 Unterschiede zwischen FTP, FTPS und SFTP

Die folgende Tabelle stellt die drei Verfahren gegenüber. Die wichtigste Unterscheidung ist die, dass FTPS und FTP dasselbe Protokollmodell haben, während SFTP ein anderes Modell mit einer einzigen Verbindung ist.

Merkmal	FTP	FTPS	SFTP
Basis	FTP	FTP mit TLS	SSH
Standardport	21, 20	21	22
Verbindungen	zwei	zwei	eine
Verschlüsselung	keine	TLS	SSH
Passwort im Klartext	ja	nein	nein
Firewall-Probleme	häufig	teilweise	selten

Tabelle 6.1. Gegenüberstellung von FTP, FTPS und SFTP.

6.12 Sichere Dateiübertragung

Für neue Umgebungen wird meist SFTP gewählt, weil es mit einer Verbindung auskommt und die Anmeldung über SSH-Schlüssel erlaubt. Wer bestehende FTP-Dienste nutzt, setzt FTPS ein. Die Auswahl hängt davon ab, was der Server anbietet, und wie sicher die Anbindung sein muss.

6.13 Anwendungsfälle

FTP und FTPS begegnen dir in älteren Webhosting-Umgebungen, bei Herstellern, die Firmware bereitstellen, und in Unternehmen mit altem Dokumentenaustausch. SFTP ist die Wahl für Server-Administration, automatisierte Übertragungen und Backups.

6.14 Praxis unter Windows und Linux

Unter Linux überträgt sftp Dateien interaktiv, scp kopiert einzelne Dateien. Ab OpenSSH-Version 9.0 nutzt scp intern das SFTP-Protokoll. Unter Windows lässt sich SFTP über Clients wie den in Windows integrierten OpenSSH-Client nutzen, der Befehl sftp steht in aktuellen Versionen zur Verfügung. Ein Klassiker zur Fehlersuche ist der Befehl ftp, der sich mit FTP verbindet und die Anmeldung zeigt, aber keine Verschlüsselung bietet.

6.15 Typische Verbindungsfehler

Häufige Fehler sind eine blockierte Datenverbindung im aktiven Modus, eine falsche Portnummer, ein Passwort, das nicht zum Konto passt, oder ein Server, dessen Zertifikat bei FTPS nicht geprüft werden kann. Bei FTP hilft oft der Wechsel in den passiven Modus. Bei SFTP ist meist die Schlüsselanmeldung der Ausgangspunkt der Suche, die das nächste Kapitel erklärt.

> **Merke**
>
> FTP trennt Steuerung und Daten in zwei Verbindungen und überträgt alles im Klartext. FTPS ergänzt TLS, SFTP läuft als eigenes Protokoll über eine SSH-Verbindung auf Port 22.

Kontrollfragen

1. Welcher Port dient bei FTP der Steuerung, und welcher der Datenübertragung im aktiven Modus?
2. Warum scheitert der aktive Modus oft hinter einer Firewall, der passive dagegen seltener?
3. Warum ist SFTP nicht einfach FTP mit Verschlüsselung?
4. Welche Vorteile hat SFTP gegenüber FTPS im Hinblick auf die Anzahl der Verbindungen?

Musterlösungen: 1. Port 21 für die Steuerung, Port 20 des Servers für die Daten im aktiven Modus. 2. Im aktiven Modus baut der Server eine eingehende Verbindung zum Client auf, die Firewalls oft blockieren. Im passiven Modus öffnet der Client die Verbindung selbst. 3. SFTP ist ein eigenes Protokoll, das in SSH läuft und ein anderes Befehlsmodell hat. FTPS ist dagegen FTP, das TLS nutzt. 4. SFTP braucht nur eine Verbindung über Port 22, dadurch gibt es weniger Firewall-Probleme und keine separate

Datenverbindung.

KAPITEL 7

SSH: Secure Shell

Wer einen Server verwaltet, sitzt selten davor. Der Server steht im Rechenzentrum oder in einer Cloud, und der Administrator arbeitet vom eigenen Rechner aus. Dafür braucht man eine Verbindung, mit der man Befehle sicher auf einem fremden Rechner ausführen kann. SSH, die Secure Shell, leistet genau das. Es verschlüsselt die gesamte Sitzung, prüft die Identität des Servers und erlaubt Anmeldung über Schlüssel statt Passwörter. Dieses Kapitel erklärt die Grundlagen, die Schlüsselverfahren und das Tunneln von Verbindungen.

7.1 Was ist SSH?

SSH ist ein Protokoll für verschlüsselte Fernanmeldung und sichere Kommunikation über ein unsicheres Netz. Die Architektur ist in RFC 4251 beschrieben, Transport, Anmeldung und Kanäle sind in den RFCs 4253, 4252 und 4254 festgelegt. SSH läuft standardmäßig auf Port 22 über TCP. Die bekannteste Umsetzung ist OpenSSH, das auf Linux, macOS und in aktuellen Windows-Versionen verfügbar ist.

7.2 Remote Administration

Mit SSH steuert ein Administrator einen entfernten Rechner, als säße er davor. Er führt Befehle aus, liest Logs, startet Dienste und ändert Konfigurationen. Die Eingabe und die Ausgabe laufen verschlüsselt. Ein einzelner Server kann so von überall aus gewartet werden, ohne dass jemand physisch vor Ort sein muss.

7.3 SSH-Client und SSH-Server

Der SSH-Client ist das Programm auf deinem Rechner, das die Verbindung aufbaut. Der SSH-Server ist der Dienst auf dem Zielrechner, der auf Port 22 lauscht und Anmeldungen annimmt. Auf Linux-Servern läuft der Dienst meist als sshd. Der Client heißt auf Linux ssh, unter Windows steht er in aktuellen Versionen als OpenSSH-Client zur Verfügung.

7.4 Port 22

Der Standardport 22 ist für SSH reserviert. Viele Angreifer suchen ihn gezielt ab, deshalb ist eine Änderung auf einen anderen Port manchmal sinnvoll, ersetzt aber keine echte Absicherung. Wichtiger ist die richtige Konfiguration des Dienstes.

7.5 SSH-Protokollaufbau

Eine SSH-Verbindung besteht aus mehreren Schichten. Zuerst handeln die Partner die Verschlüsselung und den Austausch der Schlüssel aus, das ist die Transportschicht. Danach melden sich die Benutzer an, das erledigt die Authentifizierungsschicht. Erst dann werden Kanäle geöffnet, über die Befehle, Dateien oder Tunnel laufen. Die Abbildung zeigt den Ablauf einer Verbindung.

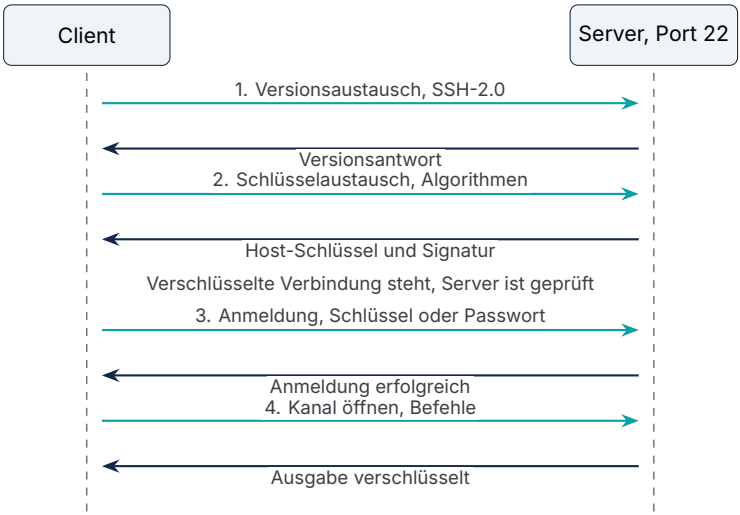


Abbildung 7.1. Ablauf einer SSH-Verbindung in vier Schritten. Zuerst entsteht die verschlüsselte Verbindung und der Server wird geprüft, danach folgt die Anmeldung und der Befehlskanal. Schematische Darstellung.

7.6 Host Keys

Jeder SSH-Server besitzt ein Schlüsselpaar, den Host Key. Der öffentliche Teil wird dem Client beim Verbindungsaufbau gezeigt. Der Client prüft damit, ob er wirklich mit dem richtigen Server spricht. Beim ersten Kontakt sieht man den Fingerabdruck des Schlüssels und bestätigt ihn. Ändert sich der Schlüssel später, warnt der Client, weil das auf einen Angriff hindeuten kann oder auf einen neu installierten Server.

7.7 Public-Key-Authentifizierung

Statt eines Passworts kann sich ein Benutzer mit einem Schlüsselpaar anmelden. Der öffentliche Schlüssel wird auf dem Server in der Datei authorized_keys hinterlegt, der private Schlüssel bleibt auf dem eigenen Rechner. Bei der Anmeldung beweist der Client, dass er den privaten Schlüssel besitzt, ohne ihn zu verraten. Die Abbildung zeigt das Prinzip.

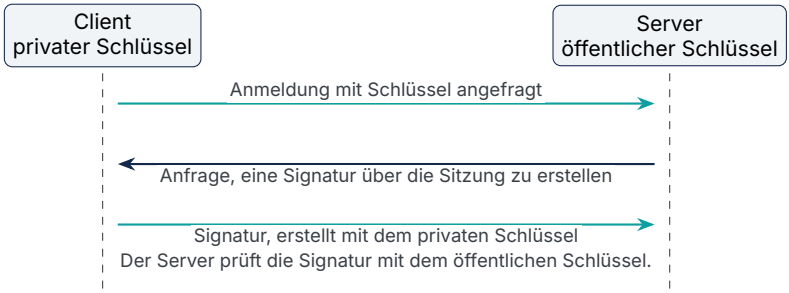


Abbildung 7.2. Public-Key-Anmeldung. Der Server verlangt eine Signatur über die aktuelle Sitzung. Nur der Besitzer des privaten Schlüssels kann sie erstellen. Der private Schlüssel verlässt den Client nicht. Schematische Darstellung.

7.8 Passwortauthentifizierung

SSH erlaubt auch Passwörter. Sie sind bequem, aber angreifbar, weil sie erraten oder durch automatisierte Versuche ausprobiert werden können. Viele Administratoren schalten Passwortanmeldung deshalb ab, sobald Schlüssel eingerichtet sind.

7.9 SSH-Key-Paare

Ein Schlüsselpaar erzeugt man mit ssh-keygen. Heute verwendet man meist ed25519, das kurze und sichere Schlüssel liefert. Die Datei des privaten Schlüssels sollte mit einer Passphrase geschützt werden. Der öffentliche Schlüssel endet auf .pub und kann gefahrlos weitergegeben werden.

7.10 Known Hosts

Der Client merkt sich die Host Keys der Server, mit denen er schon verbunden war, in der Datei known_hosts. Bei jeder neuen Verbindung vergleicht er den Schlüssel mit dem gespeicherten. Die Datei ist der Schutz gegen unbemerkte Schlüsselwechsel und sollte nicht ohne Prüfung geändert werden.

7.11 SSH-Agent

Der SSH-Agent hält entsperrte private Schlüssel im Speicher. Die Passphrase muss man dann nicht bei jeder Verbindung eingeben. Der Agent läuft im Hintergrund und beantwortet die Signaturanfragen des Clients.

7.12 SSH-Tunneling und Port Forwarding

Eine SSH-Verbindung kann andere Verbindungen tunneln. Ein lokaler Port wird über den verschlüsselten Kanal zu einem Ziel auf der Gegenseite geleitet. Das ist nützlich, wenn ein Dienst nur innerhalb eines Netzes erreichbar ist. Die Abbildung zeigt einen lokalen Tunnel.

Daneben gibt es das Remote Forwarding, bei dem der Server einen Port öffnet, der zu einem Ziel auf der Client-Seite führt, und das dynamische Forwarding, das einen SOCKS-Proxy bereitstellt. Die Optionen dafür sind -L, -R und -D.

7.13 SCP und SFTP

SCP kopiert Dateien über SSH. Ältere Versionen nutzten ein eigenes, unsicher gewordenes Protokoll. Ab OpenSSH 9.0 läuft scp über das SFTP-Protokoll. SFTP selbst ist die interaktive und programmierbare Variante, wie wir im vorigen Kapitel gesehen haben.

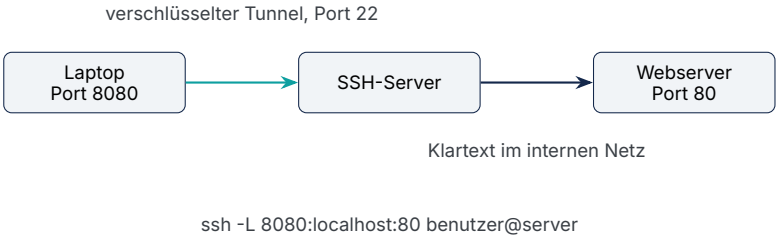


Abbildung 7.3. Lokales Port Forwarding über SSH. Die Anfrage an den lokalen Port 8080 läuft verschlüsselt zum Server, der sie an den Dienst auf Port 80 weiterleitet. Schematische Darstellung.

7.14 Sicherheitsmaßnahmen

Wichtige Maßnahmen sind das Abschalten der Anmeldung als root, das Abschalten von Passwortanmeldung, die Verwendung von Schlüsseln mit Passphrase, das Begrenzen der erlaubten Benutzer und eine Firewall, die Port 22 nur für bekannte Netze öffnet. Auch regelmäßige Updates des Dienstes gehören dazu. Die Konfiguration steht in der Datei sshd_config.

7.15 Praxis mit SSH

Ein Schlüssel entsteht mit ssh-keygen -t ed25519. Mit ssh-copy-id benutzer@server legt man den öffentlichen Schlüssel auf dem Server ab. Danach meldet sich ssh benutzer@server ohne Passwort an. Der Befehl ssh -v zeigt den Ablauf des Handshakes im Detail. Die Laborübungen führen diese Schritte aus.

7.16 Typische Fehler

Häufige Fehler sind eine geänderte Host-Schlüssel-Warnung, ein falsch gesetzter Dateirechte für den privaten Schlüssel, der von SSH abgelehnt wird, eine fehlende Freigabe in der Firewall oder ein Benutzer, der sich nicht anmelden darf. Die Warnung zu geänderten Host Keys ist ein Sicherheitssignal und sollte nicht einfach überschrieben werden.

Merke

SSH verschlüsselt die gesamte Sitzung, prüft den Server über seinen Host Key und meldet Benutzer am besten über Schlüssel an. Tunnel erlauben, andere Dienste über die verschlüsselte Verbindung zu erreichen.

Kontrollfragen

1. Welche Aufgabe hat der Host Key, und was bedeutet eine Warnung, dass er sich geändert hat?
2. Warum ist die Public-Key-Anmeldung sicherer als eine Passwortanmeldung?
3. Welcher Teil eines Schlüsselpaars bleibt auf dem Client, und welcher wird auf dem Server hinterlegt?
4. Mit welchem Befehl richtet man einen lokalen Tunnel vom Port 8080 zum Port 80 eines Servers ein?

Musterlösungen: 1. Der Host Key beweist die Identität des Servers. Eine Änderung kann auf einen Angriff hindeuten oder auf einen neu installierten Server, deshalb muss man sie prüfen, bevor man sie akzeptiert. 2. Der private Schlüssel verlässt den Client nie, und ein Angreifer kann kein Passwort erraten. Ein Passwort lässt sich dagegen ausprobieren. 3. Der private Schlüssel bleibt auf dem Client, der öffentliche Schlüssel wird auf dem Server in authorized_keys hinterlegt. 4. ssh -L 8080:localhost:80 benutzer@server.

KAPITEL 8

E-Mail-Protokolle

Eine E-Mail durchläuft auf dem Weg vom Absender zum Empfänger mehrere Stationen, und jede Station spricht ein eigenes Protokoll. Dein Mailprogramm übergibt die Nachricht an einen Server, dieser sucht den Mailserver des Empfängers, und dieser legt die Nachricht im Postfach ab, aus dem das Mailprogramm des Empfängers sie holt. Das wirkt wie ein einziger Vorgang, besteht aber aus mehreren Schritten mit unterschiedlichen Protokollen. Dieses Kapitel zeigt diese Schritte, die Rollen der Programme und die Verfahren, mit denen Empfänger heute Fälschungen erkennen.

8.1 Wie funktioniert E-Mail?

Eine E-Mail hat zwei Teile: einen Umschlag, der bestimmt, wohin die Nachricht geht, und den Inhalt mit Betreff, Text und Anhängen. Der Umschlag wird beim Versand verwendet, der Inhalt beim Lesen. Diese Trennung ist wichtig, denn der Absender im Briefkopf und der Absender im Umschlag müssen nicht übereinstimmen.

Die Zustellung läuft über das Internet ähnlich wie ein Brief über die Post. Die Nachricht wird weitergereicht, bis sie beim Empfängerserver ankommt. Dort liegt sie, bis der Empfänger sie abholt.

8.2 SMTP

SMTP, das Simple Mail Transfer Protocol, ist das Protokoll für den Versand. Es ist in RFC 5321 festgelegt. Der Client sendet eine Nachricht an einen Server, und Server reichen sie an andere Server weiter. Die Kommunikation besteht aus Befehlen wie EHLO, MAIL FROM, RCPT TO und DATA, auf die der Server mit dreistelligen Codes antwortet. Der Code 250 bestätigt eine erfolgreiche Anweisung.

Für den Versand von einem Programm zum eigenen Mailserver nutzt man Port 587, die sogenannte Submission. Port 25 bleibt dem Austausch zwischen Servern vorbehalten. Verschlüsselte Verbindungen laufen bei SMTP über STARTTLS oder über Port 465.

8.3 Mailserver

Ein Mailserver nimmt Nachrichten an, leitet sie weiter oder legt sie in Postfächern ab. Die Software übernimmt damit mehrere Rollen, die man getrennt betrachtet. Die folgenden Begriffe beschreiben diese Rollen.

8.4 Mail User Agent

Der Mail User Agent, kurz MUA, ist das Programm, das der Benutzer bedient. Dazu gehören Outlook, Thunderbird oder die Mailapp auf dem Smartphone. Der MUA verfasst die Nachricht, zeigt sie an und spricht mit dem Server.

8.5 Mail Transfer Agent

Der Mail Transfer Agent, kurz MTA, nimmt Nachrichten an und leitet sie weiter. Er sucht den nächsten Server anhand der Empfängeradresse und sendet die Nachricht per SMTP. Ein MTA ist die Stelle, an der die Weiterleitung über das Netz geschieht.

8.6 Mail Delivery Agent

Der Mail Delivery Agent, kurz MDA, legt die Nachricht im Postfach des Empfängers ab. Er ist der letzte Schritt auf dem Server. Der MDA stellt sicher, dass die Nachricht dort liegt, wo der Empfänger sie abholen kann.

8.7 MX-Records

Woher weiß ein Server, an welchen Rechner er eine Nachricht für example.org schicken soll? Die Antwort liefern MX-Records im DNS, die wir in Kapitel 2 kennengelernt haben. Der MX-Record nennt den Mailserver der Domain und eine Priorität. Der Absender versucht zuerst den Server mit der niedrigsten Zahl, bei Ausfall die nächsten.

8.8 SMTP-Relay

Ein Relay ist ein Server, der Nachrichten für andere weiterleitet. Offene Relays, die jeden Versand annehmen, werden von Spammern missbraucht und landen schnell auf Sperrlisten. Deshalb nehmen Mailserver Nachrichten zum Versand nur von angemeldeten Benutzern an und leiten Nachrichten für fremde Empfänger nur weiter, wenn es erlaubt ist.

8.9 Submission

Submission bezeichnet den Versand vom Mailprogramm zum eigenen Server, über Port 587 mit Anmeldung und Verschlüsselung. Die Trennung von Port 25 hat den Vorteil, dass Benutzer strenger geprüft werden als Server untereinander. Viele Mailprogramme verlangen deshalb eine Anmeldung, bevor sie senden.

Die folgende Abbildung zeigt den Ablauf einer SMTP-Sitzung zum Versand mit den wichtigsten Befehlen.

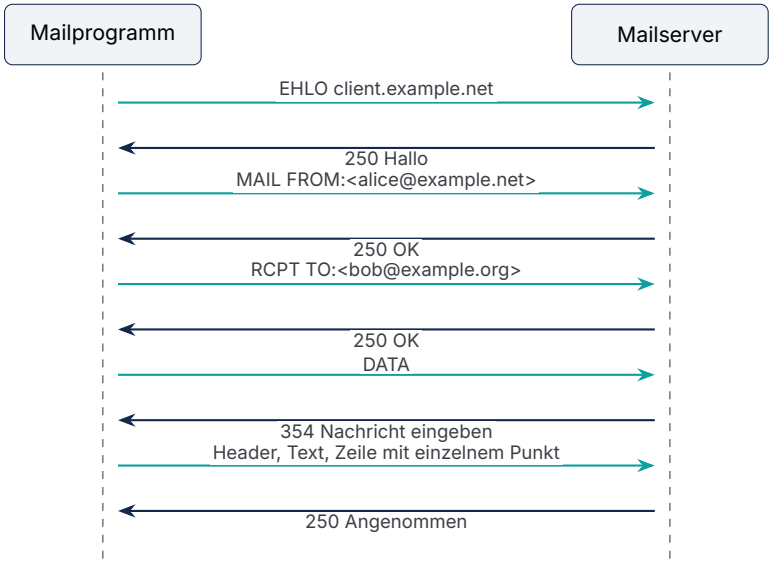
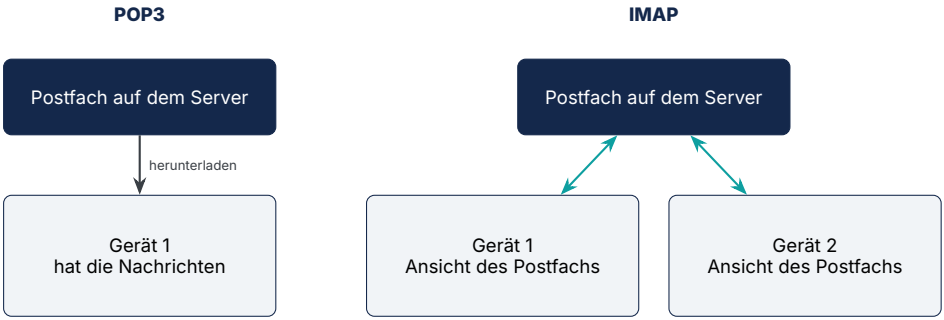


Abbildung 8.1. SMTP-Sitzung zum Versand einer Nachricht. Der Client nennt sich, den Absender und den Empfänger, sendet dann den Inhalt. Die Codes 250 und 354 sind Antworten des Servers. Schematische Darstellung.

8.10 POP3 und IMAP

Zum Abruf gibt es zwei Protokolle. POP3 lädt die Nachrichten auf das Gerät und löscht sie oft auf dem Server. Es ist in RFC 1939 beschrieben und nutzt Port 110, verschlüsselt über Port 995. IMAP lässt die Nachrichten auf dem Server und synchronisiert den Zustand zwischen Geräten. Es ist in RFC 9051 als IMAP4rev2 festgelegt und nutzt Port 143, verschlüsselt über Port 993.

Die Abbildung vergleicht beide Ansätze.



POP3 lädt Nachrichten auf ein Gerät. IMAP lässt sie auf dem Server und synchronisiert.

Abbildung 8.2. POP3 und IMAP im Vergleich. Bei POP3 landen die Nachrichten auf einem Gerät, bei IMAP bleiben sie auf dem Server und sind von mehreren Geräten aus sichtbar. Schematische Darstellung.

Für mehrere Geräte ist IMAP die bessere Wahl, weil Ordner, Lesestatus und Suche überall gleich sind. POP3 eignet sich für einfache Umgebungen mit einem einzigen Gerät und wenig Speicher auf dem Server.

8.11 MIME

Eine E-Mail enthält nicht nur Text. Anhänge, Bilder und unterschiedliche Zeichensätze brauchen ein Format, das SMTP überträgt. MIME, die Multipurpose Internet Mail Extensions, beschreibt dieses Format in den RFCs 2045 bis 2049. Die Nachricht wird in Teile zerlegt, die jeweils einen Typ und eine Kodierung tragen.

8.12 STARTTLS

STARTTLS startet eine unverschlüsselte Sitzung auf Port 25 oder 587 und wechselt dann auf TLS. Der Client fragt den Server nach dieser Möglichkeit, bevor er Daten sendet. Daneben gibt es SMTPS auf Port 465, das die Verschlüsselung von Anfang an verlangt. Beide Verfahren sind verbreitet, und ein sicherer Mailverkehr setzt auf eines davon.

8.13 SPF

Sender Policy Framework, kurz SPF, legt in einem TXT-Record der Domain fest, welche Server im Namen der Domain Mails senden dürfen. Der Empfänger prüft, ob der sendende Server auf dieser Liste steht. SPF ist in RFC 7208 beschrieben. Es prüft den Umschlagabsender, nicht den Absender im Briefkopf.

8.14 DKIM

DomainKeys Identified Mail, kurz DKIM, signiert Nachrichten mit einem privaten Schlüssel des Absenders. Den öffentlichen Schlüssel findet der Empfänger im DNS, als TXT-Record. Stimmt die Signatur, ist die Nachricht unterwegs nicht verändert worden und stammt vom Besitzer der Domain. DKIM ist in RFC 6376 festgelegt.

8.15 DMARC

Domain-based Message Authentication, Reporting and Conformance, kurz DMARC, verbindet SPF und DKIM und legt fest, was der Empfänger bei einem Fehlschlag tun soll: nichts, markieren oder verwerfen. DMARC ist in RFC 7489 beschrieben und liefert Berichte an den Domainbesitzer. Es verbindet die Prüfung mit der Angabe, welcher Absender im Briefkopf zählt.

Die folgende Abbildung zeigt das Zusammenspiel der drei Verfahren.

8.16 E-Mail-Zustellung

Die Zustellung kann scheitern, wenn ein Server nicht erreichbar ist, wenn die Prüfungen fehlschlagen oder wenn der Empfänger den Absender als Spam einstuft. Der sendende Server versucht es später erneut und meldet nach einer gewissen Zeit eine Unzustell-



Beide Prüfungen liefern Ergebnisse, DMARC entscheidet, was damit geschieht.

Abbildung 8.3. Zusammenspiel von SPF, DKIM und DMARC. SPF und DKIM prüfen unabhängig voneinander, DMARC bewertet beide Ergebnisse und legt die Reaktion fest. Schematische Darstellung.

barkeit an den Absender zurück. Diese Meldung trägt oft den Code 4xx für vorübergehende Fehler und 5xx für endgültige.

8.17 Spam-Schutz

Neben SPF, DKIM und DMARC helfen Sperrlisten, die Prüfung von Verbindungsparametern und Filter für den Inhalt. Für den Betreiber ist wichtig, dass sein Server nicht zum offenen Relay wird und dass die DNS-Einträge für Absenderprüfung vollständig sind.

8.18 Praxis mit E-Mail-Protokollen

Die MX-Records einer Domain fragt man mit nslookup -type=MX example.org ab, unter Linux mit dig example.org MX. Ein Blick in die Kopfzeilen einer Nachricht zeigt den Weg über die Server und die Ergebnisse von SPF, DKIM und DMARC. Ob ein Server Verbindungen über Port 587 mit STARTTLS annimmt, testet man mit openssl s_client -starttls smtp -connect mail.example.org:587. Die Laborübungen führen diese Schritte aus.

> **Merke**
>
> E-Mail besteht aus mehreren Stationen. MUA verfasst und zeigt an, MTA leitet weiter, MDA legt im Postfach ab. SMTP versendet, POP3 und IMAP holen ab. SPF, DKIM und DMARC prüfen die Absenderangaben.

Kontrollfragen

1. Welche Rolle hat der MX-Record bei der Zustellung?
2. Welcher Port dient dem Versand vom Mailprogramm zum eigenen Server, und welcher dem Austausch zwischen Servern?
3. Warum ist IMAP für mehrere Geräte besser geeignet als POP3?
4. Was prüft SPF, und was prüft DKIM?

Musterlösungen: 1. Der MX-Record nennt den Mailserver der Domain, an den der Absender die Nachricht schickt. 2. Port 587 für den Versand vom Mailprogramm zum eigenen Server, Port 25 für den Austausch zwischen Servern. 3. IMAP lässt die Nachrichten auf dem Server, sodass alle Geräte denselben Stand sehen. POP3 lädt sie auf ein Gerät. 4. SPF prüft, ob der sendende Server für die Domain berechtigt ist. DKIM prüft, ob die Signatur der Nachricht stimmt und die Nachricht unverändert ist.

KAPITEL 9

NTP: Network Time Protocol

Ein Computer ohne richtige Uhrzeit verursacht Probleme, die man lange nicht bemerkt. Logdateien auf mehreren Servern lassen sich nicht mehr in die richtige Reihenfolge bringen, Zertifikate gelten plötzlich nicht mehr, und Anmeldungen über Kerberos scheitern, weil die Zeitdifferenz zu groß ist. Die Zeit muss also auf allen Geräten gleich sein. NTP, das Network Time Protocol, sorgt dafür. Dieses Kapitel erklärt, wie NTP die Uhr synchronisiert, welche Rolle die Stratum-Ebenen spielen und wie man den Zustand prüft.

9.1 Warum ist Zeitsynchronisation wichtig?

Fast alle Sicherheits- und Verwaltungsmechanismen hängen an der Zeit. Zertifikate haben Gültigkeitszeiträume, Kerberos-Tickets laufen ab, Protokolle sortieren Ereignisse nach Zeitstempeln und Backups entscheiden nach Datum. Weicht eine Uhr um Minuten ab, funktionieren diese Mechanismen nicht mehr richtig. Bei Untersuchungen nach Angriffen oder Störungen ist eine korrekte Zeit oft der wichtigste Hinweis.

9.2 NTP

NTP ist ein Protokoll, mit dem ein Gerät seine Uhr an einen genauen Zeitgeber angleicht. Es ist in RFC 5905 als NTPv4 beschrieben und läuft über UDP auf Port 123. Der Client fragt einen Server nach der Zeit, misst die Laufzeit der Antwort und korrigiert seine Uhr entsprechend.

9.3 UTC

Die Zeit im NTP-Protokoll ist UTC, die koordinierte Weltzeit. Lokale Zeitzonen und Sommerzeit werden erst auf dem Gerät für die Anzeige berechnet. Ein Server rechnet intern immer in UTC, damit Zeitstempel überall vergleichbar bleiben.

9.4 Stratum

Die Genauigkeit wird in Ebenen gemessen, den Stratum-Stufen. Ein Gerät mit Stratum 0 ist eine Referenzquelle, etwa eine Atomuhr oder ein GPS-Empfänger. Ein Server mit Stratum 1 hängt direkt an einer solchen Quelle. Jede weitere Stufe ist einen Schritt weiter entfernt. Ein Client mit Stratum 2 bezieht seine Zeit von einem Stratum-1-Server. Der Wert 16 bedeutet, dass ein Gerät nicht synchronisiert ist.

9.5 NTP-Hierarchie

Die folgende Abbildung zeigt die Stratum-Ebenen mit einem Beispiel.

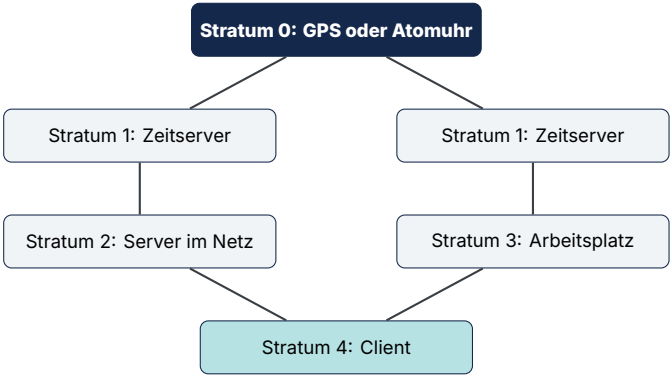


Abbildung 9.1. NTP-Hierarchie mit Stratum-Ebenen. Jede Stufe bezieht ihre Zeit von der darüberliegenden. Die Stratum-Zahl steigt mit dem Abstand zur Referenzquelle. Schematische Darstellung.

9.6 NTP-Server und NTP-Clients

Ein NTP-Server stellt die Zeit bereit, ein NTP-Client fragt sie ab. Die meisten Geräte sind Clients, einige Server im Netz verteilen die Zeit weiter. Öffentliche Zeitserver werden von Organisationen betrieben, Betriebssysteme haben eigene Standardquellen eingetragen.

9.7 Zeitabweichung

Eine Uhr weicht immer etwas ab. Diese Abweichung heißt Offset. Eine zweite Größe ist die Laufzeit der Nachrichten zwischen Client und Server, die Delay genannt wird. NTP schätzt beide Werte aus vier Zeitstempeln und wählt den zuverlässigsten Server aus mehreren.

9.8 Zeitkorrektur

Ist die Abweichung klein, läuft die Uhr langsamer oder schneller weiter, bis sie passt. Ist sie groß, springt sie auf die richtige Zeit. Ein Sprung kann Anwendungen verwirren, deshalb korrigieren Systeme möglichst sanft. Die Korrektur erfolgt dabei nie blind, sondern nach der Messung über mehrere Abfragen.

9.9 NTP-Nachrichtenaustausch

Der Austausch besteht aus einer Anfrage und einer Antwort. Der Client notiert vier Zeitstempel: T1 beim Absenden der Anfrage, T2 beim Empfang durch den Server, T3 beim Absenden der Antwort und T4 beim Empfang durch den Client. Aus diesen Werten berechnen sich Offset und Laufzeit. Die folgende Abbildung zeigt den Ablauf mit Beispielwerten in Millisekunden.

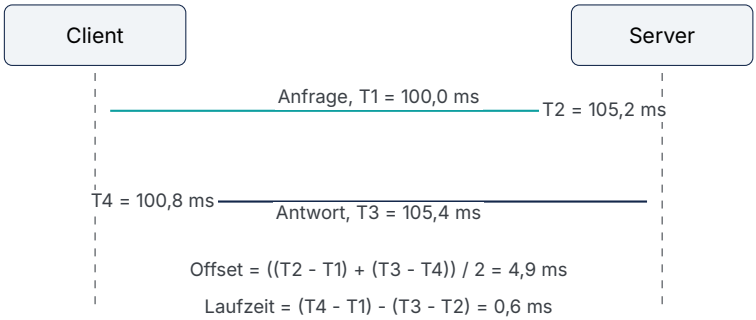


Abbildung 9.2. Nachrichtenaustausch bei NTP mit vier Zeitstempeln. Aus ihnen berechnet der Client den Offset der Uhr und die Laufzeit. Die Werte sind Beispiele. Schematische Darstellung.

Der Offset gibt an, um wie viel die Uhr des Clients vom Server abweicht. Die Laufzeit

misst die Verzögerung im Netz und erlaubt, die Messung zu gewichten. Ein großer Unterschied zwischen beiden Richtungen der Laufzeit verfälscht den Offset, deshalb schätzt NTP ihn nur ungefähr.

9.10 NTP-Sicherheit

NTP ist ein Ziel für Angriffe. Wer falsche Zeit liefert, kann Zertifikatsprüfungen und Logs verfälschen oder Anmeldungen stören. Schutz bieten die Begrenzung auf vertrauenswürdige Server, die Prüfung mehrerer Quellen und die Absicherung der Nachrichten. Eine Angriffsfläche ist auch der Verstärkungsangriff, bei dem offene NTP-Server für Überflutungsangriffe missbraucht werden. Deshalb sollte ein Server nur den eigenen Netzen Antworten geben.

9.11 NTS als Einführung

Network Time Security, kurz NTS, ergänzt NTP um Authentifizierung und Verschlüsselung. Die Spezifikation ist in RFC 8915 beschrieben. NTS schützt die Zeitnachrichten gegen Manipulation, ohne den Ablauf von NTP grundlegend zu verändern.

9.12 Zeitsynchronisation unter Windows und Linux

Unter Windows steuert der Zeitdienst die Synchronisation. Der Befehl w32tm /query /status zeigt Quelle, Stratum und letzte Synchronisation. Mit w32tm /resync stößt man eine Synchronisation an. Unter Linux arbeiten moderne Systeme mit systemd-timesyncd oder mit chrony. Der Befehl timedatectl zeigt die Zeitzone und den Synchronisationsstatus. Mit chronyc tracking erfährt man Offset, Stratum und Quelle im Detail.

9.13 Typische Fehler

Häufige Fehler sind ein blockierter UDP-Port 123 in der Firewall, ein falsch konfigurierter Zeitserver, eine Virtualisierung, die die Uhr des Gastes nicht richtig führt, oder eine Zeitzone, die mit der UTC-Angabe verwechselt wird. Eine Abweichung von wenigen Minuten erkennt man oft erst daran, dass Zertifikate oder Anmeldungen plötzlich scheitern.

Merke

NTP synchronisiert Uhren über UDP auf Port 123 und rechnet intern in UTC. Die Stratum-Stufen zeigen den Abstand zur Referenzquelle, Offset und Laufzeit werden aus vier Zeitstempeln berechnet.

Kontrollfragen

1. Welche Stratum-Stufe hat ein Server, der direkt an einer GPS-Uhr hängt?
2. Warum rechnet NTP intern in UTC?
3. Welche vier Zeitstempel braucht NTP für die Berechnung, und was misst die Laufzeit?
4. Warum sollte ein NTP-Server nicht allen Anfragen antworten?

Musterlösungen: 1. Stratum 1. 2. Damit alle Geräte dieselbe Zeitbasis haben und Zeitstempel überall vergleichbar sind. Zeitzonen werden erst bei der Anzeige berechnet. 3. T1 beim Absenden der Anfrage, T2 beim Empfang durch den Server, T3 beim Absenden der Antwort, T4 beim Empfang durch den Client. Die Laufzeit misst die Verzögerung im Netz ohne die Verarbeitungszeit des Servers. 4. Ein offener Server kann für Verstärkungsangriffe missbraucht werden, die fremde Netze überlasten.

KAPITEL 10

SNMP: Simple Network Management Protocol

Ein Netz mit Dutzenden Switches, Routern, Druckern und Servern lässt sich nicht von Hand beobachten. Wer wissen will, ob ein Gerät ausgelastet ist, ob ein Port ausgefallen ist oder ob die Temperatur im Schrank steigt, braucht eine automatische Abfrage. SNMP, das Simple Network Management Protocol, ist seit Jahrzehnten der Standard dafür. Es erlaubt, Geräte abzufragen und ihnen Werte zu setzen, und es meldet Ereignisse von sich aus. Dieses Kapitel erklärt den Aufbau, die Versionen und die Sicherheitsfragen, die man beim Einsatz kennen muss.

10.1 Netzwerküberwachung

Überwachung beantwortet drei Fragen: Läuft alles, wie es soll? Gibt es Engpässe? Wann tritt ein Fehler auf? Dafür sammelt man Messwerte wie Datenmengen pro Port, Auslastung der Prozessoren, Fehlerzähler oder Laufzeit seit dem letzten Start. Ein Überwachungssystem sammelt diese Werte regelmäßig und warnt, wenn Grenzen überschritten werden.

10.2 SNMP

SNMP ist ein Protokoll, mit dem eine zentrale Stelle Geräte abfragt und konfiguriert. Die Architektur ist in RFC 3411 beschrieben. Die Kommunikation läuft über UDP, Anfragen und Antworten meist auf Port 161, Traps vom Gerät zur Zentrale auf Port 162.

10.3 SNMP-Manager

Der SNMP-Manager ist die Zentrale. Er stellt Abfragen, sammelt die Antworten, speichert sie und stellt sie dar. In Unternehmen ist das meist ein Monitoring-System.

10.4 SNMP-Agent

Der SNMP-Agent läuft auf dem überwachten Gerät. Er kennt die Werte des Geräts, beantwortet Abfragen und kann Ereignisse melden. Router, Switches, Drucker und viele Server bringen einen Agenten mit.

10.5 MIB

Die Management Information Base, kurz MIB, ist eine Beschreibung aller Werte, die ein Gerät bereitstellt. Sie ist als Baum aufgebaut. Jeder Zweig hat eine Nummer, und die Hierarchie legt fest, was ein Wert bedeutet. Herstellerspezifische Werte liegen in eigenen Zweigen.

10.6 OID

Ein Object Identifier, kurz OID, ist die Adresse eines Werts im MIB-Baum, geschrieben als Folge von Zahlen. Die Adresse 1.3.6.1.2.1.1.1.0 bezeichnet die Systembeschreibung des Geräts, 1.3.6.1.2.1.1.5.0 den Namen. Mit der OID fragt der Manager genau den Wert ab, den er braucht.

Die folgende Abbildung zeigt einen Ausschnitt des Baums mit den Zweigen, die in der Praxis am häufigsten vorkommen.

10.7 GET, GETNEXT und GETBULK

GET fragt einen bestimmten Wert ab. GETNEXT liefert den nächsten Wert im Baum, damit der Manager eine ganze Gruppe durchlaufen kann. GETBULK holt in einer Anfrage viele Werte auf einmal und ist effizienter. Diese Befehle lesen nur, sie verändern nichts.

10.8 SET

SET schreibt einen Wert in das Gerät, etwa einen Namen oder eine Konfigurationsoption. Mit SET lassen sich Geräte konfigurieren, es ist daher besonders zu schützen.

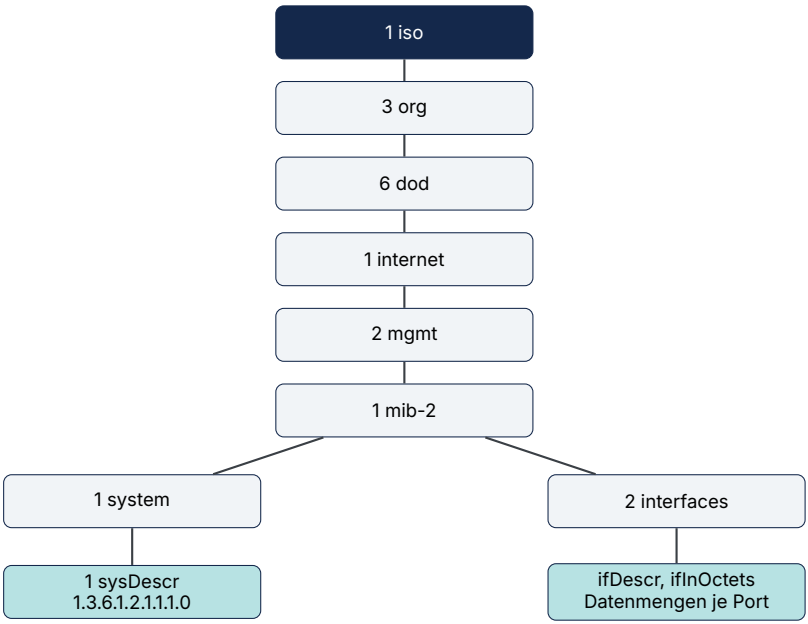


Abbildung 10.1. Ausschnitt des MIB-Baums. Die Zahlen auf jedem Knoten bilden die OID. Die Systembeschreibung und die Interface-Zähler liegen unter mib-2. Schematische Darstellung.

10.9 TRAP und INFORM

Ein Trap ist eine Meldung, die der Agent von sich aus an den Manager schickt, etwa wenn ein Port ausfällt. Er wird nicht bestätigt. INFORM ist eine ähnliche Meldung, die der Manager bestätigt. Beide erleichtern die Überwachung, weil der Manager nicht ständig abfragen muss.

Die folgende Abbildung zeigt eine Abfrage mit Antwort und einen Trap.

10.10 SNMPv1

SNMPv1 ist die erste Version und in RFC 1157 beschrieben. Sie kennt nur einfache Gemeinschaftsnamen, die sogenannten Community Strings, als Passwortersatz. Diese Namen werden im Klartext übertragen. Die Version gilt als unsicher und sollte nicht mehr

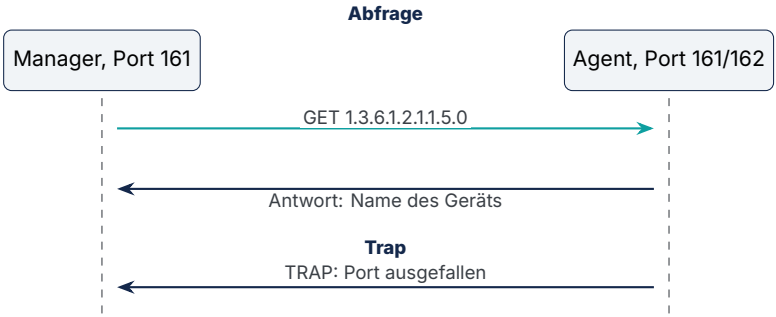


Abbildung 10.2. Abfrage und Trap im Vergleich. Der Manager fragt aktiv nach einem Wert, der Agent meldet ein Ereignis von sich aus. Schematische Darstellung.

verwendet werden.

10.11 SNMPv2c

SNMPv2c erweitert v1 um GETBULK und INFORM und verwendet ebenfalls Community Strings. Der Zusatz c steht für community. Die Sicherheit entspricht im Wesentlichen der von v1, weshalb auch v2c nur in abgeschotteten Netzen eingesetzt werden sollte.

10.12 SNMPv3

SNMPv3 bringt echte Sicherheit. Es unterstützt Authentifizierung, also den Nachweis der Identität, und Verschlüsselung der Nachrichten. Die Benutzerverwaltung erfolgt über das User-based Security Model. Die Festlegung liegt in RFC 3414. Für jeden neuen Einsatz sollte SNMPv3 gewählt werden.

10.13 Authentifizierung

Bei SNMPv3 meldet sich der Manager mit einem Benutzernamen und einem Schlüssel an. Die Nachricht trägt eine Signatur, die zeigt, dass sie vom berechtigten Manager stammt und unterwegs nicht verändert wurde. Ohne gültige Anmeldung bleibt der Agent stumm.

10.14 Verschlüsselung

Zusätzlich zur Authentifizierung kann SNMPv3 die Inhalte verschlüsseln. Dann liest ein Beobachter weder die Werte noch die Konfigurationsbefehle. Das ist wichtig, weil SNMP-Antworten oft Angaben über Netztopologie und Software enthalten, die für Angreifer wertvoll sind.

10.15 Monitoring

In der Praxis sammelt ein Monitoring-System regelmäßig Werte wie Auslastung, Speicher oder Temperatur und stellt sie als Verlauf dar. Schwellwerte lösen Warnungen aus. So erkennt man einen überlasteten Port, bevor ein Benutzer sich beschwert.

10.16 SNMP-Grundlagen unter Linux

Unter Linux bietet das Paket net-snmp die Werkzeuge snmpget, snmpwalk und snmpset. Mit snmpwalk -v2c -c public 127.0.0.1 läuft man den Baum eines lokalen Agenten durch, wenn er so konfiguriert ist. Die Option -v legt die Version fest, -c den Community-Namen. In einer Laborumgebung ist das ein gutes Werkzeug zum Verständnis, in produktiven Netzen darf man keine Standardnamen wie public stehen lassen.

10.17 Sicherheitskonfiguration

Wichtige Maßnahmen sind SNMPv3 statt v1 oder v2c, starke Schlüssel, der Verzicht auf den Standardnamen public, die Begrenzung der Zugriffe auf bekannte Manager über Firewall und Konfiguration, der Verzicht auf SET, wo es nicht gebraucht wird, und ein Schutz der Ports 161 und 162. Ein öffentlich erreichbarer SNMP-Agent ist ein häufiges Angriffsziel und sollte im Internet nie offen sein.

10.18 Typische Fehler

Häufige Fehler sind ein falscher Community-Name, eine falsche Version auf Manager und Agent, eine blockierte UDP-Verbindung auf Port 161 oder ein Agent, der nicht aktiviert ist. Auch eine falsche OID führt zu einer leeren Antwort, ohne dass ein Fehler auftritt.

Merke

SNMP fragt Geräte über eine MIB mit OIDs ab. GET liest, SET schreibt, Traps melden Ereignisse. Für echte Sicherheit nutzt man SNMPv3 mit Authentifizierung und Verschlüsselung.

Kontrollfragen

1. Was bezeichnet die OID, und wozu dient die MIB?
2. Warum sollte man SNMPv1 und v2c nicht über ungeschützte Netze betreiben?
3. Worin unterscheidet sich ein Trap von einer GET-Abfrage?
4. Welche Version wählst du für einen neuen Einsatz, und warum?

Musterlösungen: 1. Die OID ist die Adresse eines Werts im MIB-Baum. Die MIB beschreibt alle Werte und ihre Bedeutung. 2. Die Community Strings werden im Klartext übertragen und bieten keine echte Sicherheit. Wer sie mitliest, kann Geräte abfragen oder bei SET sogar verändern. 3. Der Manager fragt bei GET aktiv nach einem Wert, der Agent sendet einen Trap von sich aus bei einem Ereignis. 4. SNMPv3, weil es Authentifizierung und Verschlüsselung bietet.

KAPITEL 11

SMB und Netzwerkfreigaben

In vielen Büros liegen Dokumente nicht auf jedem Arbeitsplatzrechner, sondern auf einem zentralen Server, den alle Mitarbeiter über das Netz öffnen. Unter Windows geschieht das über Netzwerkfreigaben, und das dahinterliegende Protokoll heißt SMB, Server Message Block. SMB erlaubt, Ordner und Drucker im Netz anzubieten und darauf zuzugreifen, als lägen sie lokal. Dieses Kapitel erklärt, wie SMB arbeitet, welche Versionen es gibt, wie Berechtigungen gestuft sind und welche Risiken bestehen.

11.1 Was ist SMB?

SMB ist ein Protokoll für Datei- und Druckerfreigaben im lokalen Netz und darüber hinaus. Es wurde von Microsoft entwickelt und wird heute auch von Samba auf Linux umgesetzt. SMB läuft über TCP, direkt auf Port 445. Ältere Umsetzungen nutzten zusätzlich NetBIOS über die Ports 137 bis 139, das gilt heute als veraltet.

11.2 Datei- und Druckerfreigaben

Eine Freigabe macht einen Ordner oder Drucker im Netz sichtbar und regelt den Zugriff. Der Server bietet eine Freigabe mit einem Namen an, etwa Projekte. Der Client verbindet sich mit dieser Freigabe, und der Zugriff erfolgt dann über den Netzwerkpfad, zum Beispiel \\server\Projekte, wobei der Pfad für die Nutzer wie ein lokaler Ordner aussieht.

11.3 SMB-Client und SMB-Server

Der SMB-Client ist das System, das auf die Freigabe zugreift, etwa ein Arbeitsplatzrechner. Der SMB-Server stellt die Freigabe bereit, etwa ein Windows-Server oder ein Linux-System mit Samba. Client und Server handeln bei jeder Verbindung die Version und die Sicherheitsoptionen aus.

11.4 Windows-Freigaben

Unter Windows legt man eine Freigabe über den Explorer oder über PowerShell mit New-SmbShare an. Die Freigaben und ihre Verbindungen sieht man mit Get-SmbShare und Get-SmbConnection. Der Befehl net share listet die Freigaben im klassischen Format.

11.5 Samba unter Linux

Samba implementiert SMB auf Linux und Unix. Mit Samba kann ein Linux-Server Freigaben für Windows-Clients anbieten und sich in Windows-Netze einfügen. Die Konfiguration steht in der Datei smb.conf. Auf dem Client verbindet man sich mit smbclient oder mountet die Freigabe mit mount.cifs, das CIFS nutzt, eine Variante von SMB.

11.6 Authentifizierung

Der Zugriff erfolgt nach Anmeldung. Unter Windows-Domänen übernimmt Kerberos die Anmeldung, in einfachen Netzen die NTLM-Verfahren oder ein Benutzerkonto auf dem Server. Wer angemeldet ist, bekommt die Rechte, die ihm zugewiesen wurden.

11.7 Berechtigungen

Der Zugriff hängt von zwei Ebenen ab. Die Freigabeberechtigungen bestimmen, wer überhaupt über das Netz auf die Freigabe zugreifen darf. Die Dateisystemberechtigungen bestimmen, was mit den einzelnen Dateien und Ordnern geschieht, etwa Lesen, Schreiben oder Ändern. Beide Ebenen wirken zusammen, und die strengere gilt. Ein Benutzer mit vollen Dateirechten, aber ohne Freigabezugriff, kommt nicht hinein. Die Abbildung zeigt das Zusammenspiel.

11.8 SMB-Versionen

Es gibt mehrere Versionen. SMB1 ist uralt und gilt als unsicher, es wurde durch Angriffe bekannt und sollte abgeschaltet werden. SMB2 wurde mit Windows Vista und Windows Server 2008 eingeführt, verbessert den Verkehr und reduziert die Anzahl der Befehle. SMB3 kam mit Windows 8 und Windows Server 2012, es bringt Verschlüsselung und Schutz gegen Manipulation. Moderne Systeme handeln die höchste gemeinsame Version aus.

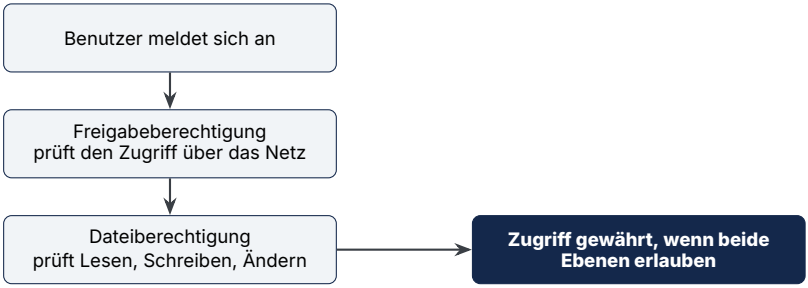


Abbildung 11.1. Zusammenspiel von Freigabe- und Dateiberechtigungen. Der Zugriff gelingt nur, wenn beide Ebenen ihn erlauben. Schematische Darstellung.

11.9 SMB-Signing

Signing signiert jede Nachricht mit einem Schlüssel, der aus der Anmeldung entsteht. So erkennt der Empfänger, wenn eine Nachricht verändert wurde, etwa durch einen Angriff auf dem Weg. Auf Domänencontrollern ist Signing vorgeschrieben, auf anderen Systemen sollte man es aktivieren.

11.10 SMB-Verschlüsselung

SMB3 kann den gesamten Verkehr einer Freigabe verschlüsseln. Das schützt Dateien und Anmeldedaten vor dem Mitlesen. Die Verschlüsselung kostet etwas Rechenzeit, bringt aber Schutz für Daten, die das Netz verlassen oder in fremden Netzen laufen.

11.11 Typische Sicherheitsrisiken

Die größten Risiken sind veraltete Versionen, besonders SMB1, offene Freigaben mit zu weitreichenden Rechten, Freigaben, die aus dem Internet erreichbar sind, und schwache Passwörter. Schadsoftware nutzt SMB, um sich im Netz zu verbreiten. Port 445 gehört deshalb nicht ins Internet, sondern hinter eine Firewall.

11.12 Unterschied zwischen Freigabe- und Dateisystemberechtigungen

Die Freigabe ist die Tür, das Dateisystem sind die Räume dahinter. Wer die Tür freigibt, aber die Räume nicht absperrt, lässt jeden hinein, der die Tür öffnen darf. Wer nur die Räume absperrt, aber die Tür offen lässt, gewährt zwar weniger Rechte, verliert aber die Übersicht, wer sie betritt. Beide Ebenen sollten bewusst gesetzt werden.

11.13 Praxis mit SMB

Unter Windows zeigt net view \\server die Freigaben eines Servers. Mit net use Z: \\server\Projekte verbindet man ein Laufwerk. Unter Linux listet smbclient -L //server -U benutzer die Freigaben, und mit smbclient //server/Projekte verbindet man sich interaktiv. Ein Befehl wie Test-NetConnection -Port 445 prüft, ob der Port erreichbar ist.

> **Merke**
>
> SMB ist das Protokoll für Netzwerkfreigaben und läuft auf Port 445 über TCP. Der Zugriff braucht zwei Berechtigungsebenen, und die strengere gilt. Aktuelle Versionen mit Signing und Verschlüsselung sind Pflicht, SMB1 gehört abgeschaltet.

Kontrollfragen

1. Welcher Port nutzt SMB, und welches Protokoll liegt darunter?
2. Ein Benutzer hat volle Dateirechte, bekommt aber beim Zugriff auf die Freigabe eine Ablehnung. Wie ist das möglich?
3. Welche SMB-Version sollte man heute nicht mehr verwenden, und warum?
4. Warum sollte Port 445 nicht aus dem Internet erreichbar sein?

Musterlösungen: 1. SMB nutzt Port 445 über TCP. 2. Die Freigabeberechtigung verweigert den Zugriff über das Netz, auch wenn die Dateirechte es erlauben. Die strengere Ebene gilt. 3. SMB1, weil es unsicher ist und durch bekannte Angriffe ausgenutzt wurde. 4. Schadsoftware nutzt SMB, um sich zu verbreiten, und offene Freigaben aus dem Internet sind ein beliebtes Ziel.

KAPITEL 12

LDAP und Verzeichnisdienste

In jedem größeren Unternehmen stellt sich dieselbe Frage: Wer ist diese Person, welche Konten hat sie und welche Rechte gelten für sie? Die Antwort steht nicht auf jedem Rechner einzeln, sondern in einem zentralen Verzeichnis. Dort liegen Benutzer, Gruppen, Computer und Angaben wie Telefonnummern oder Abteilungen. LDAP ist das Protokoll, mit dem solche Verzeichnisse abgefragt und gepflegt werden. Dieses Kapitel erklärt, was ein Verzeichnisdienst ist, wie LDAP seine Daten ordnet und wie es mit Active Directory und Kerberos zusammenwirkt.

12.1 Was ist LDAP?

LDAP, das Lightweight Directory Access Protocol, ist ein Protokoll für den Zugriff auf Verzeichnisdienste. Die aktuelle Version LDAPv3 ist in RFC 4511 festgelegt. LDAP beschreibt, wie ein Client mit einem Verzeichnisserver spricht: Er meldet sich an, sucht Einträge, liest oder ändert sie. Der Name trägt den Begriff leichtgewichtig, weil die Vorgängerversion X.500 deutlich aufwendiger war.

12.2 Verzeichnisdienste

Ein Verzeichnisdienst ist eine Datenbank, die für Abfragen optimiert ist. Anders als eine Buchhaltungsdatenbank ändert sich der Inhalt selten, dafür wird er sehr oft gelesen. Typische Verzeichnisse speichern Personen, Gruppen, Geräte und Dienste mit ihren Eigenschaften. Ein Adressbuch ist ein einfaches Beispiel, ein Unternehmensverzeichnis ist das professionelle Gegenstück.

12.3 Distinguished Name

Jeder Eintrag hat einen eindeutigen Namen, den Distinguished Name, kurz DN. Er beschreibt den Weg vom Eintrag zur Wurzel des Verzeichnisses. Ein Beispiel ist cn=Anna Muster,ou=Vertrieb,dc=example,dc=org. Die Teile lesen sich von links nach rechts vom Konkreten zum Allgemeinen: die Person, ihre Organisationseinheit, die Domain.

12.4 Relative Distinguished Name

Der Relative Distinguished Name, kurz RDN, ist der vorderste Teil des DN, also der Name innerhalb seines Containers. Im Beispiel ist cn=Anna Muster der RDN. Gleiche RDNs dürfen in einem Container nicht zweimal vorkommen, sonst wären die DNs nicht eindeutig.

12.5 Directory Information Tree

Die Einträge bilden einen Baum, den Directory Information Tree, kurz DIT. Die Wurzel steht oben, darunter folgen Organisationseinheiten, Gruppen und Personen. Die folgende Abbildung zeigt einen kleinen Ausschnitt.

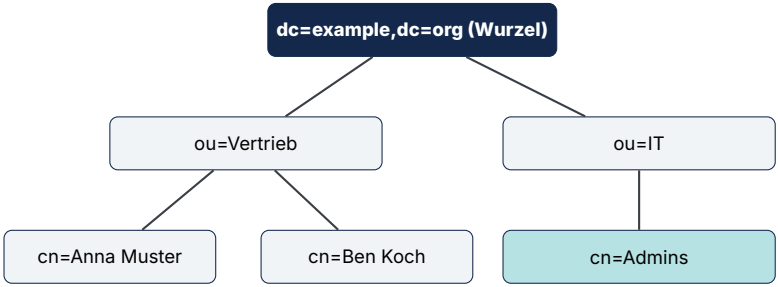


Abbildung 12.1. Ausschnitt eines Directory Information Tree mit Organisationseinheiten, Personen und einer Gruppe. Der Pfad vom Eintrag zur Wurzel ergibt den Distinguished Name. Schematische Darstellung.

12.6 LDAP-Einträge

Ein Eintrag besteht aus seinem DN und einer Menge von Attributen. Jedes Attribut hat einen Typ und einen oder mehrere Werte. Ein Benutzer hat etwa die Attribute cn für den Namen, sn für den Nachnamen, mail für die Adresse und memberOf für die Gruppen, in denen er ist.

12.7 Attribute

Attribute sind die Eigenschaften eines Eintrags. Welche Attribute erlaubt sind, legt ein Schema fest. Ein Schema beschreibt die Klassen von Einträgen und welche Attribute

sie tragen müssen oder dürfen. So bleiben Verzeichnisse einheitlich, auch wenn verschiedene Programme auf sie zugreifen.

12.8 LDAP-Suche

Die Suche ist die wichtigste Operation. Der Client gibt einen Startpunkt im Baum, einen Suchbereich und einen Filter an. Der Filter beschreibt die gesuchten Einträge, etwa alle Personen mit einem bestimmten Nachnamen. Der Server antwortet mit den passenden Einträgen und den angeforderten Attributen.

12.9 LDAP-Bind

Bevor ein Client sucht oder ändert, meldet er sich an. Diese Operation heißt Bind. Der einfache Bind verwendet DN und Passwort, andere Verfahren nutzen SASL für Kerberos oder Zertifikate. Ohne gültige Anmeldung liefert der Server nur die öffentlich freigegebenen Angaben oder nichts.

Die folgende Abbildung zeigt den Ablauf einer Anmeldung mit anschließender Suche.

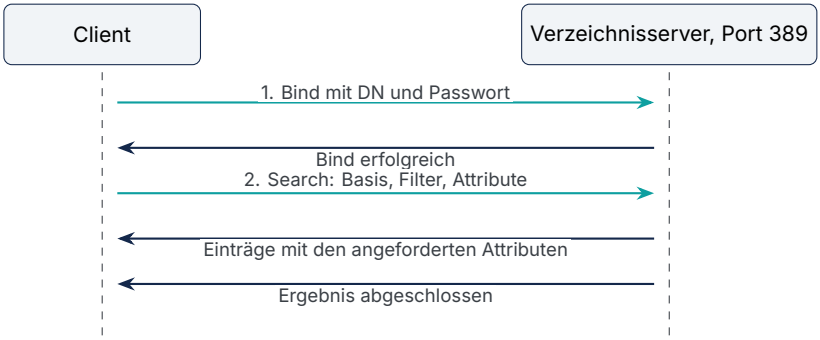


Abbildung 12.2. Anmeldung und Suche mit LDAP. Nach dem Bind sucht der Client mit einem Filter und erhält die passenden Einträge. Schematische Darstellung.

12.10 LDAP und TLS

LDAP überträgt Passwörter im Klartext, wenn es nicht verschlüsselt wird. Es gibt zwei Wege zur Verschlüsselung. Bei LDAPS ist die Verbindung von Anfang an mit TLS

geschützt und läuft auf Port 636. Bei StartTLS beginnt die Verbindung auf Port 389 unverschlüsselt und wechselt dann auf TLS. Beide Verfahren sind üblich, LDAPS ist einfacher zu prüfen.

12.11 Active Directory als Anwendungsbeispiel

Active Directory ist der Verzeichnisdienst von Microsoft für Windows-Netze. Es nutzt LDAP als Zugriffsprotokoll, speichert Benutzer, Computer und Gruppen und bietet Anmeldung und Richtlinien. Die Struktur aus Domänen und Organisationseinheiten entspricht dem Baum, den wir oben betrachtet haben. Viele Anwendungen fragen Active Directory per LDAP ab, etwa um Benutzer zu finden.

12.12 LDAP und Kerberos unterscheiden

LDAP und Kerberos werden oft verwechselt, weil sie in Active-Directory-Umgebungen zusammen auftreten. LDAP verwaltet die Verzeichnisinformationen: Wer existiert, welche Attribute und Gruppen hat er. Kerberos prüft die Identität bei der Anmeldung und stellt Tickets aus. Kerberos ist in RFC 4120 beschrieben. Eine Anwendung kann Kerberos für die Anmeldung und LDAP für die Suche nach Benutzern nutzen.

12.13 Authentifizierung und Autorisierung

Authentifizierung beantwortet die Frage, wer jemand ist. Autorisierung beantwortet, was diese Person darf. Ein Verzeichnisdienst liefert beides: Die Anmeldung prüft die Identität, die Gruppenmitgliedschaften bestimmen die Rechte. Wer die Unterscheidung kennt, kann Fehler gezielt suchen. Ein Passwortfehler ist ein Authentifizierungsproblem, ein fehlendes Recht auf eine Ressource ein Autorisierungsproblem.

12.14 Praxis mit LDAP

Unter Windows zeigt dsquery user die Benutzer im Active Directory. Die Gruppenmitgliedschaften einer Person liest man mit dsget user oder der Powershell mit Get-ADUser. Unter Linux liefert ldapsearch eine Abfrage im Terminal, etwa ldapsearch -x -H ldap://server -b dc=example,dc=org. Die Option -x wählt die einfache Anmeldung. Die Laborübungen führen diese Befehle aus, auf einem eigenen Testverzeichnis.

12.15 Typische Fehler

Häufige Fehler sind ein falscher DN, der auf einen anderen Container zeigt, ein Bind mit abgelaufenem Passwort, ein Filter, der die Groß- und Kleinschreibung nicht beachtet, oder eine blockierte Verbindung auf Port 389 oder 636. Ein leeres Ergebnis ist oft kein Fehler, sondern ein falsch gesetzter Suchbereich.

> **Merke**
>
> LDAP fragt Verzeichnisdienste ab. Einträge sind über den Distinguished Name eindeutig und enthalten Attribute. Authentifizierung prüft die Identität, Autorisierung die Rechte. Kerberos prüft die Anmeldung, LDAP liefert die Verzeichnisinformationen.

Kontrollfragen

1. Wie lautet der Distinguished Name des Eintrags Anna Muster im Container Vertrieb der Domain example.org?
2. Was ist der Unterschied zwischen LDAPS und StartTLS?
3. Ein Benutzer kann sich nicht anmelden, obwohl das Passwort stimmt. Welche Ursache ist plausibel, und warum?
4. Welche Aufgabe hat Kerberos im Unterschied zu LDAP?

Musterlösungen: 1. cn=Anna Muster,ou=Vertrieb,dc=example,dc=org. 2. LDAPS startet verschlüsselt auf Port 636. StartTLS beginnt auf Port 389 unverschlüsselt und wechselt dann auf TLS. 3. Das Konto kann gesperrt, deaktiviert oder das Passwort abgelaufen sein. Auch die Anmeldemethode kann nicht passen. Die Passwortprüfung selbst ist damit richtig, die Autorisierung oder der Kontostatus schlägt fehl. 4. Kerberos prüft die Identität bei der Anmeldung und stellt Tickets aus. LDAP verwaltet die Verzeichnisinformationen zu Personen und Gruppen.

KAPITEL 13

Weitere wichtige Netzwerkprotokolle

Die bisherigen Kapitel haben die Protokolle behandelt, die im Alltag am häufigsten vorkommen. Daneben gibt es zahlreiche weitere Verfahren, die in bestimmten Bereichen unverzichtbar sind: Telefonie, Industrie, Fernwartung, Gerätekommunikation im Heimnetz. Dieses Kapitel stellt ausgewählte davon vor. Für jedes Protokoll erklären wir die Aufgabe, die Funktionsweise, die Einsatzgebiete, die Ports und die Sicherheitsaspekte. Eine Vergleichstabelle am Ende bringt alles übersichtlich zusammen.

13.1 Telnet

Telnet ist eine Fernanmeldung über TCP auf Port 23. Es ist sehr alt und überträgt alles im Klartext, auch Passwörter. Heute sollte man Telnet nicht mehr verwenden, sondern SSH. Man begegnet ihm noch in Netzkomponenten älterer Bauart und in Laborübungen, um zu zeigen, wie unverschlüsselte Anmeldung aussieht.

13.2 TFTP

Das Trivial File Transfer Protocol überträgt Dateien über UDP, meist Port 69. Es kennt keine Anmeldung und keine Verschlüsselung und ist sehr einfach aufgebaut. Typisch ist der Einsatz beim Starten von Geräten über das Netz, etwa bei Telefonen oder Netzkomponenten, die ihr Betriebssystem oder ihre Konfiguration laden. Wegen fehlender Sicherheit sollte TFTP nur in abgeschotteten Netzen laufen.

13.3 RDP

Das Remote Desktop Protocol ermöglicht den grafischen Fernzugriff auf Windows-Rechner. Es nutzt meist TCP auf Port 3389, daneben auch UDP für bessere Bildübertragung. Die Verbindung ist mit TLS geschützt. RDP ist ein beliebtes Ziel für Angriffe, deshalb sollte der Port nicht ungeschützt ins Internet, sondern nur über VPN oder ein Gateway erreichbar sein.

13.4 SIP

Das Session Initiation Protocol steuert Telefonie und Videokonferenzen. Es baut Sitzungen auf, findet Gesprächspartner und handelt Medienformate aus. SIP ist in RFC 3261 beschrieben. Die Signalisierung läuft meist über Port 5060, verschlüsselt über Port 5061 mit TLS. Die eigentlichen Sprachdaten überträgt RTP.

13.5 RTP

Das Real-time Transport Protocol überträgt Audio und Video mit Zeitstempeln und Reihenfolgenummern. Es läuft über UDP auf dynamisch ausgehandelten Ports. Es ist in RFC 3550 festgelegt, zusammen mit RTCP. RTP verzichtet auf Wiederholung verlorener Pakete, weil ein verspätetes Bild oder Ton wertlos ist.

13.6 RTCP

Das RTP Control Protocol begleitet RTP und meldet Qualitätsdaten wie Verlustrate, Laufzeit und Schwankungen. Es nutzt in der Regel den Port des RTP-Stroms plus eins. Anhand von RTCP erkennt man, ob eine Sprachverbindung schlecht ist, obwohl sie noch steht.

13.7 MQTT

MQTT ist ein leichtgewichtiges Nachrichtenprotokoll für Geräte mit wenig Bandbreite, etwa Sensoren im Smart Home oder in der Industrie. Geräte veröffentlichen Nachrichten zu Themen, andere Geräte abonnieren diese Themen. Ein zentraler Broker verteilt die Nachrichten. MQTT läuft über TCP, meist auf Port 1883, verschlüsselt auf Port 8883. Die Spezifikation liegt bei OASIS, aktuell in Version 5.0.

13.8 WebSocket

WebSocket hält eine dauerhafte bidirektionale Verbindung zwischen Browser und Server offen, sodass beide Seiten jederzeit Nachrichten senden können. Die Verbindung beginnt als HTTP-Anfrage mit einem Upgrade und wechselt dann das Protokoll. Die Spezifikation ist in RFC 6455 beschrieben. Ports sind 80 oder 443, verschlüsselt als wss.

13.9 mDNS

Multicast DNS ermöglicht Namensauflösung im lokalen Netz ohne zentralen DNS-Server. Geräte fragen per Multicast nach einem Namen, etwa drucker.local, und das passende Gerät antwortet. mDNS ist in RFC 6762 festgelegt und nutzt UDP auf Port 5353. Es ist die Grundlage für viele automatisch erkannte Geräte im Heimnetz.

13.10 SSDP

Das Simple Service Discovery Protocol sucht Geräte im lokalen Netz, etwa Medienserver oder Router mit UPnP. Es nutzt UDP auf Port 1900 mit Multicast. SSDP ist kein RFC-Standard, sondern Teil der UPnP-Spezifikation eines Herstellerkonsortiums. Es gilt als unsicher, weil es Geräte offenlegt und manipulierbar ist, deshalb sollte UPnP im Netz nur bewusst aktiviert sein.

13.11 Vergleichstabelle

Die folgende Tabelle fasst die behandelten Protokolle zusammen.

> **Merke**
>
> Viele Spezialprotokolle folgen den Mustern, die wir kennen: Ein Transport, ein Port und eine Aufgabe. Entscheidend ist, ob eine Verschlüsselung vorgesehen ist und ob das Protokoll im Netz offen erreichbar sein darf.

Kontrollfragen

1. Welches Protokoll überträgt Sprache, und welches meldet die Qualität dieser Übertragung?
2. Warum eignet sich MQTT für Sensoren mit wenig Bandbreite?
3. Welches Protokoll läuft über UDP ohne Anmeldung und ist für den Start von Geräten gedacht?
4. Warum ist SSDP im Netz mit Vorsicht zu betreiben?

Musterlösungen: 1. RTP überträgt Sprache, RTCP meldet die Qualität. 2. MQTT ist ein schlankes Nachrichtenprotokoll, das wenig Overhead hat und ein Veröffentlichen-und-Abonnieren-Modell nutzt. 3. TFTP. 4. SSDP legt Geräte offen und lässt sich manipulieren,

Protokoll	Transport	Ports	Einsatz	Sicherheit
Telnet	TCP	23	Fernanmeldung, alt	unverschlüsselt
TFTP	UDP	69	Start und Konfiguration von Geräten	ohne Anmeldung
RDP	TCP, UDP	3389	grafischer Fernzugriff	TLS, angriffsgefährdet
SIP	TCP, UDP	5060, 5061	Telefonie, Signalisierung	TLS möglich
RTP	UDP	dynamisch	Sprache und Video	SRTP möglich
RTCP	UDP	RTP-Port plus 1	Qualitätsdaten	wie RTP
MQTT	TCP	1883, 8883	Sensoren, Gerätesteuerung	TLS möglich
WebSocket	TCP	80, 443	dauerhafter Datenaustausch im Browser	als wss mit TLS
mDNS	UDP	5353	lokale Namen im Heimnetz	nur lokal
SSDP	UDP	1900	Gerätesuche, UPnP	unsicher

Tabelle 13.1. Übersicht der in diesem Kapitel behandelten Protokolle mit Transport, Ports, Einsatz und Sicherheit.

deshalb sollte UPnP nur bewusst und nicht aus dem Internet erreichbar aktiviert sein.

KAPITEL 14

Wie Netzwerkprotokolle zusammenarbeiten

Die einzelnen Protokolle dieses Bandes wirken erst im Zusammenspiel. Wenn du eine Webseite öffnest, arbeiten DHCP, DNS, TCP, TLS und HTTP nacheinander, und jedes Protokoll liefert genau die Information, die das nächste braucht. Eine Mail verlangt DNS, SMTP und ein Abrufprotokoll. Eine SSH-Sitzung beginnt mit DNS und endet mit einer verschlüsselten Befehlsschnittstelle. Dieses Kapitel verfolgt vier typische Abläufe Schritt für Schritt. Jede Nachricht ist nummeriert, damit du den Ablauf in den Diagrammen wiederfindest.

14.1 Beispiel 1: Ein Benutzer öffnet eine Webseite

Der Benutzer gibt die Adresse www.example.org in den Browser ein. Der Browser braucht dafür IP-Adresse, Verbindung und Verschlüsselung. Die folgende Abbildung zeigt den vollständigen Ablauf vom Namen bis zur dargestellten Seite. Die Adressen sind Beispiele aus den Dokumentationsbereichen.

Die Reihenfolge ist nicht beliebig. Ohne Adresse aus DNS gibt es keine Verbindung, ohne Verbindung keine Verschlüsselung, und ohne Verschlüsselung dürfte der Browser die Anfrage nicht senden. Gerade deshalb lassen sich Fehler an dieser Kette ablesen: Scheitert Schritt 1, liegt das Problem bei der Auflösung. Scheitert Schritt 3, liegt es bei der Verbindung. Scheitert Schritt 5, stimmt das Zertifikat nicht.

Moderne Browser nutzen für Schritt 3 bis 5 auch QUIC, das diese Schritte in einem einzigen Verbindungsaufbau zusammenfasst, wie in Band 3 beschrieben.

14.2 Beispiel 2: Ein Benutzer verschickt eine E-Mail

Alice schreibt eine Mail an bob@example.org. Ihr Mailprogramm übergibt die Nachricht an den eigenen Server, der sie an den Server der Empfängerdomain weiterleitet. Die folgende Abbildung zeigt den Ablauf.

Auch hier gibt es eine klare Kette. Der MX-Record aus dem DNS sagt dem Absender-

server, wohin die Mail gehört. Ist die Übergabe erfolgreich, liegt die Nachricht im Postfach, und Bob holt sie mit IMAP. Schlägt die Zustellung fehl, versucht der Absenderserver es später erneut.

14.3 Beispiel 3: Ein Administrator verbindet sich mit einem Linux-Server

Der Administrator möchte sich mit ssh admin@server.example.org anmelden. Die Kette beginnt wieder mit dem Namen und führt über TCP zu einer verschlüsselten Sitzung, in der die Anmeldung stattfindet.

Die Prüfung des Host Keys in Schritt 6 schützt den Administrator vor einem falschen Server. Die Anmeldung mit Schlüssel in Schritt 7 schützt das Konto vor Passwortangriffen. Bricht die Verbindung in Schritt 3 ab, liegt eine Firewall im Weg. Schlägt Schritt 7 fehl, passt der öffentliche Schlüssel nicht.

14.4 Beispiel 4: Ein neuer Computer wird mit dem Netzwerk verbunden

Ein Laptop wird ins Büronetz gesteckt. Er kennt weder seine Adresse noch den Router. Die folgende Abbildung zeigt, wie er die Konfiguration erhält und danach kommunizieren kann.

Die Kette zeigt die Abhängigkeit zwischen den Diensten. Ohne DHCP fehlen Adresse und Gateway. Ohne Gateway ist kein Ziel außerhalb des Netzes erreichbar. Ohne DNS-Angabe funktionieren Namen nicht, auch wenn die Verbindung steht. Viele Störungen im Büronetz lassen sich so auf den fehlenden Schritt zurückführen.

> **Merke**
>
> Netzwerkdienste bauen aufeinander auf. Typische Kette: Name, Adresse, Verbindung, Verschlüsselung, Anwendung. Ein Fehler zeigt sich dort, wo die Kette abbricht, und die Schrittfolge zeigt, wo man sucht.

Kontrollfragen

1. Welche Information braucht der Browser aus DNS, bevor er den TCP-Handshake beginnt?
2. Warum findet der Absenderserver den Empfängerserver über den MX-Record und nicht über den A-Record der Domain?
3. Welcher Schritt schützt den Administrator in Beispiel 3 vor einem gefälschten Server?
4. Ein neuer Laptop hat eine Adresse, kann aber keine Namen auflösen. Welcher Schritt der Kette ist betroffen, und welche Angabe fehlt?

Musterlösungen: 1. Die IP-Adresse des Webservers, die der Resolver liefert. 2. Die Zustellung von E-Mail nutzt den MX-Record, der den Mailserver nennt. Der A-Record liefert nur eine Adresse für den Namen und sagt nichts über den Mailserver aus. 3. Die Prüfung des Host Keys in Schritt 6. 4. Die DNS-Angabe aus der DHCP-Antwort fehlt oder ist falsch. Die Adresse ist vorhanden, die Auflösung schlägt fehl.

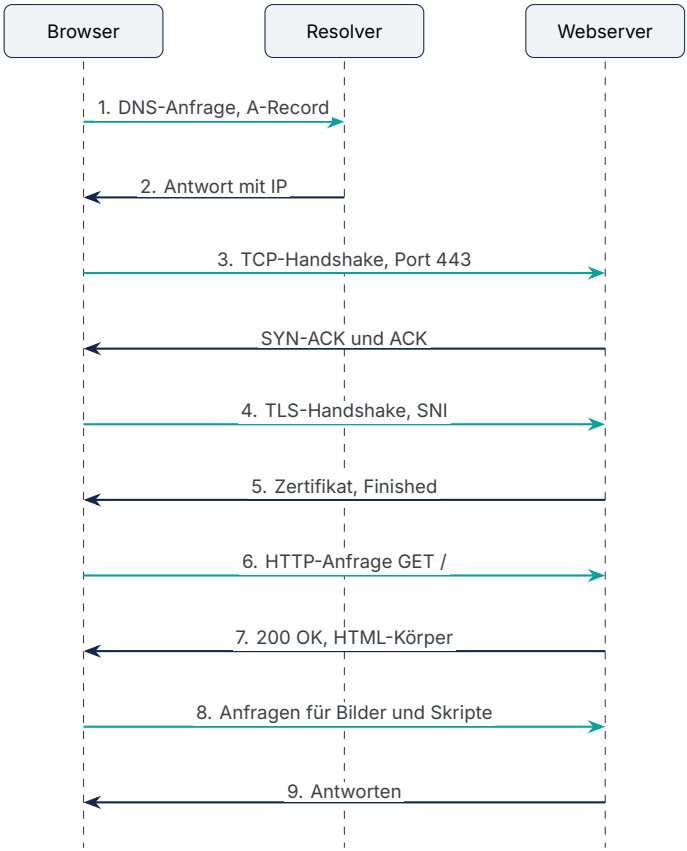


Abbildung 14.1. Ablauf beim Öffnen einer Webseite. Die Schritte 1 und 2 klären den Namen, 3 bis 5 bauen eine verschlüsselte Verbindung auf, 6 bis 9 übertragen die Seite. Schematische Darstellung, die Nachrichten sind vereinfacht.

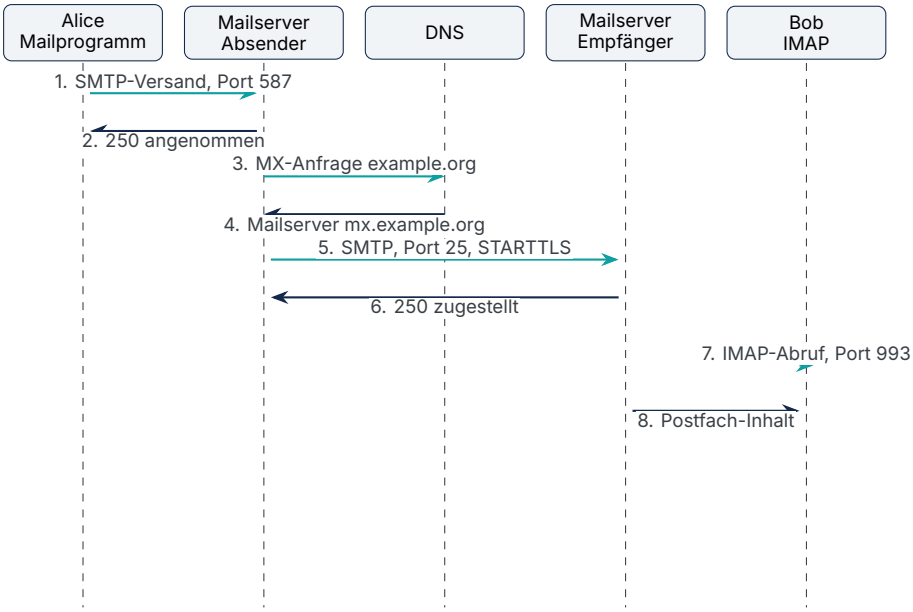


Abbildung 14.2. Ablauf des E-Mail-Versands von Alice zu Bob. Der Absender nutzt Submission, der Empfängerserver wird über den MX-Record gefunden, die Übergabe erfolgt zwischen den Servern, der Abruf per IMAP. Schematische Darstellung.

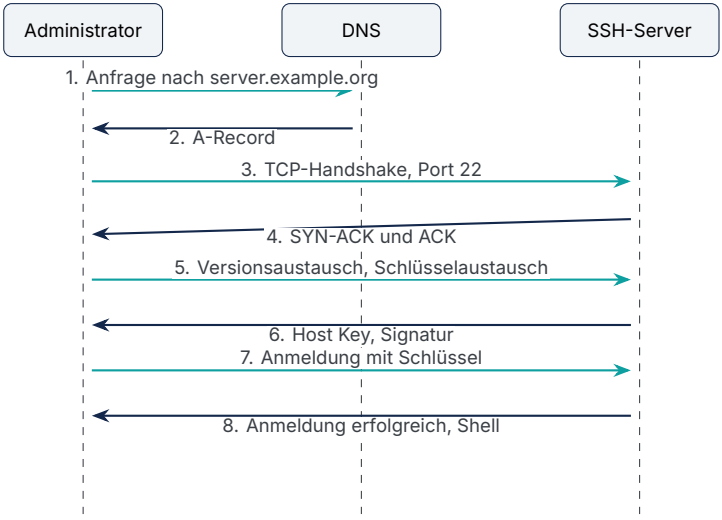


Abbildung 14.3. Ablauf einer SSH-Verbindung zu einem Linux-Server. Nach DNS und TCP entsteht die verschlüsselte Sitzung, erst dann folgt die Anmeldung. Schematische Darstellung.

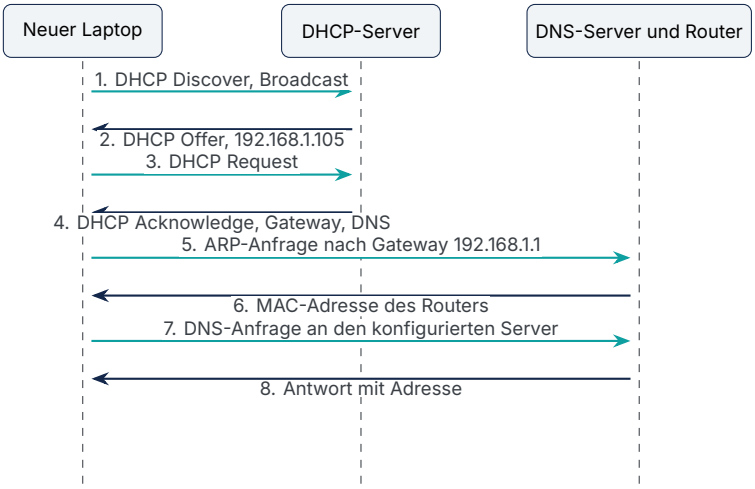


Abbildung 14.4. Anbindung eines neuen Computers. Der DHCP-Ablauf liefert Adresse, Gateway und DNS-Server. Danach löst der Rechner die Gateway-Adresse per ARP auf und kann Namen auflösen. Schematische Darstellung.

KAPITEL 15

Netzwerkanalyse mit Wireshark

Im Kapitel über die Transportprotokolle haben wir Wireshark eingeführt. Jetzt geht es um die Anwendungsprotokolle, die in diesem Band behandelt wurden. Ein Mitschnitt zeigt dir, was ein Browser, ein Mailprogramm oder ein Administrator tatsächlich über das Netz schicken. Die Pakete sind nicht mehr nur Bytes, sondern lesbare Nachrichten mit Namen, Methoden, Statuscodes und Zeitstempeln. Dieses Kapitel zeigt, wie du die wichtigsten Protokolle erkennst, mit welchen Filtern du sie herausholst und welche Fehler sich in einem Mitschnitt ablesen lassen.

15.1 Protokolle erkennen

Wireshark erkennt die meisten Protokolle anhand der Portnummer und des Inhalts. Im Feld Protocol steht dann DNS, DHCP, HTTP, TLS oder SSH. Gelingt die Zuordnung nicht, zeigt Wireshark nur TCP oder UDP. Dann hilft der Blick auf die Portnummer und die Nutzdaten im unteren Bereich. Bei verschlüsselten Verbindungen sieht man das Protokoll TLS und die Versionsangabe, nicht den Inhalt.

15.2 DNS-Verkehr analysieren

Eine DNS-Auflösung besteht aus einer Anfrage und einer Antwort. Die Antwort enthält den Record, der angefragt wurde, die TTL und den Statuscode. Der Filter dns zeigt alle DNS-Nachrichten. Mit dns.flags.response == 1 siehst du nur die Antworten. Die Zeit zwischen Anfrage und Antwort zeigt die Laufzeit zum Resolver.

15.3 DHCP-DORA untersuchen

Der DORA-Ablauf erscheint als vier Nachrichten zwischen Port 68 und Port 67. Der Filter dhcp zeigt sie. Im Detailbereich findest du den Nachrichtentyp in der Option 53, die Discover mit 1, Offer mit 2, Request mit 3 und Acknowledge mit 5 kennzeichnet. Der Filter dhcp.option.dhcp == 1 zeigt nur die Discover-Nachrichten. Die Prüfung der Feldnamen gehört zu jeder Analyse, denn die Bezeichnungen ändern sich zwischen Wireshark-

Versionen.

15.4 HTTP-Anfragen analysieren

Eine unverschlüsselte HTTP-Anfrage zeigt die Methode, den Pfad, den Host und die Header im Klartext. Der Filter http zeigt die Nachrichten. Mit http.request.method == "GET" filterst du nach Methode, und http.response.code == 404 nach Fehlern. Bei HTTPS siehst du diese Angaben nicht, weil sie in TLS verborgen sind.

15.5 TLS-Handshake erkennen

Der TLS-Handshake ist auch bei verschlüsseltem Verkehr sichtbar. Die ClientHello-Nachricht enthält den Servernamen im Feld SNI, und die Serverantwort zeigt Version und Zertifikat. Der Filter tls zeigt die Nachrichten. Mit tls.handshake.type == 1 siehst du die ClientHello-Nachrichten, mit tls.handshake.type == 11 die Zertifikate.

15.6 SSH-Verbindungen untersuchen

Eine SSH-Verbindung zeigt die Versionszeichenkette, die Schlüsselaustausch-Nachrichten und danach nur noch verschlüsselte Daten. Der Filter ssh zeigt die Nachrichten. Der Inhalt der Sitzung bleibt verborgen, aber die Länge der Pakete und ihre Zeitpunkte verraten, dass eine interaktive Sitzung läuft.

15.7 NTP-Verkehr analysieren

NTP läuft über UDP auf Port 123 und besteht aus Anfrage und Antwort mit Zeitstempeln. Der Filter ntp zeigt den Verkehr. Im Detail siehst du die vier Zeitstempel, die wir im Kapitel über NTP berechnet haben.

15.8 Protokollfilter verwenden

Die folgende Tabelle fasst die wichtigsten Filter zusammen. Sie zeigt die Protokolle dieses Bandes und die Filter, die man in der Praxis am häufigsten braucht. Die Filter sind für aktuelle Wireshark-Versionen geschrieben, Feldnamen können sich ändern und

sollten im Zweifel im Feld Anzeigefilter geprüft werden.

Filter	Zeigt
dns	alle DNS-Nachrichten
dns.flags.response == 1	nur DNS-Antworten
dhcp	alle DHCP-Nachrichten
dhcp.option.dhcp == 1	nur DHCP-Discover
http	alle HTTP-Nachrichten
http.response.code == 404	Antworten mit 404
tls	alle TLS-Nachrichten
tls.handshake.type == 1	ClientHello
ssh	alle SSH-Nachrichten
ntp	alle NTP-Nachrichten
tcp.port == 443	TCP-Verkehr mit Port 443
udp.port == 53	UDP-Verkehr mit Port 53
tcp.port == 22	TCP-Verkehr mit Port 22

Tabelle 15.1. Anzeigefilter für die Protokolle dieses Bandes. Die Filternamen beziehen sich auf aktuelle Wireshark-Versionen.

15.9 Fehler erkennen

Ein Mitschnitt zeigt viele Fehler früh. Eine DNS-Antwort mit Fehlercode, ein TCP-Reset direkt nach dem SYN, ein TLS-Alert mit einem Zertifikatsfehler oder ein DHCP-Discover ohne Offer sind typische Spuren. Die Zeitstempel helfen, die Ursache auf einen Abschnitt einzugrenzen, wie in Kapitel 14 gezeigt.

> **Merke**
>
> Wireshark macht Protokolle sichtbar, die im Alltag verborgen bleiben. Verschlüsselung verbirgt den Inhalt, aber nicht den Ablauf. Filter holen die relevanten Nachrichten heraus, und die Zeitstempel zeigen, wo eine Kette abbricht.

Kontrollfragen

1. Welcher Filter zeigt nur die Antworten einer DNS-Auflösung?
2. Welche Option in DHCP kennzeichnet den Nachrichtentyp, und welcher Wert steht für Discover?
3. Warum sieht man bei HTTPS den Pfad einer Anfrage nicht, den Servernamen aber oft schon?
4. Welcher Filter zeigt die ClientHello-Nachrichten einer TLS-Verbindung?

Musterlösungen: 1. dns.flags.response == 1. 2. Option 53, der Wert 1 steht für Discover. 3. Pfad und Header liegen bei HTTPS innerhalb der verschlüsselten TLS-Nutzdaten. Der Servername steht im SNI-Feld des ClientHello, das unverschlüsselt übertragen wird. 4. tls.handshake.type == 1.

KAPITEL 16

Praktische Laborübungen

Dieses Kapitel enthält fünfzehn Übungen zu den Protokollen des Bandes. Alle laufen auf deinem eigenen Rechner, in einer virtuellen Maschine oder gegen Testdienste, die du selbst startest. Sicherheitsrelevante Änderungen, etwa an der SSH-Konfiguration, führst du nur auf einem Testsystem durch, und du hältst immer einen zweiten Zugang bereit, damit du dich nicht aussperrst.

Jede Übung ist gleich aufgebaut: Lernziel, Voraussetzungen, benötigte Software, Schritte, Befehle, eine Beispielausgabe, die Erklärung, typische Fehler und die Musterlösung. Die Beispielausgaben sind gekürzt, deine eigenen Werte werden abweichen.

16.1 DNS

Übung 1: DNS-Auflösung untersuchen

Lernziel: Du verfolgst, wie ein Name in eine Adresse übersetzt wird.

Voraussetzungen: Internetzugang.

Benötigte Software: nslookup unter Windows, dig oder host unter Linux.

Schritte: Frage den Namen www.example.org ab und notiere Antwortserver, Adresse und TTL.

Befehle: `nslookup www.example.org` oder `dig www.example.org A`

Beispielausgabe (gekürzt)

```
;; ANSWER SECTION:
www.example.org.    3600  IN A  203.0.113.10
```

Erklärung: Der Eintrag zeigt Name, TTL in Sekunden, Klasse IN und Typ A. 3600 bedeutet eine Stunde Gültigkeit im Cache.

Typische Fehler: Eine leere Antwort bedeutet oft einen falschen Namen oder einen nicht erreichbaren Resolver.

Musterlösung: Du nennst den Antwortwert, die TTL und den verwendeten Resolver.

Übung 2: A- und AAAA-Records abfragen

Lernziel: Du unterscheidest IPv4- und IPv6-Einträge desselben Namens.

Voraussetzungen: Übung 1.

Benötigte Software: nslookup oder dig.

Schritte: Frage den Namen zweimal ab, einmal mit dem Typ A, einmal mit AAAA.

Befehle: `nslookup -type=AAAA www.example.org` und `dig www.example.org AAAA`

Beispielausgabe (gekürzt)

```
www.example.org.  3600  IN AAAA  2001:db8::10
```

Erklärung: Der AAAA-Record liefert eine IPv6-Adresse. Fehlt er, erreichen IPv6-Clients den Dienst nicht direkt.

Typische Fehler: Ein fehlender AAAA-Eintrag wird oft mit einem Verbindungsfehler verwechselt.

Musterlösung: Du zeigst, dass der Name je nach Typ eine IPv4- oder IPv6-Adresse liefert.

Übung 3: MX-Records analysieren

Lernziel: Du findest den Mailserver einer Domain.

Voraussetzungen: Übung 1.

Benötigte Software: nslookup oder dig.

Schritte: Frage die MX-Einträge der Domain ab und vergleiche die Prioritäten.

Befehle: `nslookup -type=MX example.org` und `dig example.org MX`

Beispielausgabe (gekürzt)

```
example.org.  3600  IN MX  10 mx1.example.org.
example.org.  3600  IN MX  20 mx2.example.org.
```

Erklärung: Der Server mit der niedrigeren Priorität 10 wird zuerst versucht, Priorität 20 dient als Ausweichweg.

Typische Fehler: Ein MX-Eintrag, der auf einen Namen mit nur einem CNAME zeigt, ist nach der Norm unzulässig.

Musterlösung: Du nennst den Server mit Priorität 10 als primär und begründest die

Reihenfolge.

16.2 DHCP

Übung 4: DHCP-Konfiguration anzeigen

Lernziel: Du liest die vom DHCP-Server erhaltenen Einstellungen aus.

Voraussetzungen: Ein Rechner mit DHCP im Netz.

Benötigte Software: ipconfig unter Windows, nmcli unter Linux mit NetworkManager.

Schritte: Lies Adresse, Gateway, DNS und den DHCP-Server aus.

Befehle: `ipconfig /all` oder `nmcli dev show`

Beispielausgabe (gekürzt)

```
DHCP aktiviert . . . . . : Ja
DHCP-Server . . . . . : 192.168.1.1
Standardgateway . . . . . : 192.168.1.1
```

Erklärung: Der Wert DHCP aktiviert zeigt, dass die Konfiguration vom Server stammt. Der angegebene DHCP-Server ist der Ausgabeort der Vergabe.

Typische Fehler: Eine feste Adresse ohne DHCP-Angabe wirkt wie ein DHCP-Problem, ist aber eine manuelle Konfiguration.

Musterlösung: Du nennst DHCP-Status, Server und Gateway und begründest, woher jede Angabe stammt.

Übung 5: DHCP-DORA mit Wireshark beobachten

Lernziel: Du siehst die vier Nachrichten des DORA-Ablaufs.

Voraussetzungen: Ein Rechner mit DHCP, Administrator- oder Schreibrechte für den Neustart der Adressvergabe.

Benötigte Software: Wireshark.

Schritte: Starte eine Aufzeichnung auf der Netzwerkschnittstelle, gib die Adresse frei und fordere sie neu an. Setze den Filter auf dhcp.

Befehle: `ipconfig /release` und danach `ipconfig /renew`, unter Linux je nach Dienst `sudo dhclient -r` und `sudo dhclient`

Beispielausgabe (gekürzt)

```
DHCP Discover   0.0.0.0 > 255.255.255.255
DHCP Offer      192.168.1.1 > 192.168.1.105
DHCP Request    0.0.0.0 > 255.255.255.255
DHCP ACK        192.168.1.1 > 192.168.1.105
```

Erklärung: Die vier Nachrichten entsprechen Discover, Offer, Request und Acknowledge. Discover und Request sind Broadcasts, weil der Client noch keine Adresse hat.

Typische Fehler: Fehlt das Offer, ist der Server nicht erreichbar oder der Adressbereich erschöpft. Wird der Filter nicht gesetzt, geht der Überblick verloren.

Musterlösung: Du zeigst die vier Nachrichten in der richtigen Reihenfolge und erklärst die Quelladressen.

16.3 HTTP und HTTPS

Übung 6: HTTP-Header mit curl anzeigen

Lernziel: Du liest StatusZeile und Header einer Antwort.

Voraussetzungen: Internetzugang.

Benötigte Software: curl, unter Windows in aktuellen Versionen enthalten.

Schritte: Frage nur die Header einer Webseite ab.

Befehle: `curl -I https://www.example.org/`

Beispielausgabe (gekürzt)

```
HTTP/2 200
content-type: text/html; charset=UTF-8
cache-control: max-age=600
```

Erklärung: Die Statuszeile zeigt Version und Code 200. Content-Type beschreibt den Körper, Cache-Control die Speicherdauer.

Typische Fehler: Eine Umleitung mit 301 oder 302 folgt man mit curl -L, sonst bleibt der Befund unvollständig.

Musterlösung: Du nennst Version, Statuscode und zwei Header mit ihrer Bedeutung.

Übung 7: HTTP-Statuscodes testen

Lernziel: Du erzeugst gezielt 404 und 200 und liest die Codes ab.

Voraussetzungen: Python 3.

Benötigte Software: Python, curl.

Schritte: Starte im Verzeichnis deiner Wahl einen lokalen Webserver, frage eine vorhandene und eine nicht vorhandene Datei ab.

Befehle: `python3 -m http.server 8000` und `curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/fehlt`

Beispielausgabe

```
404
```

Erklärung: Die Datei fehlt, der Server meldet 404 Not Found. Eine vorhandene Datei ergäbe 200 OK.

Typische Fehler: Ein Port, der belegt ist, verhindert den Start des Servers. Dann wählt man einen anderen.

Musterlösung: Du zeigst 404 für die fehlende und 200 für die vorhandene Datei.

Übung 8: HTTPS-Zertifikate untersuchen

Lernziel: Du prüfst Aussteller und Gültigkeit eines Zertifikats.

Voraussetzungen: Internetzugang, openssl.

Benötigte Software: openssl.

Schritte: Hole das Zertifikat des Servers und zeige Aussteller, Subject und Gültigkeit.

Befehle: `openssl s_client -connect www.example.org:443 -servername www.example.org < /dev/null | openssl x509 -noout -subject -issuer -dates`

Beispielausgabe (gekürzt)

```
subject=CN = www.example.org
issuer=C = US, O = Beispiel-CA
notBefore=Jan  1 00:00:00 2026 GMT
notAfter=Jan  1 23:59:59 2027 GMT
```

Erklärung: Subject nennt den Namen, für den das Zertifikat gilt. Issuer nennt die Stelle, die es signiert hat. Die Daten zeigen den Gültigkeitszeitraum.

Typische Fehler: Ein Zertifikat, das nicht zum Namen passt, ist ein häufiger Fehler. Fehlt die Option -servername, kann der Server das falsche Zertifikat liefern.

Musterlösung: Du nennst Subject, Issuer und Gültigkeit und prüfst, ob der Name passt.

Übung 9: TLS-Verbindung analysieren

Lernziel: Du siehst den TLS-Handshake im Mitschnitt.

Voraussetzungen: Übung 8, Wireshark.

Benötigte Software: Wireshark, curl.

Schritte: Starte eine Aufzeichnung, rufe eine HTTPS-Seite auf und setze den Filter tls.handshake.type == 1.

Befehle: `curl -v https://www.example.org/` und Filter `tls.handshake.type == 1`

Erklärung: Die ClientHello-Nachricht zeigt die unterstützten Verfahren und im Feld SNI den Servernamen. Der Inhalt der Seite bleibt verborgen.

Typische Fehler: Bei aktiviertem QUIC erscheint kein TLS über TCP, sondern QUIC über UDP.

Musterlösung: Du zeigst das ClientHello mit SNI und erklärst, dass der Seiteninhalt verschlüsselt ist.

16.4 SSH und SFTP

Übung 10: SSH-Verbindung aufbauen

Lernziel: Du baust eine SSH-Sitzung zu einem Testserver auf.

Voraussetzungen: Ein Linux-Testserver mit laufendem sshd und einem Benutzerkonto.

Benötigte Software: OpenSSH-Client.

Schritte: Verbinde dich mit dem Benutzer und bestätige den Host Key beim ersten Kontakt.

Befehle: `ssh benutzer@192.168.1.50`

Beispielausgabe (gekürzt)

```
The authenticity of host '192.168.1.50' can't be established.
ED25519 key fingerprint is SHA256:...
Are you sure you want to continue connecting (yes/no)?
```

Erklärung: Beim ersten Kontakt kennt der Client den Host Key nicht. Den Fingerabdruck prüfst du und bestätigst dann mit yes.

Typische Fehler: Ein geänderter Host Key führt zu einer Warnung, die man nicht einfach überschreibt.

Musterlösung: Du bestätigst den Fingerabdruck und erklärst, warum er geprüft wird.

Übung 11: SSH-Key-Authentifizierung konfigurieren

Lernziel: Du richtest die Anmeldung mit Schlüssel ein.

Voraussetzungen: Übung 10, Testsystem mit zweitem Zugang.

Benötigte Software: OpenSSH.

Schritte: Erzeuge ein Schlüsselpaar, lege den öffentlichen Schlüssel auf dem Server ab und teste die Anmeldung ohne Passwort.

Befehle: `ssh-keygen -t ed25519` und `ssh-copy-id benutzer@192.168.1.50`

Erklärung: Der öffentliche Schlüssel steht danach in authorized_keys auf dem Server. Der private Schlüssel bleibt beim Client.

Typische Fehler: Zu lockere Dateirechte auf dem privaten Schlüssel lassen SSH die Datei ablehnen. Richtig ist 600.

Musterlösung: Du zeigst die erfolgreiche Anmeldung ohne Passwort und nennst den Speicherort des öffentlichen Schlüssels.

Übung 12: SFTP-Dateiübertragung durchführen

Lernziel: Du überträgst eine Datei verschlüsselt über SSH.

Voraussetzungen: Übung 11.

Benötigte Software: OpenSSH mit sftp.

Schritte: Öffne eine SFTP-Sitzung, lade eine Datei hoch und lade sie zurück.

Befehle: `sftp benutzer@192.168.1.50`, darin `put test.txt` und `get test.txt`

Erklärung: SFTP läuft über den einen SSH-Kanal auf Port 22. Es ist keine zweite Datenverbindung nötig.

Typische Fehler: Ein Schreibrecht auf dem Zielordner fehlt oft. Dann meldet sftp Permission denied.

Musterlösung: Du zeigst Upload und Download mit gleichem Inhalt.

16.5 Zeit

Übung 13: NTP-Zeitsynchronisation prüfen

Lernziel: Du prüfst, ob dein Rechner synchronisiert ist, und erkennst den Offset.

Voraussetzungen: Internetzugang oder ein Zeitserver im Netz.

Benötigte Software: w32tm unter Windows, chronyc unter Linux.

Schritte: Lies Quelle, Stratum und Offset aus dem Status.

Befehle: `w32tm /query /status` oder `chronyc tracking`

Beispielausgabe (gekürzt)

```
Reference ID  : 0A0B0C0D (ptbtime1.ptb.de)
Stratum       : 2
System time   : 0.000012 seconds fast of NTP time
```

Erklärung: Stratum 2 bedeutet, dass der Server einen Schritt von der Referenz entfernt ist. Der kleine Offset zeigt eine gute Synchronisation.

Typische Fehler: Ein blockierter UDP-Port 123 verhindert die Synchronisation, die Uhr driftet dann.

Musterlösung: Du nennst Quelle, Stratum und Offset und bewertest die Synchronisation.

16.6 SMB

Übung 14: SMB-Freigabe verbinden

Lernziel: Du verbindest dich mit einer Netzwerkfreigabe.

Voraussetzungen: Ein Server mit Freigabe im Testnetz, Zugangsdaten.

Benötigte Software: Windows oder Samba-Client unter Linux.

Schritte: Liste die Freigaben, verbinde dich und prüfe den Zugriff.

Befehle: `net view \\server` und `net use Z: \\server\Projekte`, unter Linux `smbclient -L //server -U benutzer`

Erklärung: Die Freigabe läuft über TCP auf Port 445. Die Berechtigungen von Freigabe und Dateisystem wirken zusammen.

Typische Fehler: Ein Zugriff, der trotz gültiger Anmeldung scheitert, liegt oft an der Freigabeberechtigung.

Musterlösung: Du listest die Freigabe auf, verbindest das Laufwerk und zeigst den Zugriff.

16.7 Vergleich

Übung 15: Netzwerkprotokolle mit Wireshark vergleichen

Lernziel: Du siehst im selben Mitschnitt, wie sich HTTP und HTTPS unterscheiden.

Voraussetzungen: Lokaler Webserver aus Übung 7, Wireshark.

Benötigte Software: Wireshark, curl.

Schritte: Starte eine Aufzeichnung auf der Loopback-Schnittstelle, rufe einen lokalen HTTP-Dienst auf und vergleiche den Verkehr mit einem HTTPS-Aufruf.

Befehle: `curl http://127.0.0.1:8000/` und Filter `http`, danach `curl https://www.example.org/` und Filter `tls`

Erklärung: Bei HTTP erscheinen Methode, Pfad und Header im Klartext. Bei HTTPS siehst du den Handshake und verschlüsselte Daten.

Typische Fehler: Ohne Filter findest du dich im Mitschnitt schwer zurecht. Die Loopback-Schnittstelle muss gewählt sein.

Musterlösung: Du nennst für beide Verbindungen, welche Angaben sichtbar sind und welche verborgen bleiben.

> **Merke**
>
> Die Übungen zeigen, dass jedes Protokoll einen erkennbaren Ablauf hat. Wer Befehl, Ausgabe und Mitschnitt zusammenbringt, versteht das Protokoll und findet

> Fehler schneller.

KAPITEL 17

Troubleshooting und Fehlersuche

Netzwerkfehler sehen von außen oft gleich aus. Eine Webseite lädt nicht, ein Drucker ist verschwunden, eine Mail kommt nicht an. Die Ursache kann bei jedem der Protokolle dieses Bandes liegen. Dieses Kapitel gibt dir ein Vorgehen, mit dem du die Ursache systematisch eingrenzt, und zeigt für die häufigsten Störungen typische Ursachen, Diagnosebefehle und Lösungen. Die Grundregel aus Band 1 gilt weiterhin: von unten nach oben prüfen, ob die Verbindung steht, bevor man den Dienst selbst verdächtigt.

17.1 Systematisches Vorgehen

Ein bewährter Ablauf beginnt mit der Frage, ob das Gerät überhaupt eine Verbindung hat. Danach prüfst du die Adresskonfiguration, die Erreichbarkeit des Gateways, die Namensauflösung und zuletzt den Dienst selbst. Jeder Schritt schließt eine Gruppe von Ursachen aus. Die folgende Abbildung zeigt den Ablauf als Entscheidungsdiagramm.

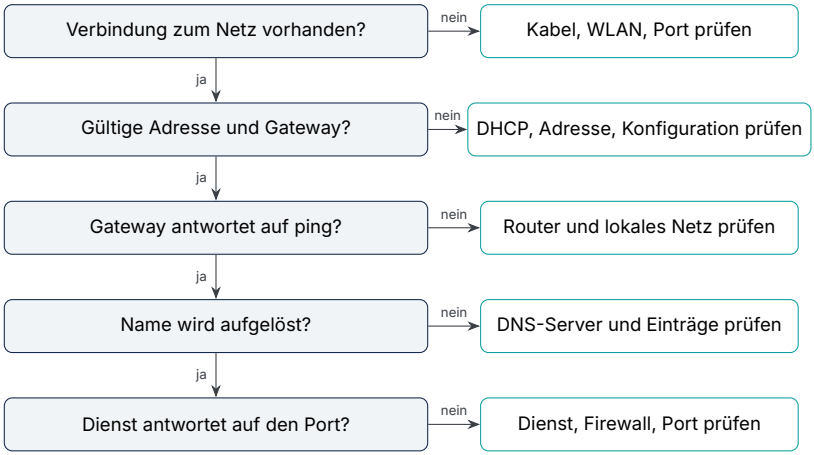


Abbildung 17.1. Entscheidungsdiagramm für die systematische Fehlersuche. Bei jeder Frage führt Ja nach unten, Nein zu einer Prüfung auf der jeweiligen Schicht. Schematische Darstellung.

Wichtig ist, nach jeder Änderung erneut zu prüfen. Nur so lässt sich sagen, welche Maßnahme geholfen hat. Mehrere Änderungen gleichzeitig verschleiern die Ursache.

17.2 DNS funktioniert nicht

Die Symptome sind eine fehlende Namensauflösung, obwohl die Adresse per ping erreichbar ist. Typische Ursachen sind ein falscher DNS-Server in der Konfiguration, ein veralteter Cache, ein fehlender Eintrag oder ein blockierter UDP- und TCP-Port 53. Diagnose: nslookup oder dig gegen den konfigurierten Server, dann gegen einen öffentlichen Resolver zum Vergleich. Lösung: DNS-Server korrigieren, Cache leeren mit ipconfig /flushdns, fehlenden Record anlegen.

17.3 DHCP vergibt keine Adresse

Der Client zeigt eine APIPA-Adresse aus 169.254.0.0/16. Typische Ursachen sind ein erschöpfter Pool, ein falscher Scope, ein Relay ohne richtige Konfiguration, ein nicht erreichbarer Server oder ein Switch-Port im falschen VLAN. Diagnose: ipconfig /all, ein Mitschnitt des DORA-Ablaufs mit Wireshark, Prüfung der Poolgröße am Server. Lösung: Pool erweitern, Relay korrigieren, Port zuordnen, bei Bedarf ipconfig /release und ipconfig /renew.

17.4 Webseite nicht erreichbar

Hier lohnt die Unterscheidung nach dem Statuscode. Keine Antwort deutet auf Verbindung, DNS oder Firewall hin. Ein 404 zeigt eine falsche Adresse, ein 500 einen Serverfehler, ein 502 oder 503 einen Proxy oder Backend ohne Antwort. Diagnose: curl -v gegen die Adresse, dann gegen die IP-Adresse, um DNS auszuschließen. Lösung je nach Ursache: Verbindung, Namen oder Dienst korrigieren.

17.5 HTTPS-Zertifikatsfehler

Der Browser meldet ein ungültiges Zertifikat. Typische Ursachen sind ein abgelaufenes Zertifikat, ein Name, der nicht passt, eine fehlende Zwischenzertifikatsangabe, eine unbekannte Root-CA oder eine falsche Systemzeit. Diagnose: openssl s_client mit -servername, Prüfung der Gültigkeit mit openssl x509 und der Vergleich der Systemzeit. Lösung: Zertifikat erneuern, Kette vollständig ausliefern, Zeit synchronisieren. Eine War-

nung umgehen darf man nur bewusst und nicht auf Produktivsystemen.

17.6 SSH-Verbindung verweigert

Die Meldung Connection refused bedeutet, dass auf dem Port kein SSH-Dienst lauscht. Connection timed out deutet auf eine Firewall. Permission denied zeigt, dass die Anmeldung scheitert. Ursachen sind ein gestoppter sshd, eine Firewall, ein falscher Benutzer, ein falscher Schlüssel, zu lockere Dateirechte oder ein Verbot der Anmeldung in der Konfiguration. Diagnose: Port prüfen, ssh -v für den Ablauf, auf dem Server die Logs. Lösung: Dienst starten, Firewall öffnen, Schlüssel oder Rechte korrigieren.

17.7 E-Mail-Zustellung fehlgeschlagen

Ein Absender erhält eine Unzustellbarkeitsmeldung. Typische Ursachen sind ein fehlender oder falscher MX-Record, ein nicht erreichbarer Empfängerserver, fehlgeschlagene SPF-, DKIM- oder DMARC-Prüfungen oder ein Spam-Filter. Diagnose: dig MX der Domain, Prüfung der Erreichbarkeit auf Port 25, Auswertung der Kopfzeilen mit den Ergebnissen der Prüfungen. Lösung: MX korrigieren, Prüfeinträge ergänzen, Absenderserver konfigurieren.

17.8 SMB-Freigabe nicht erreichbar

Die Freigabe lässt sich nicht öffnen. Ursachen sind eine blockierte Verbindung auf Port 445, eine falsche Freigabeberechtigung, falsche Zugangsdaten, eine veraltete Version oder ein Name, der nicht aufgelöst wird. Diagnose: Test-NetConnection -Port 445, net view, smbclient -L, Prüfung der Berechtigungen. Lösung: Firewall korrigieren, Berechtigung nachtragen, Version prüfen, Namen auflösen.

17.9 NTP synchronisiert nicht

Die Uhr driftet, Zertifikate und Anmeldungen schlagen fehl. Ursachen sind ein blockierter UDP-Port 123, ein falscher Zeitserver, eine Virtualisierung ohne Zeitsynchronisation des Gastes oder ein Stratum-Konflikt. Diagnose: w32tm /query /status oder chronyc tracking. Lösung: Port freigeben, Zeitserver korrigieren, Zeitsynchronisation des Hypervisors prüfen.

17.10 Firewall blockiert einen Dienst

Der Dienst läuft, ist aber nicht erreichbar. Ein Test auf dem Server zeigt, dass er lauscht, der Test von außen scheitert. Diagnose: Portprüfung von beiden Seiten, Liste der Firewall-Regeln, Mitschnitt auf beiden Seiten. Lösung: Regel für Port und Quelle anlegen, möglichst eng begrenzt auf bekannte Netze.

17.11 Falsche Portnummer

Der Client verbindet sich mit dem falschen Port, etwa 80 statt 443, oder der Dienst läuft auf einem anderen Port als dem Standard. Die Verbindung wird verweigert oder timeout. Diagnose: Prüfung des Ports mit Test-NetConnection oder nc, Abgleich mit der Dokumentation des Dienstes. Lösung: richtigen Port eintragen, bei Bedarf Konfiguration des Dienstes anpassen.

17.12 Fehler-Ursache-Lösung-Übersicht

Die folgende Tabelle fasst die häufigsten Fälle zusammen.

> **Merke**
>
> Ein Fehler zeigt sich dort, wo die Kette abbricht. Die Reihenfolge Verbindung, Adresse, Gateway, Name, Dienst grenzt die Ursache ein. Jede Maßnahme wird geprüft, bevor die nächste folgt.

Kontrollfragen

1. Ein Client zeigt 169.254.12.7. Welche Schicht ist betroffen, und welcher Dienst fehlt?
2. Eine Verbindung endet mit Connection refused. Was schließt du daraus?
3. Die Ping-Antwort auf die IP-Adresse funktioniert, der Name nicht. Wo suchst du?
4. Welche Reihenfolge wählst du bei einer Störung, die nach einem Umzug auftritt?

Musterlösungen: 1. Die Adresskonfiguration ist betroffen, der DHCP-Dienst hat keine Adresse geliefert. 2. Auf dem Zielport lauscht kein Dienst, der Server antwortet mit

Symptom	Ursache	Lösung
Namen nicht auflösbar	falscher DNS-Server	DNS-Server korrigieren
APIPA-Adresse	kein DHCP-Angebot	Pool, Relay, Port prüfen
Zeitüberschreitung	Firewall blockiert	Regel anlegen
Connection refused	Dienst lauscht nicht	Dienst starten
Zertifikatsfehler	Name oder Ablauf falsch	Zertifikat erneuern
Mail nicht zugestellt	MX oder SPF falsch	Eintrag korrigieren
SMB nicht erreichbar	Port 445 gesperrt	Freigabe und Firewall prüfen
Uhr driftet	NTP-Port gesperrt	Port 123 öffnen

Tabelle 17.1. Häufige Störungen mit Ursache und Lösung. Die Zuordnung ist ein Ausgangspunkt, die Diagnose entscheidet.

RST. 3. Bei der Namensauflösung, also im DNS-Server oder im Cache. 4. Verbindung, Adresse, Gateway, Name, Dienst, weil jede Stufe die nächste voraussetzt.

KAPITEL 18

Wissensüberprüfung und Prüfungsvorbereitung

Dieses Kapitel fasst die Protokolle des Bandes zusammen und bietet Fragen von der Einstiegsstufe bis zur fortgeschrittenen Analyse. Versuche die Aufgaben zuerst ohne Blick in die Lösungen. Die Lösungen am Ende sind knapp gehalten, damit du sie gut vergleichen kannst.

18.1 Zusammenfassung der Protokolle

DNS übersetzt Namen in Informationen über eine Hierarchie aus Root-, TLD- und autoritativen Servern. DHCP vergibt Netzkonfigurationen über den DORA-Ablauf für eine begrenzte Zeit. HTTP folgt dem Muster Anfrage und Antwort mit Methoden und Statuscodes. HTTPS legt HTTP in eine TLS-Verbindung, Zertifikate binden Schlüssel an Namen. FTP trennt Steuerung und Daten, FTPS ergänzt TLS, SFTP läuft über SSH. SSH verschlüsselt eine Sitzung, prüft den Server und meldet Benutzer per Schlüssel an. E-Mail nutzt SMTP für den Versand, POP3 oder IMAP für den Abruf, und SPF, DKIM und DMARC für die Prüfung. NTP synchronisiert Uhren über Stratum-Ebenen. SNMP überwacht Geräte über eine MIB. SMB stellt Freigaben bereit. LDAP fragt Verzeichnisse ab. Die Kette aus Name, Adresse, Verbindung, Verschlüsselung und Anwendung verbindet alles.

18.2 Multiple-Choice

1. Welcher Record verbindet einen Namen mit einer IPv4-Adresse?
   a) MX b) A c) TXT d) SOA
2. Welche Methode ist sicher und idempotent?
   a) POST b) GET c) PATCH d) CONNECT
3. Welcher Statuscode meldet eine dauerhafte Umleitung?
   a) 200 b) 302 c) 301 d) 404
4. Welcher Port ist der Standard für HTTPS?
   a) 80 b) 443 c) 22 d) 25
5. In welcher DHCP-Option steht der Nachrichtentyp, etwa Discover?

a) Option 6 b) Option 53 c) Option 3 d) Option 15

6. Welche Version von TLS ist heute aktuell?
   a) TLS 1.0 b) TLS 1.1 c) TLS 1.2 d) TLS 1.3
7. Welcher Port wird für SMTP-Versand vom Mailprogramm genutzt?
   a) 25 b) 110 c) 587 d) 143
8. Welches Protokoll holt E-Mails und lässt sie auf dem Server?
   a) POP3 b) IMAP c) SMTP d) SFTP
9. Welche Stratum-Stufe hat ein Server direkt an einer Referenzquelle?
   a) 0 b) 1 c) 2 d) 16
10. Welches Protokoll setzt auf Port 445 auf?
    a) SMB b) SNMP c) LDAP d) NTP
11. Mit welchem Befehl fragt man ein LDAP-Verzeichnis ab?
    a) ldapsearch b) snmpget c) tracert d) nslookup
12. Welcher Mechanismus schützt E-Mail gegen gefälschte Absender im Umschlag?
    a) SPF b) MIME c) POP3 d) NTP
13. Welche Angabe steht im ClientHello und nennt den Servernamen?
    a) SNI b) ALPN c) PTR d) CAA
14. Welches Verfahren verhindert, dass alte Sitzungsschlüssel spätere Verbindungen offenlegen?
    a) Perfect Forward Secrecy b) NAT c) DHCP-Snooping d) Trap
15. Welcher OID-Zweig gehört zu den Systemdaten der MIB-2?
    a) 1.3.6.1.2.1.1 b) 1.3.6.1.2.1.2 c) 1.3.6.1.4 d) 1.3.6.1.6
16. Welche Ports nutzt SSH standardmäßig?
    a) 21 b) 22 c) 23 d) 53
17. Welches Protokoll läuft über UDP und dient dem Start von Geräten ohne Anmeldung?
    a) TFTP b) FTP c) SFTP d) SSH
18. Welches Feld im TLS-Handshake nennt, welches Anwendungsprotokoll gewünscht ist?
    a) SNI b) ALPN c) SAN d) CAA
19. Welcher Record legt die erlaubten Zertifizierungsstellen fest?
    a) TXT b) CAA c) SRV d) MX
20. Welcher Port nutzt LDAPS?
    a) 389 b) 636 c) 443 d) 993
21. Welches Protokoll meldet Qualitätsdaten einer RTP-Übertragung?
    a) RTCP b) SIP c) MQTT d) SSDP

22. Welche Fehlerklasse steht für Probleme auf Seiten des Servers?
    a) 2xx b) 3xx c) 4xx d) 5xx
23. Welcher Befehl zeigt Header und Statuszeile einer Webseite?
    a) curl -I b) ping c) tracert d) arp -a

18.3 Zuordnungsaufgaben

Ordne jedem Protokoll den passenden Port zu.

24. DNS
25. DHCP Server
26. SSH
27. NTP
28. SMTP für den Versand zwischen Servern
29. IMAP über TLS
30. LDAP
31. SMB
32. HTTPS
33. SNMP-Traps

Ordne jedem Begriff die Erklärung zu.

34. DORA
35. Stratum
36. Lease
37. Host Key
38. MX-Record
39. MIB
40. Duplicate ACK
41. Distinguished Name
42. Connection refused
43. Relay

18.4 Praxisaufgaben

44. Frage den A-Record und den MX-Record einer Domain ab und nenne den zuständigen Mailserver.
45. Zeige mit curl die Statuszeile einer Webseite und erkläre den Code.
46. Lies mit ipconfig /all die Adresse, das Gateway und den DNS-Server deines Rechners aus.
47. Prüfe mit Test-NetConnection, ob Port 443 einer Webseite erreichbar ist.
48. Erstelle einen SSH-Schlüssel vom Typ ed25519 und erkläre, welche Datei öffentlich ist.

18.5 Protokollanalyse und Fehlerdiagnose

49. Ein Mitschnitt zeigt eine DNS-Antwort mit Fehlercode. Welche Schicht ist betroffen?
50. Ein Mitschnitt zeigt ein ClientHello mit SNI und danach keine Antwort. Welche Ursache ist plausibel?
51. Ein Client zeigt 169.254.8.9. Welcher Ablauf ist gescheitert?
52. Ein Server antwortet mit 503. Welche Stelle ist wahrscheinlich betroffen?
53. Eine Mail bleibt liegen, der Empfänger meldet fehlgeschlagene SPF-Prüfung. Was prüfst du zuerst?

18.6 Vollständige Lösungen

Multiple-Choice

1 b, 2 b, 3 c, 4 b, 5 b, 6 d, 7 c, 8 b, 9 b, 10 a, 11 a, 12 a, 13 a, 14 a, 15 a, 16 b, 17 a, 18 b, 19 b, 20 b, 21 a, 22 d, 23 a.

Zuordnung

Die Ports lauten 24 bis 33: DNS 53, DHCP Server 67, SSH 22, NTP 123, SMTP Server-Austausch 25, IMAP über TLS 993, LDAP 389, SMB 445, HTTPS 443, SNMP-Trap 162.

Die Erklärungen lauten 34 bis 43: DORA ist der Ablauf Discover, Offer, Request, Acknowledge. Stratum gibt den Abstand zur Referenzquelle an. Lease ist die begrenzte

Zeit einer Adressvergabe. Host Key beweist die Identität eines SSH-Servers. MX-Record nennt den Mailserver einer Domain. MIB beschreibt die Werte eines Geräts. Duplicate ACK zeigt eine Lücke im Datenstrom an. Distinguished Name ist der eindeutige Weg zu einem Verzeichniseintrag. Connection refused bedeutet, dass auf dem Port kein Dienst lauscht. Relay ist ein Server, der Nachrichten für andere weiterleitet.

Praxisaufgaben

Die Ergebnisse hängen vom System ab. Ein A-Record liefert eine IPv4-Adresse, der MX-Record nennt den Mailserver mit Priorität. Eine Statuszeile wie HTTP/2 200 zeigt Erfolg, 404 eine fehlende Ressource. Die Adressen aus ipconfig /all müssen zum eigenen Netz passen. Test-NetConnection meldet bei erreichbarem Port True. Die öffentliche Datei endet auf .pub.

Analyse und Diagnose

Die Aufgaben 49 bis 53 lauten: 49 betrifft die Namensauflösung über DNS auf der Anwendungsschicht. 50 bedeutet, dass der Server nach dem ClientHello nicht antwortet, also keine Verbindung oder kein Zertifikat zustande kommt. 51 zeigt, dass der DHCP-Ablauf gescheitert ist. 52 meint einen Dienst oder ein Backend, das nicht antwortet. 53 bedeutet, zuerst den SPF-Eintrag im DNS und die sendende Adresse zu prüfen.

Anhang

Der Anhang bündelt die Nachschlagewerte dieses Bandes. Die Tabellen sind kurz gehalten, die Erklärungen stehen in den Kapiteln. Nutze den Anhang beim Üben und in der Praxis, um Ports, Records und Befehle schnell zu finden.

Große Protokollübersicht

Protokoll	Transport	Port	Aufgabe
DNS	UDP, TCP	53	Namen in Adressen und Informationen übersetzen
DHCP	UDP	67, 68	IP-Konfiguration vergeben
DHCPv6	UDP	546, 547	IPv6-Konfiguration vergeben
HTTP	TCP	80	Webseiten übertragen
HTTPS	TCP, UDP	443	HTTP über TLS, bei QUIC über UDP
FTP	TCP	21, 20	Dateien übertragen, Klartext
FTPS	TCP	21, 990	FTP über TLS
SFTP	TCP	22	Dateien über SSH
SSH	TCP	22	verschlüsselte Fernanmeldung
SMTP	TCP	25, 587, 465	E-Mail versenden
POP3	TCP	110, 995	E-Mail abrufen, lokal
IMAP	TCP	143, 993	E-Mail abrufen, auf dem Server
NTP	UDP	123	Uhren synchronisieren
SNMP	UDP	161, 162	Geräte überwachen

Protokoll	Transport	Port	Aufgabe
SMB	TCP	445	Netzwerkfreigaben
LDAP	TCP	389, 636	Verzeichnisdienst
RDP	TCP, UDP	3389	grafischer Fernzugriff
Telnet	TCP	23	Fernanmeldung, unverschlüsselt
TFTP	UDP	69	einfacher Dateitransfer ohne Anmeldung
SIP	TCP, UDP	5060, 5061	Telefonie-Signalisierung
RTP und RTCP	UDP	dynamisch	Medien und Qualitätsdaten
MQTT	TCP	1883, 8883	Nachrichten für Geräte
WebSocket	TCP	80, 443	dauerhafter Datenaustausch
mDNS	UDP	5353	lokale Namen
SSDP	UDP	1900	Gerätesuche, UPnP

TCP- und UDP-Porttabelle

Port	Transport	Dienst
20, 21	TCP	FTP, Daten und Steuerung
22	TCP	SSH und SFTP
23	TCP	Telnet
25	TCP	SMTP zwischen Servern
53	TCP, UDP	DNS
67, 68	UDP	DHCP
69	UDP	TFTP
80	TCP	HTTP
110	TCP	POP3
123	UDP	NTP
143	TCP	IMAP
161, 162	UDP	SNMP und Traps
389	TCP, UDP	LDAP

Port	Transport	Dienst
443	TCP, UDP	HTTPS und QUIC
445	TCP	SMB
465	TCP	SMTPS
587	TCP	SMTP Submission
636	TCP	LDAPS
993	TCP	IMAP über TLS
995	TCP	POP3 über TLS
1883, 8883	TCP	MQTT ohne und mit TLS
3389	TCP, UDP	RDP

DNS-Record-Übersicht

Typ	Zweck	Hinweis
A	IPv4-Adresse	Name zu IPv4
AAAA	IPv6-Adresse	Name zu IPv6
CNAME	Alias	nicht an der Zonenwurzel
MX	Mailserver	Priorität, niedrige Zahl zuerst
NS	Nameserver einer Zone	Delegation
PTR	Name zu einer Adresse	Reverse-Zone
TXT	freier Text	SPF, DKIM, DMARC
SOA	Zone beschreiben	Seriennummer, Zeitwerte
SRV	Dienst mit Port	Priorität, Gewicht, Port
CAA	erlaubte CAs	Schutz vor Fehlzertifikaten

HTTP-Statuscode-Tabelle

Code	Bedeutung
100	Continue, Anfrage läuft weiter

Code	Bedeutung
200	OK, erfolgreich
201	Created, neue Ressource angelegt
204	No Content, erfolgreich ohne Körper
301	Moved Permanently, dauerhaft umgeleitet
302	Found, vorübergehend umgeleitet
304	Not Modified, Cache ist gültig
400	Bad Request, fehlerhafte Anfrage
401	Unauthorized, Anmeldung fehlt oder ist falsch
403	Forbidden, Zugriff verweigert
404	Not Found, Ressource unbekannt
500	Internal Server Error, Fehler im Server
502	Bad Gateway, Backend antwortet nicht gültig
503	Service Unavailable, vorübergehend nicht verfügbar

Wichtige Windows-Befehle

Befehl	Zweck
ipconfig /all	Konfiguration, DHCP, DNS
ipconfig /release und /renew	Adresse neu anfordern
ipconfig /flushdns	DNS-Cache leeren
ipconfig /displaydns	DNS-Cache anzeigen
nslookup name	Namen auflösen
nslookup -type=MX domain	Mailserver abfragen
Resolve-DnsName name	DNS in PowerShell
netstat -ano	Verbindungen mit Prozessen
Get-NetTCPConnection	TCP-Verbindungen in PowerShell
Test-NetConnection -Port	Portprüfung

Befehl	Zweck
route print	Routingtabelle
w32tm /query /status	Zeitstatus
net view und net use	Freigaben und Laufwerke

Wichtige Linux-Befehle

Befehl	Zweck
ip addr und ip route	Adressen und Routen
ss -tulpen und ss -tan	Ports und Verbindungen
dig name A, AAAA, MX	DNS-Abfragen
host name	einfache Namensauflösung
curl -I und curl -v	HTTP und TLS
openssl s_client -connect host:443	Zertifikat prüfen
ssh und ssh-keygen	Anmeldung und Schlüssel
sftp benutzer@host	Dateien über SSH
chronyc tracking und timedatectl	Zeitstatus
smbclient -L //host	Freigaben anzeigen
snmpwalk -v3	Geräte abfragen, mit SNMPv3
ldapsearch -x -H ldap://host	Verzeichnis abfragen
tcpdump -i any port 53	Verkehr mitschneiden

Wireshark-Filter-Spickblatt

Filter	Zeigt
dns	DNS-Nachrichten
dns.flags.response == 1	DNS-Antworten
dhcp	DHCP-Nachrichten
dhcp.option.dhcp == 1	DHCP-Discover
http	HTTP-Nachrichten
http.response.code == 404	Antworten mit 404
tls	TLS-Nachrichten
tls.handshake.type == 1	ClientHello
ssh	SSH-Nachrichten
ntp	NTP-Nachrichten
tcp.port == 443	TCP-Verkehr mit Port 443
udp.port == 53	UDP-Verkehr mit Port 53
tcp.flags.syn == 1	Verbindungsaufbau
tcp.analysis.retransmission	Wiederholungen

Glossar

Begriff	Erklärung
Anwendungsprotokoll	Regeln für die Kommunikation von Programmen, etwa HTTP oder DNS.
Authentifizierung	Prüfung der Identität einer Person oder eines Geräts.
Autorisierung	Festlegung, was eine identifizierte Person darf.
Bind	Anmeldung an einem Verzeichnisserver.
Cache	Zwischenspeicher für Antworten, etwa bei DNS oder HTTP.
Certificate Authority	Stelle, die Zertifikate ausstellt und Vertrauen herstellt.
Cookie	kleines Datenstück, das der Server dem Client mitgibt.
DORA	Ablauf der DHCP-Vergabe: Discover, Offer, Request, Acknowledge.
Head-of-Line Blocking	Verzögerung nachfolgender Daten durch ein verlorenes Paket.
Host Key	Schlüssel, mit dem sich ein SSH-Server ausweist.
Idempotent	mehrfache Ausführung hat dasselbe Ergebnis wie einmalige.

Begriff	Erklärung
Lease	begrenzte Zeit einer DHCP-Adressvergabe.
MIB	Beschreibung der Werte, die ein Gerät für SNMP bereitstellt.
MX-Record	DNS-Eintrag, der den Mailserver einer Domain nennt.
OID	Adresse eines Werts im MIB-Baum.
Perfect Forward Secrecy	Schutz alter Sitzungen durch kurzlebige Schlüssel.
Relay	Server, der Nachrichten für andere weiterleitet.
Resolver	Dienst, der DNS-Anfragen für Clients auflöst.
Session	serverseitig gespeicherter Zustand eines Benutzers.
SNI	Angabe des Servernamens im TLS-ClientHello.
Stratum	Abstand eines Zeitservers zur Referenzquelle.
TTL	Gültigkeit eines DNS-Eintrags in Sekunden.
Zone	Teil des DNS-Namensraums, für den ein Server zuständig ist.

Abkürzungsverzeichnis

Abkürzung	Bedeutung
ALPN	Application-Layer Protocol Negotiation
APIPA	Automatic Private IP Addressing
CAA	Certification Authority Authorization
DHCP	Dynamic Host Configuration Protocol
DKIM	DomainKeys Identified Mail
DMARC	Domain-based Message Authentication, Reporting and Conformance
DNS	Domain Name System
DoH	DNS over HTTPS
DoT	DNS over TLS
DORA	Discover, Offer, Request, Acknowledge
FTP	File Transfer Protocol
HTTP	Hypertext Transfer Protocol
IMAP	Internet Message Access Protocol
LDAP	Lightweight Directory Access Protocol
MIB	Management Information Base
MIME	Multipurpose Internet Mail Extensions

Abkürzung	Bedeutung
MTA, MUA, MDA	Mail Transfer, User und Delivery Agent
MQTT	Message Queuing Telemetry Transport
NTP	Network Time Protocol
OID	Object Identifier
PKI	Public Key Infrastructure
POP3	Post Office Protocol Version 3
RDP	Remote Desktop Protocol
RTCP und RTP	RTP Control Protocol und Real-time Transport Protocol
SFTP	SSH File Transfer Protocol
SIP	Session Initiation Protocol
SMB	Server Message Block
SMTP	Simple Mail Transfer Protocol
SNMP	Simple Network Management Protocol
SPF	Sender Policy Framework
SSH	Secure Shell
SSDP	Simple Service Discovery Protocol
TLS	Transport Layer Security
URI und URL	Uniform Resource Identifier und Locator

Literatur- und Quellenverzeichnis

- RFC 1034 und RFC 1035: Domain Names, Konzepte und Spezifikation.
- RFC 2131: Dynamic Host Configuration Protocol.
- RFC 3261: Session Initiation Protocol.
- RFC 3550: RTP, Real-time Transport Protocol.
- RFC 3411: An Architecture for Describing SNMP Management Frameworks.
- RFC 4120: The Kerberos Network Authentication Service (V5).
- RFC 4251: The Secure Shell (SSH) Protocol Architecture.
- RFC 4511: Lightweight Directory Access Protocol (LDAP): The Protocol.
- RFC 5321: Simple Mail Transfer Protocol.
- RFC 5905: Network Time Protocol Version 4.
- RFC 6376: DomainKeys Identified Mail (DKIM) Signatures.
- RFC 6455: The WebSocket Protocol.
- RFC 6762: Multicast DNS.

- RFC 7208: Sender Policy Framework (SPF).
- RFC 7489: Domain-based Message Authentication, Reporting, and Conformance (DMARC).
- RFC 8415: Dynamic Host Configuration Protocol for IPv6 (DHCPv6).
- RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3.
- RFC 8915: Network Time Security for the Network Time Protocol.
- RFC 9051: Internet Message Access Protocol (IMAP) Version 4rev2.
- RFC 9110: HTTP Semantics. RFC 9112: HTTP/1.1. RFC 9113: HTTP/2. RFC 9114: HTTP/3.
- RFC 959: File Transfer Protocol. RFC 4217: Securing FTP with TLS.
- OpenSSH-Dokumentation, Samba-Dokumentation, Wireshark-Dokumentation und Microsoft Learn.
- Mozilla Developer Network (MDN) für HTTP und Browserverhalten.
- IANA Port Number Registry für die Zuordnung der Ports.

Weiterführende Lernressourcen

Die RFCs sind bei der IETF frei verfügbar und bilden die verbindliche Grundlage. Für die Praxis lohnen die Handbuchseiten der Linux-Werkzeuge und die Dokumentation von Microsoft zu den Windows-Diensten. Die Wireshark-Dokumentation erklärt Filter und Protokolle im Detail. Wer sich für Verschlüsselung interessiert, findet bei der IETF und bei den Browserherstellern gute Erläuterungen zu TLS und Zertifikaten.

NETZWERK VERSTEHEN • BAND 4

Netzwerkprotokolle in der Praxis

DNS, DHCP, HTTP, HTTPS, SSH, E-Mail und NTP bestimmen, was im Netz täglich geschieht. Dieser Band erklärt, wie diese Dienste ihre Nachrichten austauschen, auf welchen Transportprotokollen sie laufen und welche Ports sie nutzen.

Das Buch richtet sich an Einsteiger, Quereinsteiger, Umschüler, Auszubildende und angehende Netzwerkadministratoren. Jedes Protokoll wird nach einem festen Muster erklärt, mit Ablaufdiagrammen, Vergleichen und Übungen.

Du lernst, Namensauflösung und Adressvergabe nachzuvollziehen, Webverkehr und Verschlüsselung zu prüfen, Zertifikate und Anmeldungen zu verstehen und Dienste mit Befehlen und Wireshark zu untersuchen. Fünfzehn Laborübungen für Windows und Linux helfen dabei, das Gelernte am eigenen System zu erproben.

Wer die Dienste versteht, findet Fehler schneller und erkennt, ob ein Problem bei der Adresse, beim Namen, bei der Verbindung oder beim Dienst selbst liegt.

Band 4 der Reihe *Netzwerk verstehen* baut auf den Bänden 1 bis 3 auf. Er zeigt, wie Anwendungen Netzwerkprotokolle nutzen, die auf IP, TCP und UDP aufsetzen.

Eugen Becker